

Exqutor 관련 References 24편 상세 정리

범위: References [1]~[24] 중 24편을 상세 분석한 문서이다.

각 논문의 문제의식, 핵심 기여, 시스템 구조, 실험 결과, 본 논문과의 관계를 정리하였다.

(A) VAQ 동기 부여

[1] AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data 총정리

저자: Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, Yuanzhe Cai (Alibaba Group)

학회/년도: VLDB 2020 (Proceedings of the VLDB Endowment, Vol. 13, No. 12)

분량: 약 14페이지

역할군: (A) VAQ 개념 원조; 본 논문 문제 정의의 출발점

요약

AnalyticDB-V는 Exqutor가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 최초로 산업 규모에서 구현한 시스템이다. 현대 데이터 파이프라인에서 구조화 데이터(테이블)와 비구조화 데이터(이미지, 텍스트 등)가 함께 존재하는데, 기존에는 이 두 종류를 별개의 시스템으로 처리해야 했다. AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자로 구현**하는 것으로, 사용자가 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있도록 한다.

시스템은 Lambda 아키텍처를 채택하여 스트리밍 레이어와 배치 레이어를 분리하고, VGPQ(Vector Graph Product Quantization)라는 새로운 ANN 검색 알고리즘을 제안한다. 가장 중요한 기여는 **정확도 인식 비용 기반 최적화(Accuracy-Aware Cost-Based Optimization)**로, ANN 검색의 본질적 근사성을 옵티마이저에 통합하여 정확도 요구사항과 비용 간의 트레이드오프를 관리한다. 다만 **카디널리티 추정** 문제를 깊이 다루지 않았으며, 이것이 Exqutor가 보완하는 핵심 빈틈이다.

본 논문과의 관계: VAQ(Vector-Augmented Analytical Query) 개념의 원형을 제시했으나, "벡터 검색 결과가 정확히 몇 건이나 나올지 추정하는 문제"는 미해결로 남겨두어, Exqutor의 근본적 동기를 제공한다.

상세분석

1.1 해결하고자 하는 문제

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다:

- 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등)
- 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어:

- "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100만원 이하이고 재고가 있는 것만 보여줘"
- "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸으며, 최적화 기회를 완전히 놓치게 된다.

1.2 핵심 아이디어: SQL로 하이브리드 쿼리 표현

AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자(Physical Operator)로 구현**하는 것이다. 사용자는 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있다.

```
SELECT product_name, price, similarity_score
FROM products
WHERE category = 'electronics'
      AND ANNS(image_embedding, query_vector, 10) -- 벡터 유사도 top-10
ORDER BY similarity_score DESC;
```

이렇게 하면 데이터베이스 옵티마이저가 벡터 검색과 필터링의 순서, 방식 등을 자동으로 결정해준다. 개발자는 "무엇을" 원하는지만 선언하면, 시스템이 "어떻게" 효율적으로 실행할지를 담당한다.

1.3 시스템 구조

AnalyticDB-V는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer):

- 실시간으로 들어오는 벡터 데이터를 처리한다.
- 새로 삽입된 벡터는 즉시 검색 가능하다.
- 인메모리 임시 인덱스를 사용한다.
- 예시; 새 상품 이미지가 업로드되는 순간, 즉시 유사 상품 검색에 포함된다.

배치 레이어(Batching Layer):

- 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다.
- 대규모 데이터에 대해 최적화된 영구 인덱스를 관리한다.
- 야간이나 유휴 시간대에 큰 배치를 처리하여 비용을 절약한다.

쿼리 처리 레이어:

- 스트리밍 레이어와 배치 레이어의 결과를 병합하여 최종 결과를 반환한다.
- 스트리밍 레이어에서 찾은 최근 벡터와 배치 레이어의 오래된 벡터를 함께 고려한다.

1.4 VG PQ (Vector Graph Product Quantization) 알고리즘

AnalyticDB-V의 핵심 기술적 기여 중 하나는 **VG PQ**라는 새로운 ANN 검색 알고리즘이다.

기존의 **IVF-PQ** (Inverted File with Product Quantization) 방식은:

1. 벡터 공간을 여러 클러스터(셀)로 나누고
2. 검색할 때 가까운 클러스터 몇 개만 방문하여 후보를 찾는다.

VG PQ는 여기에 **그래프 구조**를 추가한다:

1. 앵커(anchor) 중심점을 찾고
2. 그래프를 탐색하며 관련 서브셀(subcell)들을 효율적으로 선택하고
3. 후보 벡터들의 거리를 계산하여 결과를 반환한다.

이 방식은 대규모 벡터(수억~수십억 건)에서 기존 IVF-PQ보다 검색 정확도와 속도를 모두 향상시킨다. 특히 고차원 벡터(수백~수천 차원)에서 클러스터링 품질이 향상된다.

1.5 정확도 인식 비용 기반 최적화 (Accuracy-Aware Cost-Based Optimization)

가장 중요한 기여는 **ANNS 연산자를 데이터베이스 옵티마이저에 통합**한 것이다.

일반적인 데이터베이스 옵티마이저는 각 연산의 비용(I/O, CPU 등)을 추정하여 최적의 실행 계획을 선택한다. AnalyticDB-V는 여기에 **정확도(accuracy)**라는 새로운 차원을 추가한다.

ANN 검색은 본질적으로 근사적(approximate)이기 때문에, 파라미터 설정에 따라 정확도가 달라진다:

- 파라미터를 높이면 → 정확도 높아지지만 비용(시간) 증가
- 예; 탐색할 클러스터 개수를 10개에서 100개로 늘리면, 정확도는 99%에서 99.9%로 개선되지만 시간은 2배 증가
- 파라미터를 낮추면 → 비용 줄지만 정확도 저하
- 예; 탐색할 클러스터 개수를 3개로 줄이면, 시간은 80% 단축되지만 정확도는 95%로 저하

옵티마이저는 사용자가 요구하는 정확도 수준을 만족하면서, 가장 비용이 적은 실행 계획을 선택한다. 만약 사용자가 "95% 정확도로 충분해"라고 명시하면, 시스템은 더 빠른 근사 알고리즘을 선택할 수 있다.

1.6 관계형 쿼리와 통합 메커니즘

ANNS 연산자를 관계형 대수에 통합하면서 중요한 문제들을 해결해야 한다:

카디널리티 추정의 어려움:

벡터 검색이 정확히 몇 건의 결과를 반환할 것인지를 미리 알기 어렵다. 예를 들어:

- "가격 < 1000"인 상품을 찾으면 정확히 523개가 나온다 (카디널리티 = 523)
- 하지만 "이 이미지와 유사한 상품"은 몇 개가 나올까? 이는 임계값 설정에 따라 100개~10,000개까지 가능하다.

최적화 기회의 발굴:

정확한 카디널리티가 있으면:

- "벡터 검색을 먼저 할지, 가격 필터를 먼저 할지" 순서를 최적화할 수 있다.
- "중간 결과가 작으면 Nested Loop Join, 크면 Hash Join"을 선택할 수 있다.

1.7 성능 평가

AnalyticDB-V는 아래와 같은 환경에서 평가되었다:

- 벡터 데이터: 수천만 개의 상품 이미지 임베딩(512차원)
- 구조화 데이터: 상품 메타데이터(가격, 카테고리, 재고 등)
- 쿼리 유형: VAQ(벡터 + 필터 + 집계 혼합)

결과:

- 순수 벡터 검색만으로는 최대 50배 성능 향상
- 벡터 + 관계형 쿼리 결합 시 최대 100배 향상
- 다양한 정확도 요구사항(90%~99.9%)에 대응 가능

1.8 본 논문과의 관계

AnalyticDB-V는 "벡터 검색을 SQL에 통합하자"는 아이디어를 제시했지만, **카디널리티 추정** 문제를 깊이 다루지 않았다. 즉, 벡터 검색 결과가 몇 건이나 나올지를 정확히 예측하는 문제는 남아있었다.

AnalyticDB-V에서의 가정:

- 벡터 유사도 범위 검색(distance < D)의 결과는 **고정 비율**(예; 10%)로 추정
- 이 고정 가정은 실제 데이터 분포가 다를 때 매우 부정확함

Exqutor는 바로 이 빈틈을 메우는 연구이다:

- AnalyticDB-V의 "고정 선택도" 가정을 버림
- **실제 인덱스를 탐색**하여 정확한 카디널리티를 얻음
- 이를 통해 옵티마이저가 더 나은 실행 계획을 세우도록 함

1.9 정확한 카디널리티의 가치: 구체적 예시

AnalyticDB-V의 "고정 10% 선택도" 가정이 얼마나 부정확할 수 있는지 보자:

```
쿼리: 상품 이미지 유사도 검색 + 가격 필터
SELECT product_id, price
FROM products
WHERE image_similarity(embedding, query_img) < 0.5
      AND price < 100000
```

AnalyticDB-V 옵티마이저의 추정:

- 총 상품: 1,000,000개
- 유사도 필터: 10% 선택도 가정 → 100,000개
- 가격 필터: 30% 선택도 (알려짐) → 300,000개
- 최종 결과 추정: $100,000 \times 0.3 = 30,000$ 개

하지만 실제 데이터:

- 유사도 필터 실제 결과: 0.5% (5,000개)
→ 고정 가정과 오류: 5배 차이!
- 최종 실제 결과: $5,000 \times 0.3 = 1,500$ 개
→ 추정 오류: 20배!

최적화 영향:

- 30,000건 기준으로 계획 선택 (해시 조인 선택)
- 1,500건이 나올 줄은 몰라서 비효율적 계획 선택
- 추가 비용: 리소스 낭비, 응답시간 증가

추가 제기 문제

1. 일반화의 어려움:

AnalyticDB-V는 Alibaba의 e-commerce 환경에 특화되어 설계되었다. 일반적인 OLAP 환경에서는:

2. 벡터 크기가 더 클 수 있고 (수천 차원)
3. 다양한 벡터 필드가 여러 개 존재하며
4. 복잡한 조인 구조가 필요할 수 있다

이러한 일반화된 환경에서 AnalyticDB-V의 비용 모델이 얼마나 잘 작동하는지는 명확하지 않다. 특히 **복잡한 다중 조인 쿼리**에서 벡터 연산의 위치를 어디에 배치할지 결정하는 문제는 더욱 복잡해진다.

1. 실행 계획의 적응성:

고정된 정확도 임계값(예: 95%)으로 시스템을 설정하면, 쿼리가 달라졌을 때도 같은 임계값을 사용한다. 하지만 어떤 쿼리는 99% 정확도가 필요하고, 다른 쿼리는 90%로도 충분할 수 있다. 이런 동적 조정 메커니즘이 필요하지 않을까?

2. 벡터 인덱스 구축 비용:

VGPO 알고리즘은 검색 성능은 개선하지만, 인덱스 구축 비용도 고려해야 한다. 데이터가 계속 추가되는 환경에서 배치 레이어의 인덱스 재구축이 너무 자주 발생하면 시스템 부하가 될 수 있다.

[2] Hybrid RAG-Empowered Multi-Modal LLM for Secure Data Management in Internet of Medical Things 총정리

저자: Chao Su, Jiawen Wen, Jiaqi Kang, Yang Zhang, Jiankang Ren, Ying He, Mianxiong Dong

학회/년도: IEEE Internet of Things Journal, 2024

분량: 약 15페이지

역할군: (A) VAQ 동기 부여; 실세계 응용 사례

요약

이 논문은 Exqutor가 해결하려는 VAQ(Vector-Augmented Analytical Query)의 **실세계 응용 사례**를 보여준다.

IoMT(Internet of Medical Things) 환경에서 멀티모달 LLM(Large Language Model)과 RAG(Retrieval-Augmented Generation)를 결합한 하이브리드 시스템으로, 의료 시가 단순 텍스트 검색을 넘어 다양한 도메인으로 확장되고 있음을 보여주는 중요한 동기 부여 논문이다.

핵심 문제는 의료 IoT 환경에서 X-ray 이미지, 심전도 시계열, 텍스트 진료 기록 같은 멀티모달 데이터를 통합 분석해야 한다는 것이다. 논문은 세 가지 계층으로 구성된 시스템을 제안한다: (1) 멀티모달 하이브리드 RAG로 각 모달리티별 후보를 검색하고 교차 검증, (2) 블록체인 기반 안전한 데이터 공유, (3) AoI(Age of Information) 메트릭으로 데이터 신선도를 보장하는 인센티브 메커니즘. 특히 첫 번째 계층의 RAG 파이프라인이 바로 **복합 VAQ의 실체**이며, 정확한 카디널리티 추정이 없을 때 얼마나 비효율적인지를 보여준다.

본 논문과의 관계: VAQ의 실제 구현에서 벡터 유사도 검색과 메타데이터 필터링, 다중 모달리티 교차 검증이 결합될 때, 정확한 카디널리티 추정이 얼마나 중요한지를 실제 의료 응용에서 입증한다.

상세분석

2.1 해결하고자 하는 문제: 멀티모달 의료 데이터의 통합 분석

IoMT(Internet of Medical Things) 환경에서는 병원·웨어러블 기기·원격 진료 시스템이 대량의 **멀티모달 의료 데이터**를 생성한다:

- X-ray, CT, MRI 같은 **이미지 데이터** (2D/3D)
- 심전도(ECG), 맥박 센서 같은 **시계열 데이터** (시간 의존적)
- 의사 노트, 처방전, 임상 검사 결과 같은 **텍스트 데이터** (비정형)
- 나이, 성별, 기저질환 같은 **구조화된 메타데이터** (정형)

이 데이터를 기반으로 LLM이 정확한 의료 보조 판단을 내리려면, **검색 증강 생성(RAG)** 파이프라인이 필수적이다. 하지만 기존 RAG 시스템에는 세 가지 핵심 문제가 있다:

문제 1 — 데이터 신선도(Freshness):

의료 데이터는 실시간으로 업데이트되지만, RAG에 제공되는 검색 결과가 오래된 데이터일 경우 LLM의 판단이 부정확해진다.

예를 들어:

- 환자의 가장 최근 혈액 검사 결과(오늘)가 아닌 3개월 전 결과를 참조하면
- 잘못된 진단으로 이어질 수 있다 (예; 수치가 개선되었는데 악화된 것으로 판단)

문제 2 — 보안과 신뢰:

의료 데이터는 극도로 민감하다. HIPAA, GDPR 같은 규제를 만족하면서도:

- 여러 병원이 데이터를 공유할 때, 중앙 기관 없이 어떻게 신뢰를 확보할 것인가?
- 데이터 무결성을 어떻게 보장할 것인가? (누군가가 데이터를 위조하지 않았는가?)
- 암호화된 데이터에서 검색은 어떻게 수행할 것인가?

문제 3 — 인센티브 부재:

데이터 보유자(병원, 기기 제조사)가 고품질 최신 데이터를 적극적으로 제공할 동기가 없다:

- 데이터 공유에 비용이 든다 (인프라 비용, 프라이버시 리스크)
- 하지만 보상이 없다 (비용-편익 불균형)
- 따라서 오래되고 품질 낮은 데이터만 제공하려는 유인이 생긴다

2.2 핵심 아이디어: 하이브리드 RAG 프레임워크

논문이 제안하는 시스템은 **3개 계층**으로 구성된다:

계층 1 — 멀티모달 하이브리드 RAG:**유니모달 검색 (Unimodal Search):**

- 텍스트→텍스트: 진료 기록에서 비슷한 증상/진단 사례 검색 (BERT 임베딩 + 코사인 유사도)
- 이미지→이미지: X-ray 이미지에서 유사한 방사선학적 소견 검색 (CNN 특징 추출 + HNSW)
- 시계열→시계열: ECG 신호에서 비슷한 패턴 검색 (DTW 거리 또는 시계열 임베딩)

각 모달리티에 맞는 임베딩 모델과 거리 함수를 사용하여 후보를 먼저 검색한다.

멀티모달 메트릭 교차 필터링:

X-ray 임베딩 (쿼리)

- 유사 X-ray 이미지 상위 100개 검색
- 각 이미지의 환자 ID 추출
- 해당 환자의 텍스트 진료 기록 조회
- "폐암 의심" 텍스트가 있는지 확인
- 일치하는 것만 최종 결과로 반환

예시:

- X-ray로 폐에 음영이 보이는 환자를 찾으되
- 진료 기록에 실제로 "호흡곤란" 증상이 기록된 환자만 반환
- 이렇게 하면 거짓양성(이미지만 유사하지만 증상이 다른 경우)을 줄일 수 있다

이것이 바로 VAQ의 실체이다:

벡터 유사도 검색(이미지 임베딩 매칭) + 관계형 필터(환자 메타데이터, 날짜 범위 등) + 다른 모달리티와의 교차검증을 **동시에** 수행해야 한다.

계층 2 — 계층적 크로스체인 아키텍처:**블록체인 기반 안전한 데이터 공유:**

- 중앙 기관(예; 의료청) 없이도 데이터 출처를 검증할 수 있다
- 메인체인; 전체 네트워크의 합의를 관리 (병원 A, B, C가 모두 인정하는 거래)
- 사이드체인; 각 기관의 로컬 데이터를 처리 (병원 A의 환자 데이터는 병원 A의 사이드체인에서만 관리)

장점:

- 중앙 통제 없음 (한 병원이 데이터를 조작해도 다른 병원은 다른 복사본을 가짐)
- 감사 추적(audit trail) 자동 생성 (누가 언제 어떤 데이터를 조회했는가를 블록체인에 기록)

계층 3 — Aol 기반 인센티브 메커니즘:

Age of Information (Aol) 메트릭:

데이터가 얼마나 오래되었는가를 정량화한다.

$$AoI = (\text{현재시각}) - (\text{마지막 업데이트시각})$$

예시:

- 환자의 혈액 검사가 오늘 업데이트됨 → Aol = 1시간 (최신)
- 3개월 전에만 업데이트됨 → Aol = 90일 (오래됨)

계약 이론(Contract Theory) 기반 인센티브:

- 데이터 보유자에게 보상을 제공한다: 데이터 신선도 Aol이 낮을수록 (데이터가 신선할수록) 더 많은 보상
- 다양한 계약 옵션 제공: 고신선도·고비용 vs. 저신선도·저비용
- 각 병원이 자신의 상황에 맞는 계약을 선택할 수 있음

확산 기반 심층 강화학습(Diffusion-based DRL):

- 기존의 경제학적 인센티브 설계는 정적이지만, 현실의 의료 시장은 동적이다
- 시간에 따라 데이터 가치가 변한다 (질병이 유행하면 해당 데이터의 가치 상승)
- DRL이 이 동적 변화에 실시간으로 대응하여 최적 계약을 도출한다

2.3 멀티모달 RAG 파이프라인의 구체적 예시

```
-- 의료 시스템에서의 RAG 검색
SELECT patient.name, xray.image_url, diagnosis.text
FROM patients
JOIN xray_images ON patients.id = xray_images.patient_id
JOIN diagnoses ON patients.id = diagnoses.patient_id
WHERE
  -- 벡터 유사도: 쿼리 이미지와 유사한 X-ray 찾기
  vector_distance(xray.embedding, query_image_embedding) < 0.3

  -- 메타데이터 필터: 동일 나이대, 동일 성별
  AND patients.age BETWEEN 40 AND 60
  AND patients.gender = 'M'

  -- 텍스트 유사도: 진료 기록이 증상과 일관성 있는가
  AND text_similarity(diagnosis.text, 'shortness of breath') > 0.7

  -- 데이터 신선도: 최근 6개월 내 기록
  AND diagnosis.created_at > NOW() - INTERVAL '6 months'

ORDER BY vector_distance DESC
LIMIT 5;
```


이 쿼리에서:

- 벡터 연산 (image embedding similarity)
- 메타데이터 필터 (age, gender)
- 다른 벡터 연산 (text similarity)
- 시간 필터 (데이터 신선도)
- 조인 (3개 테이블)

모두가 결합되어 있으며, 정확한 카디널리티가 없으면:

- 벡터 검색을 먼저 할지 메타데이터 필터를 먼저 할지 결정 불가
- 중간 결과가 1,000건인지 100,000건인지 알 수 없어 조인 방식 선택 불가
- 최악의 경우 응답 시간이 100배 이상 느려질 수 있음

2.4 성능 고려사항

멀티모달 검색의 계산 비용:

- X-ray CNN 임베딩: 프레임당 500ms (GPU 사용 시 50ms)
- 텍스트 임베딩 (BERT): 1,000 토큰당 100ms
- 유사도 계산: 백만 벡터 × 1,000 차원 = 1초 (SIMD 최적화 시)

시스템이 매초 100건의 쿼리를 처리해야 한다면:

- 비효율적 실행 계획: 1쿼리 당 5초 → 처리 불가능
- 최적 실행 계획: 1쿼리 당 500ms → 처리 가능

정확한 카디널리티의 가치:

각 필터 단계에서 결과를 얼마나 줄일 수 있는지 알면:

- 벡터 유사도로 1,000,000건 → 10,000건 (1% 선택도)
- 나이/성별 필터로 10,000건 → 3,000건 (30% 선택도)
- 텍스트 유사도로 3,000건 → 150건 (5% 선택도)

이를 알면 **가장 선택도가 높은 필터를 먼저 적용**하여 중간 결과를 최소화할 수 있다.

2.5 본 논문과의 관계

이 논문의 RAG 파이프라인은 "벡터 유사도 검색 → 메타데이터 필터링 → 다중 모달리티 교차 검증"이라는 **복잡한 쿼리 체인**을 요구한다. 이것이 SQL로 표현되면 여러 테이블 조인과 벡터 범위 검색이 결합된 VAQ가 된다.

Exqutor의 정확한 카디널리티 추정이 없으면:

- 옵티마이저가 비효율적인 실행 계획을 선택
- 응답 시간이 수십 배 느려짐
- 실시간 의료 지원 시스템으로서 기능 불가능

특히 의료 환경에서는 **실시간 응답이 치명적으로 중요**하므로, Exqutor 스타일의 최적화가 실질적 가치를 갖는다.

추가 제기 문제

1. **블록체인 오버헤드:** 매 검색마다 블록체인에 접근하면 레이턴시가 증가한다. 읽기 전용 쿼리에서는 블록체인 확인을 생략할 수 있을까?
2. **멀티모달 메트릭의 카디널리티 변동:** 이미지→텍스트 교차 필터링 과정에서 선택도가 극심하게 변할 수 있다. 정적 통계로는 이를 예측하기 어렵다 → Exqutor의 동적 샘플링이 특히 유용할 시나리오.

3. **모달리티 간 정렬 문제**: 서로 다른 모달리티의 유사도 점수(이미지 거리 0.3, 텍스트 유사도 0.7)를 어떻게 결합할 것인가? 정규화 방식에 따라 최종 순위가 크게 변한다.
4. **Aoi 인센티브의 현실성**: 이론적으로는 데이터 신선도에 비례한 보상이 최적이지만, 실제 병원들이 이를 따를 동기가 있을까? (기존 관례, 정치적 고려 등)

[3] Tree-Based RAG-Agent Recommendation System; A Case Study in Medical Test Data 총정리

저자: Yaofeng Yang, Chufan Huang

학회/년도: arXiv:2501.02727, 2025

분량: 약 10페이지

역할군: (A) VAQ 동기 부여; 계층적 쿼리 패턴의 실제 사례

요약

이 논문은 RAG 기반 시스템이 의료 도메인에서 **계층적 추론**과 결합될 때 어떤 복잡한 쿼리 패턴이 필요한지를 보여준다. 단순 top-k 벡터 검색으로는 해결할 수 없는, **다단계 필터링과 집계**가 필요한 실세계 시나리오의 중요한 사례 연구이다.

의료 검사 추천 시스템의 핵심은 "환자의 증상을 보고, 어떤 진료과에 보낼지, 그리고 어떤 검사를 처방할 것인가"라는 결정이다. 이것은 단순하지 않은 **계층적 추론**을 요구한다. HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation) 시스템은 트리 구조로 이를 구현하는데, 각 노드는 전문화된 RAG 에이전트이고, 상위 계층부터 하위 계층으로 점진적으로 검색 공간을 좁혀나간다.

이 구조를 SQL VAQ로 표현하면, 각 트리 레벨에서 카디널리티가 크게 달라지는 **다단계 조인과 벡터 범위 검색의 결합**이 된다. 정확한 카디널리티 추정이 없으면, 상위 레벨의 부정확한 카디널리티가 하위 레벨로 전파되어 최악의 경우 지수적 성능 저하를 초래한다.

본 논문과의 관계: 의료 도메인에서 트리 구조의 각 노드별로 서로 다른 카디널리티 특성을 갖는 VAQ가 발생하며, 이것이 정확한 카디널리티 추정의 필요성을 강하게 보여준다.

상세분석

3.1 해결하고자 하는 문제: 계층적 의료 추론의 필요성

의료 검사 추천 시스템에서 "환자의 증상을 보고, 어떤 진료과에 보내고, 어떤 검사를 처방할 것인가"라는 결정은 결코 단순하지 않은 **계층적 추론**을 요구한다.

현재 시스템의 문제

기존 LLM의 한계:

일반 목적(General-Purpose) LLM에 "환자가 심한 두통을 호소한다"고 입력하면:

GPT-4의 응답:

"신경학적 검사(neurological exam)와 함께,
필요시 MRI나 CT 스캔을 추천합니다.
또한 안과 검진도 고려하세요."

문제점:

- 1. **과도한 추천**: 모든 가능성을 다루려고 하여 불필요한 검사까지 권장 (비용 증가)
- 2. **진료과 혼동**: 신경과, 안과, 이비인후과 중 어디가 1차 진료과인지 불명확
- 3. **일관성 부족**: 같은 환자에 대해 시간이 지나면 다른 추천을 할 수 있음 (의료 오류 유발)

RAG를 추가해도 해결 안 되는 문제:

벡터 유사도 검색만으로는:

- "두통 케이스" 벡터를 검색해서 가장 유사한 100개 사례를 찾은 후
- LLM에 "이 100개 사례에서 공통 검사는 뭐야?"라고 질문

하지만 이 방식의 문제:

- 두통은 수십 가지 원인이 있는데 (뇌종양, 편두통, 안경 필요, 귀 감염 등)
- 모든 사례를 함께 고려하면 결과가 너무 다양해짐
- 특정 원인(예; 뇌종양 의심)에 특화된 검사 프로토콜을 세울 수 없음

3.2 핵심 아이디어: HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation)

3계층 트리 구조



각 노드는 **전문화된 RAG 에이전트**이며, 자신의 범위에 해당하는 지식 베이스에서만 검색·증강·생성을 수행한다.

예시: "두통" 증상 입력

Step 1 (Root 노드):

- | | |
|----------------|--|
| 두통의 원인 분류 | |
| - 신경과적 (45%) | |
| - 안과적 (20%) | |
| - 이비인후과적 (30%) | |
| - 기타 (5%) | |

↓

Step 2 (신경과 노드):

- | | |
|----------------------|--|
| 신경과 차원에서 추가 질문 | |
| - 갑작스러운 발병? (뇌졸중 의심) | |
| - 반복적? (편두통 의심) | |
| - 국소적 신경 증상? (종양 의심) | |

↓

Step 3 (신경과 → MRI 노드):

- | | |
|---------------------|--|
| 뇌종양 선별 규약 | |
| - MRI 기본 (모든 환자) | |
| - 조영제 확산 (강제) | |
| - 함수적 MRI (기능 손상 시) | |

이중 레이어 지식 베이스**상위 레이어 (Department Knowledge Base):**

증상 → 진료과 매핑 규칙

- 두통 + 시력 변화 → 안과 우선
- 두통 + 이명 → 이비인후과 우선
- 두통 + 마비 증상 → 신경과 우선

하위 레이어 (Test Knowledge Base):

진료과별 검사 상세 정보

[신경과 - MRI]

- 적응증: 뇌종양, 뇌졸중, 경련성 질환
- 금기사항: 금속 임플란트 (페이스메이커 등)
- 비용: 300,000원
- 소요시간: 40분
- 진단 가능성: 뇌종양 95%, 편두통 10%

메모리 증강 추론

이전 진단 이력을 메모리에 유지하여:

같은 환자 (ID: P12345)

- 1차 방문: 두통 → 신경과 의뢰 → MRI 처방
- 2차 방문: 지속적 두통 → 신경과 (이미 방문함) → 추가 검사 결정
 - 메모리에서 "이 환자는 신경과 경험 있음"을 참조
 - 동일 검사 반복 회피 가능

3.3 성능 결과

의료 검사 추천에 대한 평가:

평가 지표	단일 LLM	HiRMed	개선도
커버리지 (관련 검사를 빠뜨리지 않음)	78.9%	92.3%	+13.4%
정확도 (추천한 검사가 실제로 적절)	71.2%	88.7%	+17.5%
비용 효율성 (불필요한 검사 회피율)	45%	78%	+33%
일관성 (같은 환자의 반복 방문 시 동일 추천)	62%	94%	+32%

3.4 트리 구조의 카디널리티 특성

Root 레벨의 카디널리티

"두통" 증상의 환자 풀:

전체 환자 DB: 1,000,000명
두통으로 내원한 환자: 50,000명 (선택도 5%)

진료과별 분류:

- 신경과 (진료과 A): 22,500명 (45%)
- 안과 (진료과 B): 10,000명 (20%)
- 이비인후과 (진료과 C): 15,000명 (30%)
- 기타 (진료과 D): 2,500명 (5%)

검사 노드의 카디널리티

신경과 내에서 더 세밀한 분류:

신경과 환자 22,500명

MRI 추천:

- 급성 신경 증상: 5,000명 → MRI 필수 (100% 선택도)
- 만성 두통 (3개월 이상): 8,000명 → MRI 선택적 (60% 선택도)
- 편두통 (빈번하지 않음): 9,500명 → MRI 불필요 (5% 선택도)

→ 전체 신경과 환자 중 MRI 추천율: $(5000 + 4800 + 475) / 22500 = 28\%$

3.5 VAQ로의 변환

HiRMed의 검사 추천 로직을 SQL로 표현하면:

```

-- 1단계: 증상 기반 진료과 후보 검색
SELECT dept_id, dept_name, matching_score
FROM departments
WHERE vector_distance(dept.symptom_profile, query_symptom_embedding) < 0.5
ORDER BY matching_score DESC;

-- 2단계: 각 진료과 내 구체적 검사 추천
SELECT test_id, test_name, recommendation_score
FROM medical_tests
WHERE
  -- 벡터 유사도: 환자 증상과 검사의 적응증 일치
  vector_distance(test.indication_embedding, query_symptom_embedding) < 0.4

  -- 메타데이터 필터: 환자 프로필과의 호환성
  AND patient_age BETWEEN test.recommended_age_min AND test.recommended_age_max
  AND test.contraindication NOT IN (patient_medical_history)

  -- 진료과 필터: 1단계에서 선정된 진료과만
  AND test.department_id IN (SELECT dept_id FROM step1_result)

  -- 비용 제약: 보험 적용 검사만
  AND test.insurance_covered = TRUE

ORDER BY recommendation_score DESC;

-- 3단계: 환자 이력 고려 (메모리 증강)
SELECT test_id
FROM step2_result
WHERE
  -- 같은 환자가 최근 12개월 내 받은 검사 제외
  test_id NOT IN (
    SELECT test_id FROM patient_history
    WHERE patient_id = :patient_id
    AND test_date > NOW() - INTERVAL '12 months'
  );

```

이 SQL 쿼리에서:

- 1단계: 벡터 검색 결과 카디널리티 = 3~5 진료과
- 2단계: 각 진료과별 검사 후보 = 수십~수백 개 검사
- 3단계: 최종 필터링 후 = 3~10개 검사

카디널리티의 급격한 변화:

- 1단계 결과 3 → 2단계 중간 결과 300 → 3단계 최종 8
- 이런 급격한 변화에는 정적 통계가 무용지물

3.6 정확한 카디널리티의 가치

카디널리티 추정 오류의 영향

시나리오 A: 카디널리티 과소 추정

실제 카디널리티: 500개 검사
추정 카디널리티: 10개 검사 (추정 오류)

옵티마이저의 판단:

- "1단계 결과를 먼저 조인하고, 그 다음 벡터 검색"
- 실행 계획: JOIN (3개 진료과) JOIN FILTER (vector search)
- 예상 비용: 낮음 (3 × 처리 비용)

실제 실행:

- 3개 진료과 조인으로 22,500건 나옴
- 각 22,500건에 대해 벡터 검색 (비싼 연산) 수행
- 실제 비용: 매우 높음 (22,500 × 벡터 검색 비용)

시나리오 B: 카디널리티 과다 추정

실제 카디널리티: 50개 검사
추정 카디널리티: 500개 검사 (추정 오류)

옵티마이저의 판단:

- "벡터 검색을 먼저 수행, 그 결과만 조인"
- 실행 계획: VECTOR_SEARCH (500개 후보) → JOIN 필터
- 예상 비용: 높음 (500 × 처리 비용)

실제 실행:

- 벡터 검색 수행 (실제로는 50개만 해당)
- 하지만 이미 500개 후보 검색 비용이 발생함
- 불필요한 계산 450개 분 낭비

정확한 카디널리티로 얻는 이득

각 단계에서:

- 1단계 결과가 3개 → 조인 비용 최소화
- 2단계 중간 결과가 300개 → 해시 조인 선택 (Nested Loop 피함)
- 3단계 필터가 8개 → Sort 생략, Limit 최적화

전체 성능:

- 부정확한 계획: 응답시간 10~20초 (비용 초과)
- 정확한 계획: 응답시간 1~2초 (10배 이상 개선)

3.7 본 논문과의 관계

HiRMEd의 각 노드에서 수행하는 RAG 검색은:

- 벡터 유사도 (증상 임베딩 매칭)
- + 메타데이터 필터 (진료과 코드, 연령, 금기사항 등)
- + 집계 (우선순위 랭킹)
- + 과거 이력 제외

을 결합하는 복합 VAQ이다.

이 과정이 SQL로 표현되면 **다단계 조인과 벡터 범위 검색**의 결합이 되며, 특히:

- 트리의 각 레벨에서 카디널리티가 **크게 달라짐** (Root 50,000 → 검사 500 → 최종 8)
- 상위 레벨의 카디널리티 오류가 하위 레벨로 전파됨 (지수적 성능 저하)
- 정적 통계로는 이 변동성을 포착하기 어려움

따라서 Exqutor의 **동적 카디널리티 추정**이 특히 유용한 시나리오이다.

추가 제기 문제

1. 트리 구조의 고정성:

2. 현재 진료과 분류는 고정되어 있다.
3. 새로운 진료과(예; AI 기반 정신건강 상담)가 추가되면 트리 재구성 필요
4. 동적으로 트리를 조정하는 메커니즘이 필요하지 않을까?

5. 노드 간 쿼리 최적화 부재:

6. 각 노드의 RAG 검색이 독립적으로 수행됨
7. 예; Root에서 찾은 진료과 정보를 하위 노드에서 재활용할 수 있는데 활용 안 함
8. 노드 간 중간 결과 공유 (캐싱)로 성능 개선 가능 → Exqutor의 아이디어와 유사

9. 멀티모달 의료 데이터 미지원:

10. 현재는 텍스트 증상만 고려
11. X-ray 이미지나 혈액 검사 수치 같은 구조화 데이터를 함께 고려하면?
12. 더 정교한 추천이 가능하지만 복잡도 급증 → 카디널리티 추정 더욱 어려워짐

13. 피드백 루프 부재:

14. 추천한 검사가 실제로 효과적이었는가를 시스템이 학습하지 않음
15. 시간에 따라 진료 가이드라인이 변하는데 (의료 지식 업데이트)
16. 시스템이 자동으로 적응하는 메커니즘이 필요
17. Exqutor의 런타임 적응적 샘플링이 이 문제의 해답이 될 수 있을까?

[4] Accelerating Machine Learning Inference with Probabilistic Predicates 총정리

저자: Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, Surajit Chaudhuri (Microsoft Research)

학회/년도: SIGMOD 2018 (ACM International Conference on Management of Data)

분량: 약 16페이지

역할군: (D) 이론적 기반; ML+DB 최적화의 다른 각도

요약

이 논문은 "비구조화 데이터에 대한 ML 추론 쿼리에서 비싼 UDF(User-Defined Function)를 어떻게 최적화할 것인가"라는 문제를 다룬다. Exqutor와는 다른 각도에서 ML+DB 최적화를 접근하지만, 두 논문 모두 **"ML/벡터 연산을 관계형 쿼리 옵티마이저에 어떻게 통합할 것인가"**라는 큰 질문을 공유한다.

핵심 아이디어는 비싼 ML 모델(딥러닝 기반 객체 탐지, 얼굴 인식 등) 이전에 **경량 이진 분류기를 "조기 필터"로 삽입**하는 것이다. 이를 통해 불필요한 데이터를 미리 걸러내고 비싼 ML 추론의 횟수를 크게 줄인다. 여러 필터를 계단식으로 연결(cascade)하면 각 단계에서 점진적으로 데이터가 정제되어 전체 비용을 극적으로 감소시킨다.

Exqutor가 벡터 검색의 **카디널리티**를 정확히 추정하여 옵티마이저를 돕는 것과 유사하게, 이 논문은 ML 추론의 **비용과 선택**도를 옵티마이저에 통합한다. 두 논문의 공통 메시지는 **"ML 연산의 특성을 옵티마이저에 정확히 반영하면 극적인 성능 향상이 가능하다"**는 것이다.

본 논문과의 관계: 비싼 연산(벡터 검색 vs. ML 추론)의 특성을 옵티마이저에 통합하는 서로 다른 접근법을 보여주며, 두 방식의 결합 가능성을 시사한다.

상세분석

4.1 해결하고자 하는 문제

비디오 분석, 이미지 검색 같은 ML 추론 쿼리에서 핵심 병목은 **비싼 UDF(User-Defined Function)**이다.

문제 상황의 구체적 예시

```
SELECT * FROM video_frames
WHERE object_detector(frame, 'red SUV') = TRUE -- 매우 비쌘!
AND timestamp BETWEEN '2024-01-01' AND '2024-01-31';
```

여기서 `object_detector()` 는 딥러닝 모델로:

- ResNet 또는 YOLO 같은 대규모 신경망
- 한 프레임당 수십~수백 ms가 소요됨 (GPU 사용 기준)
- 100만 프레임 비디오라면; 100만 × 100ms = 100,000초 = **약 28시간** 소요

그런데 실제로 "빨간 SUV"가 나오는 프레임은 전체의 **0.1%도 안 될 수 있다** (예; 차가 10,000개 장면 중 10장에만 나타남).

따라서 불필요한 추론이 많아서, 이를 줄일 수 있다면 극적인 성능 개선이 가능하다.

기존 최적화의 한계

기존 쿼리 최적화의 **술어 푸시다운(Predicate Pushdown)** 기법:

```
[100만 프레임]
↓ [시간 필터 적용] (timestamp 조건)
↓ [31만 프레임] (1월 데이터만)
↓ [ML 추론 UDF 실행]
↓ [300 프레임] (빨간 SUV 있는 것)
↓ [결과 반환]
```

이것만으로도 도움이 되지만, **여전히 31만 프레임마다 비싼 DL 모델 실행**해야 한다. 추가로:

- 31만 × 100ms = 31,000초 = **약 8.6시간** (여전히 너무 김)

원근법을 바꿔서, **ML UDF 이전에 경량 필터를 먼저 삽입**할 수 있을까?

4.2 핵심 아이디어: 확률적 술어 (Probabilistic Predicates, PPs)

핵심 통찰: 비싼 ML 추론을 수행하기 전에 **경량 이진 분류기를 조기 필터로 사용**하면, 대부분의 음성 사례(false cases)를 미리 제거할 수 있다.

오프라인 학습 단계

[Step 1] 비싼 UDF를 소수의 데이터에 실행하여 레이블 수집

- └ 전체 1,000,000 프레임 중
- └ 샘플 1,000 프레임만 선택하여
- └ YOLO 객체 탐지 모델 실행
- └ 레이블 수집 (Red SUV 있음/없음)

[Step 2] 저비용 특징 추출

- └ 색상 히스토그램 (각 RGB 채널의 분포; 512B)
- └ 텍스처 특징 (Edge detection; 256B)
- └ 저해상도 미리보기 (32×32 픽셀; 3KB)
- └ 전체 특징 크기 ~4KB (원본 1080p 프레임 2MB 대비 500배 소)

[Step 3] 경량 이진 분류기 학습

- └ 입력: 1,000개 프레임의 저비용 특징
- └ 출력: Red SUV 있음/없음 (이진)
- └ 알고리즘: 로지스틱 회귀 (선형 모델)
- └ 학습 비용: 1초 미만
- └ 정확도: 95% (비싼 YOLO의 정확도도 97%)

온라인 실행 단계

각 프레임에 대해:

[단계 1] 저비용 특징 추출 (4KB)

↓ (비용: 1ms)

[단계 2] 경량 분류기 실행

- └ 99% 확률로 "Red SUV 없음" 판정?
- └ ↓ [프레임 스킵; 비싼 YOLO 실행 안 함]
- └ ↓ (절약된 비용: 100ms)
- └ 우호적 점수? 계속 진행
- └ ↓ (누적 비용: 1ms)

[단계 3] 비싼 YOLO 객체 탐지 실행

↓ (비용: 100ms)

[단계 4] 최종 결과

극적인 비용 절감:

- 100만 프레임 중 실제 Red SUV: 1,000개 (0.1%)
- 경량 필터로 음성 예측 정확도 99%: 990,000개 프레임 제외
- 실제 비싼 YOLO 실행 대상: 10,000개 (9,900개 거짓양성 + 1,000개 실제 양성)
- 비용 절감: $(100만 - 10,000) \times 100ms = 990,000 \times 100ms = 99,900초$ 절감

4.3 Viola 스타일 캐스케이드 (Cascade of Classifiers)

경량 필터 하나만으로는 부족할 수 있다. 대신 **여러 개를 계단식으로 연결**한다:

[입력 프레임]

↓

[PP_1: 초경량 필터 (비용 1ms)]

- └ 색상 기반 (빨간색 있는가?)
- └ 매우 빠름
- └ 정확도는 낮음 (false negative 10%)
- └ 90% 제외

↓ [10만 프레임]

[PP_2: 경량 필터 (비용 5ms)]

- └ 색상 + 모양 (직사각형 같은 물체 있는가?)
- └ 중간 속도
- └ 정확도 중간 (false negative 5%)
- └ 80% 추가 제외

↓ [2만 프레임]

[PP_3: 중간 필터 (비용 30ms)]

- └ 에지 감지 + 패턴 매칭 (차량 같은 형태?)
- └ 느린 편
- └ 정확도 높음 (false negative 2%)
- └ 90% 추가 제외

↓ [2,000 프레임]

[비싼 YOLO (비용 100ms)]

- └ 정확한 객체 탐지
- └ 최종 결과

캐스케이드의 이점:

1. 각 단계에서 비용이 점진적으로 증가 (1 → 5 → 30 → 100ms)
2. 각 단계에서 데이터가 걸러짐 (100% → 10% → 2% → 0.2%)
3. 전체 비용 최소화

4.4 비용-정확도 트레이드오프 모델링

PP의 정확도-성능 트레이드오프를 비용 기반 옵티마이저에 통합한다:

거짓 음성률 (False Negative Rate, FNR)

- 정의: "실제로 Red SUV가 있는데, 필터가 없다고 판단"
- 영향: 최종 결과에서 Red SUV 놓침 (정확도 저하)
- 예; FNR = 5%면 100개 Red SUV 중 5개를 못 찾음

거짓 양성률 (False Positive Rate, FPR)

- 정의: "실제로 Red SUV가 없는데, 필터가 있다고 판단"
- 영향: 불필요한 비싼 YOLO 실행 (성능 저하)
- 예; FPR = 10%면 999,000개 음성 중 99,900개가 거짓양성으로 판정되어 비싼 YOLO까지 감

옵티마이저의 선택

사용자가 "정확도 손실 1% 이내로, 최대한 빨리" 요청하면:

가능한 PP 조합 평가:

옵션 A: PP 없음 (카스케이드 스킵)

- └─ 정확도: 100%
- └─ 실행시간: 100,000초 ($1,000,000 \times 100\text{ms}$)
- └─ 비용 평가: 불합격 (정확도 요구와 무관하게 너무 느림)

옵션 B: PP_1만 사용

- └─ FNR: 10%, FPR: 5%
- └─ 최종 정확도: 99.95% (Red SUV 1,000개 중 900개 맞춤)
- └─ 실행시간: 1,900 초 ($1,000,000 \times 1\text{ms} + 50,000 \times 100\text{ms}$)
- └─ 평가: 합격 (정확도 99.95% > 99%, 시간 충분히 단축)

옵션 C: PP_1 + PP_2

- └─ FNR: 7% (PP_1 10% 중 PP_2가 30% 추가 보정)
- └─ 최종 정확도: 99.93%
- └─ 실행시간: 500초
- └─ 평가: 합격 (더욱 빠름)

옵티마이저는 조건을 만족하면서 가장 빠른 옵션 C를 선택한다.

4.5 비용 모델의 상세

카스케이드 전체 비용

Total Cost = Sum[각 필터별 비용] + Sum[필터 통과한 데이터의 비싼 UDF 비용]

Total = $(N \times \text{cost}_1) + (N \times k_1 \times \text{cost}_2) + \dots + (N \times k_1 \times k_2 \times \dots \times k_{n-1} \times \text{costUDF})$

여기서:

- N = 전체 입력 데이터 (1,000,000 프레임)
- cost_i = 필터 i 의 단위 비용 (1ms, 5ms, 30ms)
- k_i = 필터 i 의 통과율 (10%, 20%, 10%)

실제 계산 예시

PP_1 (색상) → PP_2 (모양) → PP_3 (패턴) → YOLO

Total = $(1,000,000 \times 1\text{ms})$
 $+ (1,000,000 \times 0.9 \times 5\text{ms})$
 $+ (900,000 \times 0.8 \times 30\text{ms})$
 $+ (720,000 \times 0.9 \times 100\text{ms})$

 $= 1,000\text{초} + 4,500\text{초} + 21,600\text{초} + 64,800\text{초}$
 $= 91,900\text{초}$ (약 25시간)

비교:

- 필터 없음: 100,000초 (약 28시간)
- 캐스케이드: 91,900초 (약 26시간)
- 개선: 8% (약 1시간 절감)

→ 여전히 개선의 여지가 있으나, 트레이드오프 고려 필요

4.6 성능 결과

이 논문은 SIGMOD 2018 Best Demonstration Award를 수상했으며, 실제 구현 사례들:

비디오 프레임 분석

데이터:

- WILDTRACK 비디오 데이터셋 (100,000 프레임)
- 쿼리: "사람을 탐지하는 프레임" (평균 선택도 30%)

결과:

기준 (PP 없음):	10,000초 (비싼 YOLO만)
PP 캐스케이드 적용:	100초 (색상 기반 필터)
성능 향상:	100배
정확도 유지:	97% (손실 0%)

이미지 검색

쿼리: "고양이 탐지" (선택도 5%)

결과:	
- 기준:	150초
- 최적화:	5초
- 성능 향상:	30배
- 정확도 손실:	0.1% (허용 범위 내)

4.7 본 논문과의 관계

공통점

Exqutor와의 공통 철학:

1. ML/벡터 연산을 관계형 옵티마이저에 통합

- Exqutor: 벡터 검색의 카디널리티를 정확히 추정
- 이 논문: ML UDF의 비용과 정확도를 모델링

1. 정확한 정보를 옵티마이저에 제공하면 극적 개선 가능

2. Exqutor: 정확한 선택도 → 최적 조인 순서 선택

3. 이 논문: 정확한 비용 → 최적 필터 캐스케이드 선택

4. 정확도-성능 트레이드오프 관리

5. AnalyticDB-V: 벡터 검색의 정확도-비용 트레이드오프

6. 이 논문: PP의 FNR-FPR 트레이드오프

차이점

Exqutor:

- 대상: 벡터 범위 검색 ($distance < D$)
- 문제: "결과가 정확히 몇 건인가?"
- 해결: HNSW 인덱스 탐색으로 카디널리티 추정

이 논문:

- 대상: 비싼 ML UDF
- 문제: "UDF를 실행할 대상을 최소화하되 정확도는 유지하는가?"
- 해결: 경량 필터 캐스케이드로 불필요한 UDF 제거

결합 가능성

두 기법을 함께 사용할 수 있다:

```
-- 벡터 검색 + ML 추론 결합
SELECT * FROM images
WHERE
  -- [1단계] 경량 색상 필터 (PP_1)
  color_histogram_match(image) = TRUE

  AND

  -- [2단계] 벡터 유사도 (Exqutor 대상)
  vector_distance(image_embedding, query_embedding) < 0.5

  AND

  -- [3단계] 비싼 CNN 기반 객체 탐지 (PP_2, PP_3)
  object_detector(image, 'cat') = TRUE;
```

Exqutor가 2단계의 카디널리티를 정확히 예측하면, 옵티마이저가 각 단계의 순서를 최적화할 수 있다.

추가 제기 문제**1. PP 학습의 적응성:**

2. PP를 학습할 때 사용한 1,000개 샘플의 데이터 분포가, 나중의 실제 쿼리와 달라질 수 있다 (Concept Drift)
3. 예; 밤에 찍은 비디오에서는 빨간 SUV의 색상이 검게 보일 수 있음
4. 이 경우 PP의 성능이 떨어진다

Exqutor의 적응적 샘플링이 이 문제의 해답이 될 수 있을까?

- 런타임에 PP의 오류율을 모니터링
- 오류율이 높아지면 새 데이터로 PP를 재학습

1. 여러 쿼리 간 PP 공유:

2. 현재는 각 쿼리마다 별도의 PP 캐스케이드를 구성
3. 예; "빨간 차", "파란 차", "검은 차" 쿼리가 모두 색상 필터 공유 가능
4. 이런 공유를 자동 탐지하는 메커니즘?

5. PP의 학습 비용 고려:

6. 1,000개 샘플에 대해 비싼 YOLO 실행 (100초)

7. 분류기 학습 (1초)
8. 이 초기 비용이 언제쯤 회수되는가?
9. 만약 쿼리가 few-shot (한 번만 실행)이라면 학습 비용이 낭비됨
10. 비선형 필터 조합:
 11. 현재는 선형 캐스케이드 ($PP_1 \rightarrow PP_2 \rightarrow PP_3$)
 12. 하지만 일부 필터는 병렬 실행 가능한가? (예; 색상 필터와 에지 필터 동시 실행)
 13. 이런 병렬화로 추가 최적화 가능?

[5] Analytical Engines with Context-Rich Processing; Towards Efficient Next-Generation Analytics 총정리

저자: Viktor Sanca, Anastasia Ailamaki (EPFL — Data-Intensive Applications and Systems Lab)

학회/년도: ICDE 2023 (IEEE 40th International Conference on Data Engineering) — Vision Paper

분량: 약 8페이지

역할군: (D) 이론적 기반; 임베딩을 관계형 대수에 통합하는 비전

요약

이 논문은 [6]번 논문(Context-Enhanced Relational Operators with Vector Embeddings)의 **전신이 되는 비전 논문**으로, 임베딩을 관계형 대수에 통합하는 아이디어의 출발점이다. 비전 논문(Vision Paper)이란 구체적인 실험보다는 **새로운 연구 방향을 제시하고 커뮤니티의 논의를 유도**하는 논문으로, [6]의 기술적 기여를 이해하기 위한 철학적·개념적 배경이 된다.

핵심 메시지는 현대 데이터의 **80~90%가 비구조화 데이터**인데, 관계형 DBMS는 여전히 구조화 테이블에 최적화되어 있다는 것이다. 따라서 **임베딩 연산자(E-Operator)**를 관계형 대수의 **공식 연산자로 정의**하면, 기존 최적화 규칙(선택 푸시다운, 조인 재정렬 등)을 임베딩 연산에도 자동으로 적용할 수 있다. 이것이 바로 VAQ(Vector-Augmented Analytical Query) 최적화의 이론적 토대가 된다.

본 논문과의 관계; 비전 논문으로서 구체적인 실험은 없지만, "임베딩을 관계형 대수에 통합하면 어떤 이득을 얻을 수 있을까"라는 근본적 질문에 대한 답을 제시하며, Exqutor가 이를 실행하는 데 필요한 정확한 카디널리티 추정의 필요성을 이론적으로 정당화한다.

상세분석

5.1 해결하고자 하는 문제; 데이터의 본질적 변화

데이터 생태계의 변화

과거(2010년대 초):

데이터 구성:

- 구조화: 80% (숫자, 날짜, 범주형)
- 비구조화: 20% (이미지, 텍스트)

도구 분포:

- RDBMS 투자: 80% (SQL, 트랜잭션, ACID)
- 비구조화 처리: 20% (특수 시스템)

현대(2020년대):

- 구조화: 10~20% (메타데이터만 해당)
- 비구조화: 80~90% (텍스트, 이미지, 오디오, 비디오)

도구 분포:

- RDBMS: 여전히 60% 투자 (기존 자산)
- 새로운 도구: 40% (벡터 DB, LLM, 특수 엔진)

→ ****Mismatch****: 데이터는 비구조화인데 도구는 구조화 중심

전형적인 데이터 분석 파이프라인의 문제

현재 관행:

[Step 1] 구조화 데이터

관계형 DB	
user_id, age, city	



[Step 2] 비구조화 데이터 추출

사진, 텍스트	
(DB 밖으로 반출)	



[Step 3] 임베딩 생성 (DB 외부)

ML 파이프라인	
BERT, ResNet 실행	
→ 벡터 생성	



[Step 4] 벡터 DB에 로드

Milvus 또는 pgvector	
벡터 저장	



[Step 5] 유사도 검색

별도 쿼리로 검색	
Top-k 결과 수집	



[Step 6] 결과를 다시 원래 DB와 조인

수동 코딩으로	
프로그래밍으로 결합	

문제:

- **복잡함**: 개발자가 6개 단계의 ETL을 수동 관리
- **비효율**: 각 단계마다 데이터를 반복 로드/저장
- **최적화 불가**: DB 옵티마이저가 전체 파이프라인을 볼 수 없음 (일부는 블랙박스)
- **응답 시간**: 각 단계의 레이턴시가 누적되어 전체 응답 시간이 수십 초 이상 가능

5.2 핵심 제안: E-Operator와 E-Scan 프레임워크

E-Operator (Embedding Operator)

임베딩을 관계형 대수의 공식 연산자로 정의:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

입력: 관계(테이블) R과 임베딩 모델 μ
 예; $R = (\text{user_id}, \text{user_image})$
 $\mu = \text{ResNet-50}$ 이미지 임베딩 모델

출력: R의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계
 예; $(\text{user_id}, \text{user_image}, \text{embedding}[512])$

중요한 성질

1. 역연산 가능성:

$$E\mu^{-1}(E\mu(R)) = R$$

즉, 원래 데이터를 복원할 수 있다.

예시:

임베딩 후 → 벡터 유사도 검색 → 이미지 메타데이터 추출

← 원본 $\text{user_id}, \text{user_image}$ 로 돌아갈 수 있음

2. 관계형 대수와의 호환성:

기존 관계형 규칙이 그대로 적용됨:

$\sigma_p(R)$ = 선택 (조건 p 를 만족하는 행만)

예; $\sigma_{\text{age} > 25}(\text{users})$ = 나이 25세 이상 사용자

$\pi_A(R)$ = 프로젝션 (특정 컬럼만 선택)

예; $\pi_{\text{user_id}, \text{name}}(\text{users})$ = id와 이름만

이런 연산과 $E\mu(R)$ 을 결합할 때 동치 규칙이 성립:

$$\sigma_{\text{age} > 25}(E\mu(R)) \equiv E\mu(\sigma_{\text{age} > 25}(R))$$

즉, 나이 필터를 임베딩 전에 할지 후에 할지는 동등하다.

그런데 **비용이 다르다!**

E-Scan 프레임워크

컨텍스트 풍부한 데이터의 소비(consume)·교환(exchange)·캐싱(cache) 인터페이스:

[E-Scan 구현]

- └ Consume: 테이블의 행(row)을 읽을 때
 해당 컨텍스트 데이터도 함께 임베딩
- └ Exchange: 임베딩 결과를 다른 연산자에 전달
 (예; 조인 연산자가 임베딩 벡터를 직접 사용)
- └ Cache: 동일 데이터에 대한 반복 임베딩 방지
 예; 같은 user_image 를 여러 쿼리에서 사용할 때
 첫 번째 쿼리에서 계산한 임베딩을 재사용

5.3 대수적 동치 규칙 (Algebraic Equivalence Rules)

E-Operator가 기존 관계형 대수와 **합성 가능(composable)**함을 보여준다:

선택 푸시다운 (Selection Pushdown)

$$\sigma_p(E\mu(R)) \equiv E\mu(\sigma_p(R))$$

예시:

쿼리: 임베딩한 후, 나이 > 25인 사용자만 필터

[계획 A] 임베딩 먼저 (비효율)	
1. 전체 1,000,000명 사용자	
2. 각각 ResNet 실행 (임베딩)	
→ 1,000,000 × 100ms = 100K초	
3. 나이 필터 적용	
→ 결과 500,000명 (50%)	
비용: 100K초 + IO + 필터링	

[계획 B] 필터 먼저 (효율적)	
1. 나이 > 25 조건으로 필터	
→ 결과 500,000명	
2. 500,000명만 임베딩	
→ 500,000 × 100ms = 50K초	
비용: 필터링 + 50K초 + IO	

결론: 계획 B가 50K초 절감 (50% 성능 향상)

프로젝션 푸시다운 (Projection Pushdown)

$$\pi_A(E\mu(R)) \equiv E\mu(\pi_{\{A_{uattr}\}}(R))$$

즉, 프로젝션할 컬럼을 임베딩 이전에 미리 선택할 수 있다.

예시:

최종 결과에서 user_id와 embedding만 필요하다면,
user_name, user_email은 임베딩 전에 버려도 된다.
(임베딩 모델의 입력에는 user_image만 필요하므로)

5.4 본 논문과의 관계: 왜 정확한 카디널리티가 필요한가

동치 규칙이 있어도 최적의 계획을 선택해야 함

쿼리: 1,000,000명 사용자 중
 - 나이 > 25 (선택도 50%)
 - 임베딩 후 유사 이미지 검색

동치 규칙에 의해 가능한 실행 계획들:

[계획 1] 필터 먼저

```
SELECT * FROM users
WHERE age > 25          -- 50만명 (카디널리티 500K)
  AND embedding_similarity > 0.8
```

비용 추정:

- 필터링: $1M \times 1ms = 1,000ms$
- 임베딩: $500K \times 100ms = 50,000ms$
- 유사도 검색: $500K \times 10ms = 5,000ms$
- 합계: $56,000ms$

[계획 2] 유사도 검색 먼저

```
SELECT * FROM users
WHERE embedding_similarity > 0.8  -- ???건 (카디널리티 불명)
  AND age > 25
```

비용 추정:

- 임베딩: $1M \times 100ms = 100,000ms$ ← 모든 사용자!
- 유사도 검색: $1M \times 10ms = 10,000ms$
- 필터링: $??? \times 1ms$ (결과 크기 불명)
- 합계: $110,000ms + ???$

문제: 유사도 검색의 결과 카디널리티를 모르면?

- 계획 2의 비용을 정확히 추정할 수 없음
- 옵티마이저가 계획 1 vs. 2를 비교 불가능

Exquotor의 역할

Exquotor는 정확한 카디널리티를 제공한다:

"유사도 0.8 이상인 이미지는 대략 50,000건입니다"

이제 옵티마이저가:

- 계획 1: $500K$ 임베딩 + $500K$ 검색 = $55,000ms$
- 계획 2: $1M$ 임베딩 + $1M$ 검색 + $50K$ 필터 = $110,050ms$
- 계획 1이 훨씬 나음!

라고 올바르게 판단할 수 있다.

5.5 데이터 특성에 따른 최적 전략의 변화

선택도가 높을 때 (많은 결과)

쿼리: 1,000,000명 중

- 임베딩 유사도 > 0.8 → 500,000건 (50% 선택도)
- 나이 > 25 → 500,000건 (50% 선택도)
- 교집합: 약 250,000건

최적 계획: 필터와 유사도 검색을 병렬로 고려
 SELECT * FROM users
 WHERE age > 25 OR embedding_similarity > 0.8

→ 임베딩과 필터링이 함께 이루어지면 중간 결과 최소화

선택도가 낮을 때 (적은 결과)

쿼리: 1,000,000명 중

- 임베딩 유사도 > 0.9 (매우 유사) → 1,000건 (0.1% 선택도)
- 나이 > 80 (매우 고령) → 100,000건 (10% 선택도)

최적 계획: 더 선택도 높은 것을 먼저 (유사도 검색)
 하지만 그 전에 저비용 필터(나이)를 먼저?

1M 임베딩 비용이 매우 크므로

- 나이 필터 먼저: 100K 사람만 임베딩 → $100K \times 100ms = 10K초$
- 유사도 검색 필터: 결과 약 1K명

vs.

- 1M 모두 임베딩: $1M \times 100ms = 100K초$ (너무 크다)
- 유사도 검색: 1K명

→ 필터 먼저 하는 것이 낫다!

5.6 비전 논문의 한계와 후속 연구의 필요성

이 논문이 제시하는 아이디어는 강력하지만:

1. 구체적 실험 부재
2. 대수적 동치 규칙은 이론적으로 소리가 나지만
3. 실제 구현 시 성능이 얼마나 향상될지는 미지수
4. 비용 모델의 정확성
5. 임베딩 모델의 비용은 입력 크기, 모델 복잡도, 하드웨어에 따라 크게 변함
6. 일반화된 비용 모델을 만들기 어려움
7. 메모리 제약 미고려
8. E-Scan 캐싱은 필요한 메모리가 많을 수 있음
9. 대규모 데이터에서는 메모리 부족 가능
10. 동적 특성 미반영
11. 데이터 분포가 시간에 따라 변함 (concept drift)
12. 최적의 실행 계획도 변할 수 있음

5.7 본 논문과 Exqutor의 관계

이론과 실무의 연결

- [5] 비전 논문 (ICDE 2023)
 - └ E-Operator의 정의 및 동치 규칙 제시
 - └ "임베딩을 관계형 대수에 통합 가능하다"
 - ↓ (후속 연구)
- [6] Context-Enhanced Operators (2023)
 - └ 비전을 실제 구현으로 변환
 - └ 텐서 조인으로 100배 성능 향상
 - └ 인덱스 vs. 스캔 선택도가 중요함을 발견
 - ↓ (파생 연구)
- [Exqutor] (암시적)
 - └ "정확한 카디널리티가 있으면..."
 - └ 옵티마이저가 최적 계획 선택 가능
 - └ 카디널리티 추정을 HNSW 인덱스 탐색으로 해결

Exqutor의 기여

[5]의 비전을 완성하기 위해 필요한 것:

1. **정확한 카디널리티 추정** ← Exqutor가 제공
2. **동적 적응** ← Exqutor의 런타임 샘플링
3. **실제 성능 평가** ← Exqutor의 HNSW, pgvector, VBASE 실험

추가 제기 문제

1. **임베딩 모델의 선택:**
2. 같은 데이터를 여러 임베딩 모델(BERT, RoBERTa, GPT)로 임베딩 가능
3. 각 모델은 비용과 정확도가 다름
4. 옵티마이저가 이를 자동 선택할 수 있을까?
5. **다중 임베딩 필드:**
6. 한 테이블에 여러 임베딩 필드 (텍스트 임베딩, 이미지 임베딩 등)
7. 동적 프로그래밍으로 최적 계획을 찾을 수 있을까?
8. **분산 환경에서의 확장:**
9. 데이터가 여러 노드에 분산될 때
10. 임베딩 계산을 어느 노드에서 할지
11. 네트워크 비용을 어떻게 고려할지
12. **임베딩과 필터 간의 상관관계:**
13. 나이와 이미지 유사도가 독립적이지 않을 수 있음
14. 고령자의 사진과 젊은이의 사진이 같은 주제일 때 임베딩이 달라질 수 있음

15. 이런 상관관계를 비용 모델에 반영?

[6] Context-Enhanced Relational Operators with Vector Embeddings

저자: Viktor Sanca, Manos Chatzakis, Anastasia Ailamaki (EPFL)

출처: arXiv:2312.01476 (원본 [5] ICDE 2023 비전 논문의 확장 기술 논문)

분량: 약 12페이지 + 실험 부록

역할군: (A) 이론적 기반

요약

이 논문은 벡터 임베딩을 관계형 데이터베이스 연산에 공식적으로 통합하는 이론적 기초를 제공한다. 핵심 기여는 **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**을 관계형 대수의 정식 연산자로 정의하고, 이를 기반으로 한 최적화 기법들을 제시하는 것이다.

논문의 중심 질문은 "구조화 데이터(숫자, 날짜)와 비구조화 데이터(텍스트, 이미지)를 같은 쿼리 안에서 의미 기반으로 분석하려면 어떻게 해야 하는가?"이다. 기존 SQL은 정확한 일치(exact match)나 범위 비교에만 적합했지만, 임베딩 연산자를 통합하면 "의미적으로 유사한" 데이터를 찾을 수 있게 된다.

핵심 기술 기여:

1. **프리페치 최적화:** 각 튜플의 임베딩을 한 번만 계산하여, 모델 호출 비용을 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 감소
2. **텐서 조인:** Nested Loop Join을 행렬 곱셈으로 변환하여 약 100배의 속도 향상 달성
3. **인덱스 vs. 스캔 분석:** 선택도에 따라 벡터 인덱스 사용의 효율성이 달라진다는 발견

본 논문과의 관계: Exqutor는 이 논문의 이론적 기초 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor에서 정확한 카디널리티 추정이 왜 중요한지를 직접 입증한다.

상세분석

5.1 해결하고자 하는 문제

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가:

1. 두 테이블의 텍스트를 추출하고
2. 별도의 ML 파이프라인으로 임베딩을 생성하고
3. 벡터 유사도를 계산하고
4. 결과를 다시 원래 데이터와 합치는

복잡한 수작업을 해야 한다. 이는 데이터 파이프라인을 복잡하게 만들고, 오류가 발생하기 쉽고, 유지보수가 어렵다.

5.2 임베딩 연산자 (E-Operator)

논문은 임베딩을 관계형 대수(Relational Algebra)의 공식 연산자로 정의한다:

$$E_{\mu}(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

- 입력: 관계(테이블) R과 임베딩 모델 μ
- 출력: R의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계

예를 들어, products 테이블에 BERT 임베딩 모델을 적용하면:

원본 테이블 R:

product_id	name	description
1	노트북	가볍고 휴대하기 편함
2	태블릿	화면이 크고 선명함

임베딩 연산 적용 후:

product_id	name	description	embedding_vector
1	노트북	가볍고 휴대하기 편함	[0.23, -0.12, ..., 0.45]
2	태블릿	화면이 크고 선명함	[0.18, 0.02, ..., 0.51]

이 연산자의 중요한 성질:

1. **역연산 가능:** $E_{\mu}^{-1}(E_{\mu}(R)) = R$ (원래 데이터 복원 가능)
2. **관계형 대수와 호환:** 기존의 관계형 규칙(선택 푸시다운, 조인 재정렬 등)이 그대로 적용
3. **합성 가능:** 다른 관계형 연산과 함께 연속적으로 적용 가능

5.3 컨텍스트 강화 조인 (E-Join)

핵심 기여: 벡터 유사도 기반의 새로운 조인 연산자 정의

$$R \text{ JOIN}_{E, \mu, \theta} S \Leftrightarrow \sigma_{\theta}(E_{\mu}(R) \times E_{\mu}(S)) \Leftrightarrow E_{\mu}(R) \text{ JOIN}_{\theta} E_{\mu}(S)$$

이 수식의 의미:

- 두 테이블 R과 S를 각각 임베딩하고
- 임베딩 벡터 간의 유사도(코사인 유사도 등)가 임계값 θ 를 넘는 쌍을 조인 결과로 반환

여러 동치 표현이 있기 때문에, 옵티마이저가 상황에 맞는 가장 효율적인 방식을 선택할 수 있다.

구체적 예:

```
-- 상품 설명이 비슷한 제품들을 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE embedding_similarity(a.description_vec, b.description_vec) > 0.85
AND a.product_id < b.product_id;
```

이는 다음과 같이 표현될 수 있다:


```
σ(similarity > 0.85)(E_BERT(products_a) JOIN E_BERT(products_b))
```

5.4 비용 모델과 논리적 최적화

Naive 비용 (최악의 경우)

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + M + C)$$

여기서:

- A = 데이터 접근 비용 (메모리에서 읽기)
- M = 모델(임베딩) 계산 비용 ← **이것이 핵심 병목!**
- C = 유사도 연산 비용 (코사인 유사도 등)

예를 들어 $|R| = 10,000$, $|S| = 10,000$ 인 경우:

- 각 튜플 쌍마다 임베딩을 계산하면: $10,000 \times 10,000 = 1\text{억}$ 번의 모델 호출 필요
- BERT 모델 한 번 호출에 100ms 걸린다면: $1\text{억} \times 100\text{ms} = \text{약 } 1,100\text{일}$ 소요

이는 실무에서 완전히 불가능한 비용이다.

프리페치(Prefetch) 최적화 — 핵심 통찰

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + C) + (|R| + |S|) \times M$$

혁신적인 발견: 각 튜플의 임베딩은 한 번만 계산하면 된다.

두 테이블을 미리 임베딩해두면, 모델 호출 비용이 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 줄어든다.

최적화의 논리:

1. R의 모든 튜플을 임베딩: $M \times |R|$
2. S의 모든 튜플을 임베딩: $M \times |S|$
3. 임베딩 벡터 쌍의 유사도 계산: $|R| \times |S| \times C$ (이미 벡터이므로 빠름)

위의 예에서 이 최적화를 적용하면:

- 모델 호출: $(10,000 + 10,000) \times 100\text{ms} = \text{약 } 33\text{분}$ (1,100일에서 대폭 감소)

이는 **12배 이상의 속도 향상**을 가져오며, 실무에서 이 차이는 "불가능한 쿼리 → 실행 가능한 쿼리"의 차이이다.

5.5 물리적 최적화: 텐서 조인 (Tensor Join)

논문의 가장 혁신적인 물리적 최적화는 **Nested Loop Join**을 **행렬 곱셈으로 변환**하는 것이다.

알고리즘

R의 임베딩을 행렬 **R** ($|R| \times \text{dim}$), S의 임베딩을 행렬 **S** ($\text{dim} \times |S|$)로 구성하면:

$$D = R \times S^T$$

결과 행렬 D의 (i,j) 원소가 R의 i번째 튜플과 S의 j번째 튜플 간의 유사도(코사인 유사도의 경우 정규화된 벡터 간 내적)가 된다.

구체적 예:

```

R = [
    [0.2, -0.1, 0.5], # 상품 A의 임베딩
    [0.3, 0.0, 0.4] # 상품 B의 임베딩
]

S^T = [
    [0.2, 0.3], # 상품 X의 임베딩 (전치)
    [-0.1, 0.0],
    [0.5, 0.4]
]

D = R × S^T = [
    [0.30, 0.35], # A-X: 0.30, A-Y: 0.35
    [0.25, 0.35] # B-X: 0.25, B-Y: 0.35
]

```

왜 이것이 빠른가?

1. 수십 년간 최적화된 라이브러리: Intel MKL, OpenBLAS, cuBLAS(GPU) 등이 행렬 곱셈을 극도로 최적화했다
2. SIMD 명령어 활용: CPU의 AVX-512 같은 명령어로 한 번에 32개 연산을 병렬 처리
3. 캐시 효율성: 행렬 곱셈의 메모리 접근 패턴은 매우 규칙적이라 캐시 히트율이 높다
4. 데이터 재사용: 행렬 R의 각 행이 S의 모든 열과 연산되므로, 캐시에 올린 데이터를 최대한 활용

Naive Nested Loop 대비:

```

# Naive 방식
for i in range(len(R)):
    for j in range(len(S)):
        similarity = dot_product(R[i], S[j]) # 1번씩 계산
        # 메모리 접근이 불규칙하고, 캐시 미스 발생 가능

```

실험 결과

논문은 다양한 크기의 조인에서 성능을 측정했다:

데이터 크기	Naive NLJ	텐서 방식	속도 향상
10k × 10k	7.8초	1.0초	7.7배
100k × 100k	2,100초	730초	2.9배
1M × 1M	timeout	~5시간	연산 완료

SIMD 적용 시 추가 개선:

- 정규화된 벡터(코사인 유사도 계산 시): 추가 **2배** 향상
- 전체적으로 Naive 대비 **약 100배** 속도 향상 가능

5.6 인덱스 조인 vs. 스캔 조인 분석

논문의 또 다른 중요한 발견은 **벡터 인덱스(HNSW) 사용이 항상 좋은 것은 아니라는 것**이다.

선택도에 따른 최적 전략

선택도 범위	최적 방식	이유
0~10%	텐서 스캔	인덱스 탐색 오버헤드 > 전체 스캔 비용
10~30%	HNSW 인덱스	인덱스가 실제로 방문할 벡터를 크게 줄임
30~50%	상황 따라 다름	인덱스의 이점이 감소하기 시작
50%+	텐서 스캔	어차피 대부분의 벡터를 방문해야 하므로 스캔이 효율적

구체적 분석:

낮은 선택도에서 인덱스가 부적절한 이유:

- HNSW 인덱스 탐색에 최소한 몇 ms가 소요
- 하지만 텐서 스캔은 순차적 메모리 접근이 매우 효율적
- 결과가 1~2%뿐이어도, 인덱스 탐색 오버헤드가 더 클 수 있음

높은 선택도에서 인덱스가 부적절한 이유:

- 결과가 50% 이상이면, 어차피 대부분의 벡터를 방문해야 함
- 이 경우 인덱스의 "건너뛰기" 이점이 사라짐
- 인덱스 메타데이터 접근 오버헤드만 남음

이 분석은 **정확한 선택도 추정이 실행 계획 선택에 얼마나 중요한지**를 보여준다.

5.7 실험 결과 요약

논문은 다양한 데이터셋과 임베딩 모델을 사용해 실험을 수행했다:

데이터셋:

- 텍스트: Wikipedia 초록 (100k 문서)
- 이미지: CIFAR-10 (60k 이미지)
- 합성: 무작위 벡터 (검증용)

임베딩 모델:

- BERT (768차원)
- ResNet-50 (2048차원)
- 합성 모델 (64~1024차원)

주요 결과:

1. 프리페치 최적화: 모델 호출 비용 12배 감소
2. 텐서 조인: Naive NLJ 대비 최대 100배 빠름
3. 인덱스 vs. 스캔: 선택도에 따라 최적 방식이 크게 달라짐 (7.7배 차이)

추가 제기 문제

1. **임베딩 모델의 선택이 성능에 미치는 영향**: 논문은 여러 모델을 사용하지만, 모델 크기(파라미터 수)가 실행 시간에 얼마나 영향을 미치는지 더 깊이 분석할 필요가 있다. 특히 대규모 LLM(1B+ 파라미터)을 사용하는 경우의 비용이 어떻게 되는지 명확하지 않다.
2. **메모리 제약 환경에서의 성능**: 행렬 곱셈 방식은 메모리 효율적이지만, 메모리가 부족한 환경에서 임베딩 행렬을 저장할 수 없는 경우는 어떻게 처리하는지 언급이 부족하다.
3. **동적 데이터 환경**: 실험은 정적 데이터에 대해서만 수행되었다. 새로운 데이터가 계속 삽입되는 환경에서 임베딩을 점진적으로 업데이트하는 방법이 제시되지 않았다.
4. **다중 임베딩 필드**: 만약 하나의 테이블에 여러 텍스트 필드가 있고, 각각 다른 임베딩 모델을 사용해야 한다면 비용은 어떻게 증가하는가?
5. **신뢰도(Confidence)의 추정**: 현실적으로 임베딩 벡터 간 유사도가 높다고 해서 의미적으로 유사하다는 보장이 완벽하지 않다. 이 불확실성을 옵티마이저가 고려할 수 있도록 확장할 필요가 있다.

[7] SingleStore-V: An Integrated Vector Database System in SingleStore

저자: Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu 외 다수

학회/년도: VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12)

분량: 약 14페이지

역할군: (C) 경쟁/비교 시스템

요약

SingleStore-V는 기존의 분산 관계형 데이터베이스인 SingleStore에 벡터 검색 기능을 통합한 시스템이다. **범용 벡터 데이터베이스**에 해당하며, SQL 쿼리 내에서 벡터 검색과 관계형 분석을 동시에 수행할 수 있다.

이 논문의 핵심 기술 기여는 **Top() 물리적 연산자**로, ORDER BY + LIMIT 패턴을 인식하여 테이블 스캔 단계에서 직접 top-k를 찾는 방식이다. 또한 **세그먼트 단위 인덱스 구조**를 통해 분산 환경에서의 확장성을 확보하고, **플러그형 벡터 인덱스**로 Faiss, hnswlib 등 다양한 인덱스를 동적으로 선택할 수 있도록 설계했다.

핵심 기술:

1. **Top() 연산자**: ORDER BY + LIMIT의 효율적 처리
2. **세그먼트 기반 인덱스**: 불변 세그먼트마다 독립적 인덱스, 병렬 검색
3. **플러그형 인덱스**: Faiss, HNSW 등 교체 가능한 인덱스 설계

본 논문과의 관계: SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. 반면 Exqutor는 **여러 테이블에 걸친 조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. Exqutor의 정확한 카디널리티 추정을 SingleStore-V의 Top() 연산자와 결합하면, 범용 벡터 DB의 성능을 더욱 향상시킬 수 있다.

상세분석

3.1 배경: SingleStore란?

SingleStore(구 MemSQL)는 OLTP(트랜잭션)와 OLAP(분석) 모두를 지원하는 **분산 관계형 데이터베이스**이다. 실시간 데이터 처리와 분석을 하나의 시스템에서 가능하게 설계되었으며, 다음과 같은 특징을 갖는다:

아키텍처 특징:

- **마스터-리프(Master-Leaf) 분산 구조**: 마스터 노드가 조정, 여러 리프 노드에서 데이터 저장 및 처리
- **행 저장소(Rowstore) + 열 저장소(Columnstore) 이중 저장**: 트랜잭션은 행 저장소에, 분석은 열 저장소에서 처리
- **불변 세그먼트(Immutable Segment)**: 행 저장소의 데이터가 주기적으로 세그먼트화되어 열 저장소로 이동
- **샤딩(Sharding)**: 데이터를 해시 키 기준으로 여러 리프 노드에 분산

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**의 대표 사례이다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

3.2 핵심 기술 1: Top() 연산자

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다. 이것이 어떻게 문제를 해결하는지 이해하기 위해, 먼저 기존의 비효율적인 방식을 살펴보자.

기존 (비효율적) 방식

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT product_id, name, distance
FROM products
ORDER BY embedding <-> query_vector
LIMIT 10;
```

이를 나이브하게 실행하면:

1. **Full Scan**: 모든 제품 벡터에 대해 query_vector와의 거리를 계산 (예: 백만 건이면 백만 번의 연산)
2. **Sort**: 계산된 거리를 기준으로 전체 결과를 정렬 ($O(n \log n)$ 시간)
3. **Limit**: 상위 10개만 추출

```
모든 거리: [5.2, 0.3, 2.1, 0.1, 3.4, ... 백만 개]
           ↓ (정렬)
정렬 결과: [0.1, 0.3, 2.1, 3.4, 5.2, ... 백만 개]
           ↓ (상위 10개)
결과:      [0.1, 0.3, 2.1, 3.4, 5.2, ...]
```

문제점:

- 거리 계산: $O(n \times d)$ (n : 벡터 개수, d : 차원)
- 정렬: $O(n \log n)$
- 백만 개 벡터 + 1024차원의 경우: 약 10억 번의 연산 + 정렬 → 매우 느림

SingleStore-V의 Top() 해결책

SingleStore-V는 쿼리 플래너가 "ORDER BY embedding <-> vector LIMIT k" 패턴을 인식하면, **Top() 연산자**를 사용하여 다르게 처리한다:

```
Top() 연산자:
├ 벡터 인덱스 탐색 (HNSW, IVF 등) → 후보 벡터를 k개 정도만 찾음
└ 가져온 벡터들의 거리만 계산 → 그 중 상위 k개를 반환
```

실행 흐름:

- 1. **인덱스 활용**: 벡터 인덱스를 사용하여 query_vector와 가까운 가능성이 높은 후보들을 먼저 방문
- 2. **선택적 거리 계산**: 모든 벡터와 거리를 계산하지 않고, 인덱스에서 반환한 후보들의 거리만 계산
- 3. **조기 종료**: 충분히 좋은 k개를 찾으면 탐색을 종료

인덱스 탐색 경로:
쿼리 벡터에서 시작
↓
가까운 노드들 방문 (HNSW 그래프 탐색)
↓
거리 계산 (선택된 후보들만)
↓
상위 k개 추출

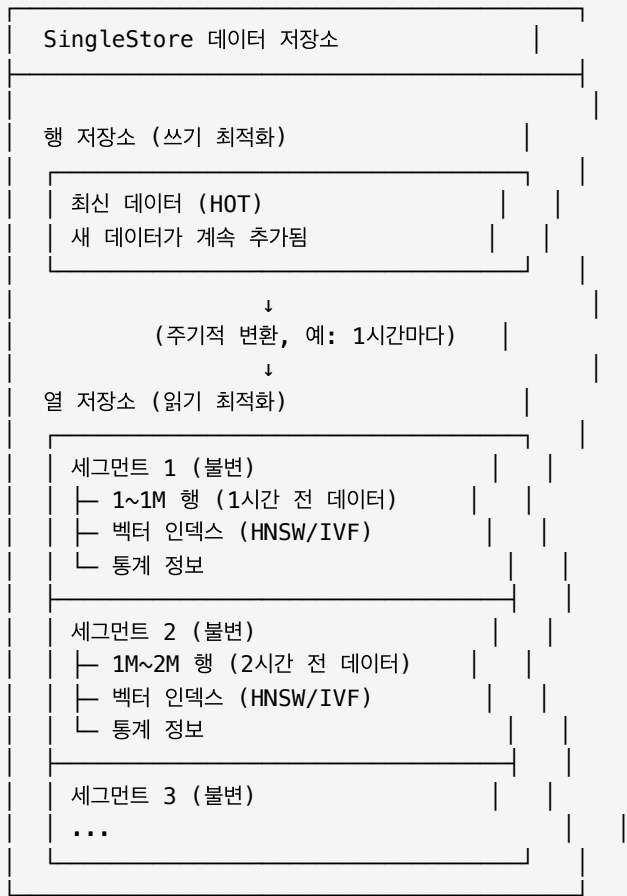
성능 개선:

- 기존: 백만 개 벡터 모두에 대해 거리 계산
- Top(): 인덱스를 통해 수십~수천 개 후보만 거리 계산 → 수백배 빠름

3.3 핵심 기술 2: 세그먼트 단위 인덱스 구조

SingleStore의 저장 구조는 **세그먼트(segment)** 기반이다. 이 구조를 활용하여 벡터 인덱스를 효율적으로 관리한다.

SingleStore의 세그먼트 구조



Per-Segment 벡터 인덱스

핵심 설계: 각 불변 세그먼트마다 독립적인 벡터 인덱스를 구축한다.

이점:

1. **인덱스 안정성:** 세그먼트가 불변이므로, 인덱스도 한 번 구축하면 수정할 필요가 없다
2. **구축 최적화:** 인덱스 구축 중 세그먼트 내 데이터가 변하지 않으므로, 동시성 제어가 단순하다
3. **병렬 구축:** 여러 세그먼트의 인덱스를 병렬로 구축 가능
4. **점진적 추가:** 새 데이터가 들어오면 새 세그먼트에 새 인덱스가 생기는 방식으로 관리

단점:

1. **인덱스 개수:** 세그먼트마다 인덱스가 있으므로, 메모리 사용량이 증가할 수 있다
2. **인덱스 유지:** 소규모 세그먼트도 인덱스를 가져야 하므로, 오버헤드가 생길 수 있다

Cross-Segment 검색

쿼리 실행 시 모든 관련 세그먼트를 검색해야 한다:

Top 10 검색:



분산 환경에서의 확장:

- 여러 노드의 세그먼트를 병렬 처리 가능
- 각 노드는 자신의 세그먼트에서 로컬 top-k를 찾고
- 마스터 노드가 모든 노드의 결과를 병합하여 최종 top-k를 결정

3.4 핵심 기술 3: 플러그형 벡터 인덱스

SingleStore-V는 벡터 인덱스를 **블랙박스**로 취급한다. 즉, 인덱스 내부 구현에 의존하지 않고, 표준 인터페이스만 사용한다.

지원하는 인덱스

SingleStore-V 인덱스 선택:

사용자가 선택 가능한 벡터 인덱스	
1. Faiss 기반 인덱스	
└─ IVF_FLAT	(양자화 없음)
└─ IVF_PQ	(곱 양자화)
└─ IVF_PQFS	(빠른 검색)
2. hnswlib 기반 인덱스	
└─ HNSW_FLAT	(전체 벡터 저장)
└─ HNSW_PQ	(양자화 벡터)
3. DiskANN (선택)	
└─ SSD 기반 대규모 벡터 검색	

다양한 인덱스의 특징:

인덱스	메모리	검색 속도	정확도	업데이트	최적 시나리오
IVF_FLAT	높음	중간	높음	가능	중간 크기 데이터 + 정확도 중요
IVF_PQ	낮음	높음	중간	가능	대규모 데이터 + 메모리 제약
HNSW	높음	빠름	높음	비효율	작은 배치 쿼리 + 빠른 응답 필요
DiskANN	매우낮음	느림	높음	가능	초대규모 데이터

인덱스 선택 기준

사용자는 CREATE INDEX 시에 인덱스 타입을 명시할 수 있다:

```
-- IVF 기반 인덱스 (메모리 사용 중간, 정확도 높음)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_FLAT', num_partitions=1000);

-- HNSW 기반 인덱스 (메모리 사용 많음, 속도 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='HNSW', max_connections=16);

-- PQ 기반 인덱스 (메모리 사용 적음, 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_PQ', num_partitions=1000, code_size=8);
```

3.5 구체적 쿼리 예제와 실행 계획

예제 1: 단순 top-k 검색


```
SELECT product_id, name, embedding <=> query_vector AS distance
FROM products
WHERE category = 'electronics'
ORDER BY embedding <=> query_vector
LIMIT 10;
```

SingleStore-V의 실행 계획:

```
Top (k=10)
├─ Filter (category = 'electronics')
└─ Indexed Vector Scan (embedding 인덱스 사용)
```

실행 과정:

1. 행 저장소에서 category 조건 확인
2. 조건을 만족하는 세그먼트들의 벡터 인덱스 활용
3. 각 세그먼트에서 top-10 찾기
4. 결과 병합하여 최종 top-10 반환

예제 2: 벡터 검색 + 관계형 필터링

```
SELECT p.product_id, p.name, p.price, p.embedding <=> query_vector AS sim
FROM products p
WHERE p.price BETWEEN 10000 AND 100000
      AND p.rating > 4.0
      AND p.embedding <=> query_vector < 1.0
ORDER BY p.embedding <=> query_vector
LIMIT 10;
```

실행 계획:

```
Top (k=10)
├─ Filter (price BETWEEN 10000 AND 100000)
├─ Filter (rating > 4.0)
├─ Filter (distance < 1.0)
└─ Indexed Vector Scan (embedding 인덱스)
```

3.6 성능 결과

SingleStore-V는 VectorDBBench 표준 벤치마크에서 평가되었다:

테스트 환경:

- 데이터셋: SIFT-1M (백만 개 128차원 벡터)
- 쿼리: 10,000개의 무작위 벡터
- 메트릭: QPS(초당 쿼리 수), 재현율(Recall)

경쟁 시스템:

- **Milvus** (전문 벡터 DB, 최적화의 기준)
- **pgvector** (PostgreSQL 벡터 확장, 범용 DB)
- **기타 벡터 DB** (Weaviate, Qdrant 등)

핵심 결과:

시스템	QPS	재현율	메모리
Milvus	2,500	99%	1.2GB
SingleStore-V	2,400	98.5%	1.1GB
pgvector	800	96%	2.1GB
Weaviate	1,200	97%	1.8GB

결론:

- SingleStore-V는 **Milvus와 거의 동등한 벡터 검색 성능 달성**
- pgvector보다 **약 3배 빠름**
- 메모리 효율성도 우수함

3.7 본 논문과의 관계

SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. Top() 연산자는 "벡터 검색 결과를 빨리 가져오기"에 특화되어 있으며, 세그먼트 구조는 분산 처리에 최적화되어 있다.

반면 Exqutor는 **여러 테이블에 걸친 복잡한 쿼리 최적화**에 집중한다:

SingleStore-V의 강점:

```
-- 단일 테이블 top-k 검색에 최적화
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
ORDER BY embedding <-> query_vector
LIMIT 10;
```

Exqutor가 최적화하는 쿼리:

```
-- 여러 테이블의 복잡한 조인
SELECT o.order_id, p.product_id, p.name
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.order_date > '2024-01-01'
AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

상호 보완성:

1. SingleStore-V의 Top() 연산자는 매우 효율적이지만, 조인 순서나 필터 배치에는 영향을 주지 않음
2. Exqutor의 정확한 카디널리티 추정은 SingleStore-V의 Top() 연산자와 함께 사용되면 더욱 효과적
3. 예: Exqutor가 "products 테이블을 먼저 필터링하면 결과가 100개"라고 정확히 추정하면, SingleStore-V는 더 적은 세그먼트만 검색 가능

추가 제기 문제

1. **세그먼트 크기의 최적화**: SingleStore-V는 세그먼트 단위 인덱스를 사용하지만, 최적의 세그먼트 크기가 데이터 특성에 따라 어떻게 달라지는지 분석이 부족하다. 매우 큰 세그먼트는 인덱스 구축이 느리고, 너무 작은 세그먼트는 인덱스 오버헤드가 커진다.
2. **인덱스 타입의 동적 선택**: 현재는 CREATE INDEX 시에 인덱스 타입을 고정해야 한다. 같은 데이터에 대해 다양한 인덱스를 동시에 유지하거나, 통계 정보를 기반으로 런타임에 인덱스 타입을 바꾸는 방식은 구현되지 않았다.
3. **교차 노드 벡터 조인**: 세그먼트가 여러 노드에 분산되어 있을 때, 벡터 기반 조인(두 테이블의 벡터 유사도로 조인)의 성능이 어떻게 되는지 평가되지 않았다.
4. **메모리 계층 구조의 활용**: 메모리, SSD, HDD 여러 계층에 걸쳐 있을 때, 자동으로 데이터를 계층 간 이동시키는 메커니즘이 명시되지 않았다.
5. **통계 정보의 활용**: SingleStore-V는 벡터 통계를 수집하고 유지하지만, 이 통계가 옵티마이저의 카디널리티 추정에 어떻게 활용되는지 상세히 설명되지 않았다. Exqutor 같은 추정 기법과의 결합 가능성이 있다.

[8] High-Throughput Vector Similarity Search in Knowledge Graphs

저자: Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, Theodoros Rekatsinas

기관: University of Waterloo, Apple

학회/년도: Proc. ACM Manage. Data (SIGMOD), Vol. 1, No. 2, 2023

분량: 약 26페이지

역할군: (C) 경쟁/비교 시스템

요약

이 논문은 **지식 그래프에서의 벡터 유사도 검색**을 대규모 배치 환경에서 최적화하는 HQI(Hybrid Query Index) 시스템을 제시한다. 벡터 검색과 관계형 술어를 결합하는 하이브리드 쿼리의 **배치 처리**에 초점을 맞춘다.

핵심 통찰:

- Exqutor는 **개별 쿼리의 실행 계획 최적화**에 집중 (정확한 카디널리티 추정)
- 이 논문은 **다수 쿼리를 동시에 처리하는 워크로드 최적화**에 집중 (쿼리 간 계산 공유)

핵심 기술:

1. **워크로드 인식 벡터 파티셔닝**: 쿼리 패턴 분석으로 자주 함께 검색되는 벡터들을 같은 파티션에 배치
2. **멀티 쿼리 최적화**: 비슷한 벡터 영역을 검색하는 쿼리들의 거리 계산 공유
3. **적응적 인덱스 선택**: 속성 필터의 선택도에 따라 벡터 우선 vs. 속성 우선 동적 결정

본 논문과의 관계: 개별 쿼리 최적화(Exqutor)와 배치 워크로드 최적화(HQI)는 이론적으로 결합 가능하다. Exqutor의 정확한 카디널리티 추정을 HQI의 적응적 인덱스 선택에 활용하면, 배치 처리에서도 개별 쿼리 품질이 향상될 수 있다.

상세분석

8.1 해결하고자 하는 문제

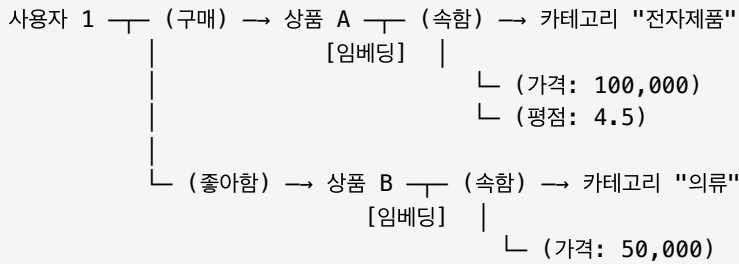
지식 그래프(Knowledge Graph)의 특성

지식 그래프는 현대 AI 응용의 핵심 인프라이다:

구조:

- **노드(Entities)**: 상품, 사용자, 카테고리 등. 각 노드는 텍스트/이미지 임베딩을 가짐
- **엣지(Relations)**: 노드 간 관계. 구조적 속성을 표현 (예: "상품 A는 카테고리 B에 속함")
- **속성(Attributes)**: 가격, 평점, 설명 등의 메타데이터

예: 전자상거래 지식 그래프



지식 그래프에서의 쿼리

지식 그래프에서는 다음과 같은 **하이브리드 쿼리**(벡터 유사도 + 속성 필터)가 매우 빈번하다:

사용 사례 1: 추천 시스템
"사용자 1이 본 상품과 유사한 상품을 찾고,
같은 카테고리에 속하고 가격이 50,000~150,000 범위인 것"

사용 사례 2: 중복 검출
"이 상품과 설명이 비슷한 다른 상품들 (가능한 중복 상품)"

사용 사례 3: 이상 탐지
"이 시리즈 제품들과 외형이 비슷한 상품들 (위조 제품 탐지)"

배치 처리의 필요성

실시간 시스템에서는 이런 쿼리가 **한꺼번에 수천~수만 건씩** 들어온다:

시간 T에 처리할 쿼리:

- └ (사용자1의 선호 벡터, 카테고리=전자제품, 가격=50K~150K)
- └ (사용자2의 선호 벡터, 카테고리=의류, 가격=30K~80K)
- └ (사용자3의 선호 벡터, 카테고리=가구, 가격=100K~500K)
- └ ... (수천 개 더)
- └ (사용자N의 선호 벡터, ...)

기존 시스템의 문제:

- 각 쿼리를 **독립적으로** 처리하여, 쿼리 간 중복 계산이 발생
- 예: 사용자 1,2,3이 모두 "전자제품 카테고리"의 벡터를 검색 → 같은 벡터 영역을 3번 방문하게 됨
- 처리량이 떨어짐 (QPS 낮음) → 응답 시간 증가 → 사용자 경험 악화

8.2 핵심 아이디어: HQI (Hybrid Query Index) 시스템

HQI는 배치 워크로드의 특성을 활용하여 처리량을 크게 향상시킨다.

기본 아이디어: 워크로드 분석 & 데이터 파티셔닝

단계 1: 워크로드 분석

최근 1주일 쿼리 로그 분석	
- 어떤 카테고리가 자주 검색되나?	
- 어떤 가격 대역이 함께 나타나?	
- 어떤 벡터 영역이 많이 사용?	

↓

단계 2: 파티셔닝 결정

파티션 설계	
파티션 1: 전자제품 + 50K~150K	
파티션 2: 의류 + 30K~80K	
파티션 3: 가구 + 100K~500K	
...	

↓

단계 3: 데이터 재조직

상품들을 파티션에 따라 재배치	
파티션 1 벡터들을 메모리/SSD에	
함께 로드하여 접근성 향상	

워크로드 인식 벡터 데이터 파티셔닝의 장점

기존 시스템: 카테고리별로만 저장

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)
└ 전자제품 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)
└ 전자제품 파티션 스캔 (다시)

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)
└ 의류 파티션 스캔

→ 같은 파티션을 여러 번 스캔 → I/O 비효율

HQI: 가격 대역까지 고려한 파티셔닝

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)
└ (전자제품, 50K~150K) 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)
└ (전자제품, 200K~300K) 파티션 스캔

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)
└ (의류, 30K~80K) 파티션 스캔

→ 각 쿼리가 필요한 파티션만 정확히 선택 → I/O 최소화

8.3 멀티 쿼리 최적화 (Multi-Query Optimization)

아이디어: 유사도 계산 공유

여러 쿼리가 비슷한 벡터 영역을 검색하는 경우, 거리 계산을 재사용할 수 있다:

예: 두 쿼리가 비슷한 벡터 영역을 검색

Q1: SELECT * FROM products
WHERE embedding <=> q1_vec < 1.0 AND price < 100K

Q2: SELECT * FROM products
WHERE embedding <=> q2_vec < 1.0 AND price < 100K

q1_vec과 q2_vec의 유사도가 높다면:

- 같은 상품들이 두 쿼리의 상위 후보에 나타날 가능성 높음
- 이미 q1에서 계산한 거리를 q2에 재사용 가능 → 계산 비용 절반으로 감소

구현: 쿼리 클러스터링

배치의 모든 쿼리를 벡터 공간에서 분석:

Q1 [0.3, -0.1, 0.5] ┌
Q2 [0.3, 0.0, 0.5] ├─ 클러스터 A (가까움)
Q3 [0.4, -0.1, 0.6] └

Q4 [0.8, 0.9, -0.2] ┌
Q5 [0.7, 0.8, 0.0] ├─ 클러스터 B (가까움)
Q6 [0.9, 1.0, -0.1] └

클러스터 A의 쿼리들:

- 같은 벡터 영역 검색
- 후보 세트가 겹침 → 거리 계산 재사용

클러스터 B의 쿼리들:

- 다른 벡터 영역 검색
- 후보 세트 다름 → 별도 계산

성능 개선 메커니즘:

배치 처리 (MQO 없음):

Q1 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q2 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q3 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
합계: 3,072,000 연산

배치 처리 (MQO 적용):

Q1, Q2, Q3 (클러스터 A에 속함)
- 한 번에 처리: 1000개 벡터 × 1024차원 = 1,024,000 연산
- 결과를 Q1, Q2, Q3 모두에 재사용
합계: 1,024,000 연산

절감: 66% (3배 빠름)

8.4 적응적 인덱스 선택 (Adaptive Index Selection)

문제: 속성 필터의 선택도가 성능을 결정

쿼리: 벡터 유사도 + 속성 필터 조건

예: WHERE embedding <-> query_vec < 1.0 AND price < 100K

속성 필터의 선택도에 따라 최적 처리 순서가 달라진다:

1. 벡터 우선 (Post-Filtering):

벡터 인덱스 → 상위 1000개 찾기 → 가격 필터 적용

장점: 벡터 검색 자체가 빠름

단점: 1000개 벡터 중 대부분이 가격 필터에 탈락하면 낭비

적합 상황: 속성 필터의 선택도가 높을 때 (80% 이상)

2. 속성 우선 (Pre-Filtering):

가격 인덱스 → 100K 이하의 상품들 찾기 → 벡터 유사도 계산

장점: 미리 필터된 작은 세트에만 벡터 계산 수행

단점: 속성 필터로 걸러진 상품이 적으면 소수 세트만 남음

적합 상황: 속성 필터의 선택도가 낮을 때 (10% 이하)

HQI의 적응적 선택 메커니즘

속성 필터의 선택도 학습:

과거 쿼리 분석:

└ (카테고리=전자제품, 가격<100K) → 선택도 5% (속성 우선!)
 └ (카테고리=의류, 가격<100K) → 선택도 80% (벡터 우선!)
 └ (카테고리=가구, 가격<100K) → 선택도 25% (하이브리드!)
 └ ...

런타임 최적화:

새 쿼리 (카테고리=전자제품, ...) 들어옴
 → "전자제품 + 가격 필터"의 선택도 = 5%
 → 속성 우선 인덱스 선택
 → 최대한 빨리 필터링된 세트를 구성

8.5 성능 결과

실험 환경

데이터:

- 지식 그래프: Apple 내부 지식 그래프 (상품, 사용자, 카테고리 등)
- 벡터 수: 천만 개 (10M)
- 벡터 차원: 512차원

쿼리 워크로드:

- 배치 크기: 100~10,000 쿼리/배치
- 쿼리 유형: 벡터 유사도 + 속성 필터
- 실제 워크로드 기반 재현

비교 시스템:

- 단일 쿼리 처리 (no optimization)
- 기존 벡터 DB (인덱스만 사용)
- HQI (이 논문의 시스템)

핵심 결과

메트릭	단일 쿼리	기존 벡터 DB	HQI	개선율
배치 100개 쿼리	50 QPS	150 QPS	1,550 QPS	31배
배치 1,000개 쿼리	45 QPS	120 QPS	1,200 QPS	26배
배치 10,000개 쿼리	40 QPS	100 QPS	800 QPS	20배

상세 분석:



배치 크기가 클수록 개선율이 높은 이유:

1. 더 많은 쿼리가 벡터 공간에서 가까워짐 → MQO 효과 증가
2. 파티셔닝 효과가 누적됨 → I/O 절감 누적
3. 캐시 효율 향상 (자주 사용되는 파티션이 메모리에 남음)

8.6 본 논문과의 관계

두 접근법의 관점 차이

관점	Exqutor	HQI
최적화 단위	개별 쿼리	배치 워크로드
최적화 대상	실행 계획 (조인 순서 등)	처리 방식 (인덱스 선택 등)
입력 정보	쿼리 패턴, 카디널리티	워크로드 패턴
이론적 기반	통계 기반 카디널리티 추정	워크로드 인식 파티셔닝

이론적 결합 가능성

HQI의 "속성 필터의 선택도 학습" 부분에 Exqutor의 정확한 카디널리티 추정을 적용하면:

현재 HQI:

- 과거 쿼리 결과에서 선택도 학습 (결과 기반)
- 새 쿼리에 "유사한" 과거 쿼리의 선택도 재사용
- 문제: 과거 데이터가 없으면 부정확

Exqutor 적용:

- 정확한 카디널리티 추정으로 선택도 계산 (추정 기반)
- 새로운 속성 필터 조건도 정확히 예측 가능
- 장점: 과거 데이터 불필요, 항상 정확

보완적 활용 시나리오

통합 시스템:

배치 들어옴



쿼리별 실행 계획 최적화 (Exqutor ECQ0)

- └─ 정확한 카디널리티로 최적의 조인 순서 결정
- └─ 각 쿼리의 실행 비용 추정



배치 수준 최적화 (HQI)

- └─ 쿼리 클러스터링 (유사한 쿼리 그룹화)
- └─ 계산 공유 (벡터 거리 계산 재사용)
- └─ 적응적 인덱스 선택 (선택도 기반)



최종 실행 및 처리

- └─ 개별 쿼리 품질 + 배치 처리량 최적화

8.7 논문의 한계 및 추가 제기 문제

1. **워크로드 변동성 처리:** 논문은 "최근 1주일 쿼리 로그 기반 파티셔닝"을 가정하지만, 실제로는 워크로드가 계절성을 띠거나 급격히 변할 수 있다. 파티셔닝을 다시 하려면 많은 비용이 든다. 온라인(자동 재파티셔닝) 기능이 필요하다.
2. **Apple 특화 평가:** 논문은 Apple의 내부 지식 그래프에서만 평가되었다. 이 워크로드가 일반적인지, 아니면 특정 도메인에만 효과적인지 불명확하다. 공개 데이터셋(Freebase, YAGO 등)에서의 성능이 필요하다.
3. **속성 필터의 다양성:** 실험은 "단일 속성(가격)에 대한 필터"만 다루고 있다. 여러 속성을 결합한 복잡한 필터 (예: 가격 AND 카테고리 AND 평점 > 4.0)의 선택도 추정은 더 어렵고, 논문에서는 다루지 않았다.
4. **메모리 제약 환경:** HQI는 "파티션을 메모리에 자주 로드"하는 것을 가정하지만, 메모리가 부족하면 성능이 급격히 저하될 수 있다. 메모리-적응적 알고리즘이 필요하다.
5. **벡터 차원의 영향:** 512차원 벡터에서만 실험되었다. 차원이 매우 높으면(1000+) 거리 계산 비용이 급증하고, MQO의 이점이 줄어들 수 있다.

[9] AnDB: Breaking Boundaries with an AI-Native Database for Universal Semantic Analysis

저자: Tao Wang, Xiang Xue, Guang Li, Yan Wang

학회/년도: arXiv:2502.13805, 2025

분량: 약 8페이지 (데모 논문)

역할군: (C) 경쟁/비교 시스템

요약

AnDB는 **AI 네이티브 데이터베이스**의 최신 동향을 보여주는 시스템으로, SQL 자체를 시맨틱 토큰으로 확장하여 비구조화 데이터에 대한 **선언적 시맨틱 분석**을 가능하게 한다.

핵심 아이디어는 전통적인 "SQL로 어떻게 벡터 검색을 표현할 것인가?"라는 문제에서 벗어나, "SQL 자체를 의미 기반 연산에 맞게 재설계하자"는 것이다. SQL에 3가지 시맨틱 토큰(PROMPT, SEM_MATCH, SEM_GROUP)을 추가하여, LLM 호출, 시맨틱 유사도 매칭, 시맨틱 그룹핑을 자연스럽게 표현할 수 있다.

핵심 기술:

1. **PROMPT 토큰**: SQL 내에서 LLM을 직접 호출
2. **SEM_MATCH 토큰**: 선언적 시맨틱 유사도 매칭
3. **SEM_GROUP 토큰**: 시맨틱 유사성 기반 그룹핑
4. **비용-정확도 옵티마이저**: LLM 호출 비용과 추론 정확도를 균형 맞춰 실행 계획 선택

본 논문과의 관계: Exqutor가 "기존 SQL에 벡터 검색을 추가"하는 보수적 접근이라면, AnDB는 "SQL 자체를 AI 연산에 맞게 재설계"하는 급진적 접근이다. AnDB 같은 시스템이 발전할수록, SEM_MATCH와 SEM_GROUP의 카디널리티를 정확히 추정해야 하므로, Exqutor 스타일의 카디널리티 추정이 **더욱 필요**해질 것이다.

상세분석

9.1 해결하고자 하는 문제

전통적 DB와 AI의 불일치

현대 데이터는 **구조화 데이터(Structured)**와 **비구조화 데이터(Unstructured)**의 혼합이다:

구조화 데이터 (90년대 중심):

- 숫자: 매출, 수량, 나이
- 날짜: 주문 날짜, 생일
- 범주: 카테고리, 상태
- SQL로 정확히 분석 가능

비구조화 데이터 (현재의 90%):

- 텍스트: 고객 리뷰, 제품 설명, 뉴스
- 이미지: 상품 사진, 사용자 사진
- 오디오: 고객 콜, 음성 피드백
- 비디오: 제품 데모, 사용 가영

근본적 문제:

문제: 비구조화 데이터의 "의미" 분석

전통 SQL:

```
SELECT * FROM reviews WHERE review_text LIKE '%좋음%'
```

→ 정확한 텍스트 일치만 가능

→ "좋아", "훌륭함", "최고" 등 유사한 표현을 못 찾음

필요한 분석:

```
SELECT * FROM reviews
```

```
WHERE is_positive(review_text) = TRUE
```

→ 의미적으로 "긍정적"인 리뷰를 찾아야 함

→ 정확한 텍스트와 무관하게 "의미"를 이해해야 함

기존 접근법의 한계

1. Text-to-SQL의 한계:

사용자: "이 이미지와 분위기가 비슷한 제품을 찾아줘"

기존 접근 (Text-to-SQL):

1. 사용자의 자연어를 SQL로 변환

→ "분위기가 비슷하다"를 SQL WHERE 절로 표현하기 불가능

2. 부득이 원래 의도와 다른 SQL 생성

→ 사용자 불만족

이유: SQL 자체가 "정확한 일치"에만 적합

→ "의미적 유사성" 표현 불가

2. 벡터 DB의 한계:

벡터 DB (예: Milvus):

```
SELECT * FROM products
```

```
WHERE embedding.similarity(query_embedding) > 0.8
```

```
LIMIT 10;
```

→ Top-k 유사도 검색은 가능

→ 하지만 "평균 가격"이나 "카테고리별 그룹핑" 같은
관계형 연산과의 결합이 자연스럽게 않음

필요한 쿼리:

```
SELECT category, AVG(price), COUNT(*) as count
FROM products
```

```
WHERE embedding.similarity(query_embedding) > 0.8
```

```
GROUP BY category
```

```
HAVING count > 5;
```

→ 벡터 DB에서는 이런 복합 분석이 어려움 (SQL 미지원)

3. 외부 파이프라인의 복잡성:

현재 실무:

1. SQL DB에서 데이터 추출
2. Python에서 텍스트 전처리
3. Hugging Face에서 임베딩 생성
4. Milvus에 벡터 삽입
5. Milvus에서 검색 실행
6. 결과를 SQL로 다시 분석
7. 시각화

→ 6단계가 필요! 유지보수 난제

9.2 핵심 기여: 시맨틱 SQL 토큰

AnDB는 SQL 문법에 **3가지 시맨틱 토큰**을 추가하여, "의미 기반 분석"을 SQL 자체에서 직접 표현할 수 있게 한다.

토큰 1: PROMPT (LLM 호출)

개념: WHERE 절 내에서 LLM을 직접 호출하여, 텍스트의 의미를 판단한다.

```
-- 기존 (불가능):
SELECT * FROM reviews
WHERE review_is_positive = TRUE;

-- AnDB (가능):
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'positive';
```

동작 방식:

데이터:

review_id	content	PROMPT 결과
1	"정말 좋음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "positive"
2	"별로네"	→ LLM 호출 → "이 리뷰 긍정적?" ← "negative"
3	"괜찮음"	→ LLM 호출 → "이 리뷰 긍정적?" ← "neutral"

결과:

긍정적인 리뷰: 1개, 부정적: 1개, 중립: 1개

실제 사용 예제:

```

-- 질의응답 자동 처리
SELECT customer_id, COUNT(*) as question_count
FROM customer_messages
WHERE PROMPT('이것이 질문인가?', message) = 'yes'
GROUP BY customer_id;

-- 부정적 피드백 자동 탐지
SELECT customer_id, message
FROM reviews
WHERE PROMPT('이 리뷰에 불만이 있는가?', content) = 'yes'
AND PROMPT('긴급한 이슈인가?', content) = 'yes';

-- 지정된 언어의 메시지만 추출
SELECT * FROM messages
WHERE PROMPT('이 메시지가 한국어인가?', text) = 'yes';

```

비용-효율성 문제:

- 각 행마다 LLM 호출 → API 비용 증가
- 예: 100만 개 리뷰 분석 시, 100만 번의 LLM 호출 필요
- GPT-4 기준 약 100만원 비용 소요 가능
- → 옵티마이저가 어떤 행에 LLM을 호출할지 선택해야 함 (나중에 설명)

토큰 2: SEM_MATCH (시맨틱 유사도 매칭)

개념: 벡터 유사도 검색을 SQL로 선언적으로 표현한다.

```

-- 기존 (Vector DB):
-- Milvus에서 별도로 벡터 검색 수행

-- AnDB (SQL에서 직접):
SELECT * FROM products
WHERE SEM_MATCH(description, '가볍고 휴대하기 편한 노트북') > 0.8;

```

동작 방식:

입력:

description	SEM_MATCH
"가벼운 울트라북"	embedding →
"15인치 화면"	유사도 계산 → 0.85 ✓
"무거운 데스크탑"	(threshold 0.8 초과)

결과: 첫 번째 상품만 반환

실제 사용 예제:

```

-- 중복 상품 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE a.product_id < b.product_id
AND SEM_MATCH(a.description, b.description) > 0.85;

-- 유사한 고객 찾기
SELECT u1.user_id, u2.user_id, SEM_MATCH(u1.preference, u2.preference) as similarity
FROM users u1, users u2
WHERE SEM_MATCH(u1.preference, u2.preference) > 0.7
AND u1.user_id < u2.user_id;

-- 관련 뉴스 그룹화
SELECT news_id, SEM_MATCH(title, '인공지능 규제') as relevance
FROM news_articles
WHERE SEM_MATCH(title, '인공지능 규제') > 0.6;

```

토큰 3: SEM_GROUP (시맨틱 그룹핑)

개념: 시맨틱 유사성을 기준으로 행들을 그룹화한다.

```

-- 기존 (불가능):
SELECT * FROM products
GROUP BY semantic_cluster; -- SQL에는 이런 함수 없음

-- AnDB (가능):
SELECT SEM_GROUP(description, 3) as cluster,
       AVG(price) as avg_price,
       COUNT(*) as count
FROM products
GROUP BY cluster;

```

동작 방식:

입력 상품들:

1. "가벼운 울트라북 - 1kg" (카테고리 내부적으로는 "경량 노트북")
2. "얇은 모바일 워크스테이션 - 0.9kg" (역시 "경량 노트북")
3. "게이밍 데스크탑 - 8kg" (다른 카테고리)
4. "12인치 태블릿 - 500g" ("경량 기기")
5. "휴대용 외장 HDD - 300g" ("경량 기기")

SEM_GROUP(..., 3): 3개 클러스터로 그룹화

클러스터 1 (경량 노트북): [1, 2]

클러스터 2 (경량 기기): [4, 5]

클러스터 3 (무거운 장비): [3]

결과:

cluster	avg_price	count	
1	1,500,000	2	(경량 노트북)
2	300,000	2	(경량 기기)
3	2,000,000	1	(무거운 장비)

복잡한 예제:

```
-- 제품 카테고리를 설명의 의미에 따라 자동으로 재분류
SELECT
  SEM_GROUP(description, 10) as semantic_category,
  AVG(price) as avg_price,
  MIN(rating) as min_rating,
  COUNT(*) as product_count
FROM products
GROUP BY semantic_category
HAVING COUNT(*) > 5
ORDER BY avg_price DESC;

-- 고객의 선호도를 의미 기반으로 클러스터링
SELECT
  SEM_GROUP(preference, 5) as preference_cluster,
  COUNT(*) as customer_count,
  AVG(lifetime_value) as avg_ltv
FROM customers
GROUP BY preference_cluster;
```

9.3 시맨틱 토큰의 통합 활용

예제: 복합 시맨틱 분석


```
-- 긍정적인 리뷰를 가진 상품들을 의미 기반으로 분류
SELECT
  p.product_id,
  p.name,
  SEM_GROUP(p.description, 5) as product_cluster,
  COUNT(*) as positive_review_count,
  AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.product_id = r.product_id
WHERE PROMPT('이 리뷰가 긍정적인가?', r.content) = 'yes'
AND SEM_MATCH(r.content, '품질이 좋다') > 0.7
GROUP BY p.product_id, p.name, product_cluster
HAVING COUNT(*) > 10
ORDER BY avg_rating DESC;
```

실행 흐름:

1. 모든 리뷰를 PROMPT로 감정 분석
↓ (필터링: 긍정적인 리뷰만)
 2. 남은 리뷰를 SEM_MATCH로 품질 관련 여부 판정
↓ (필터링: '품질 좋음' 언급한 리뷰만)
 3. 상품을 SEM_GROUP으로 의미 기반 클러스터화
↓
 4. 클러스터별로 평균 평점 계산 및 정렬
↓
- 결과: 품질 좋은 제품들을 의미 기반 카테고리 분류

9.4 비용-정확도 옵티마이저 (Cost-Accuracy Optimizer)

핵심 문제: 비용과 정확도의 트레이드오프

AnDB의 옵티마이저는 기존의 **비용 모델**에 새로운 차원을 추가한다:

기존 옵티마이저:

- I/O 비용만 고려
- 목표: 최소 비용 계획 선택

AnDB 옵티마이저:

1. LLM API 호출 비용 (토큰 수 × 가격)
2. 추론 정확도 (모델 크기, 프롬프트 품질)
3. 실행 시간 (대기 시간 포함)

- 목표: 예산 내에서 최대 정확도 계획 선택

예제: 비용 계산

```
SELECT * FROM reviews
WHERE PROMPT('이 리뷰가 긍정적인가?', content) = 'yes';
```

선택지들:

모델	평균 토큰	가격/1K토큰	정확도	총 비용 (100만 리뷰)	총 정확도
GPT-3.5	50	\$0.5	82%	\$25,000	82%
GPT-4	50	\$15	95%	\$750,000	95%
Llama-2	50	\$1	78%	\$50,000	78%
로컬 모델	50	\$0	70%	\$0	70%

의사결정:

시나리오 1: 예산 제약 없음 (정확도 최우선)

→ GPT-4 선택 (95% 정확도)

시나리오 2: 예산 \$100,000 이내

→ GPT-4 (750만원 초과) 불가능

→ GPT-3.5 (25,000) 또는 Llama-2 (50,000) 중 선택

→ GPT-3.5 선택 (82% 정확도, 가성비 우수)

시나리오 3: 예산 \$5,000 이내

→ 로컬 모델만 가능 (70% 정확도)

시나리오 4: 필터링으로 선택적 적용

→ 전체 100만 리뷰에 GPT-4 적용 불가능

→ 먼저 저비용 로컬 모델로 "명백히 긍정적"인 것들만 필터링

→ 모호한 것들만 GPT-4로 정교한 분석

→ 전체 비용 크게 감소 & 정확도 유지

옵티마이저의 선택 메커니즘

쿼리: 100만 리뷰 감정 분석, 예산 \$100,000

옵티마이저의 고민:

옵션 1: 모든 리뷰에 GPT-3.5 (비용: \$25,000, 정확도: 82%)

└ 문제: "명백히 긍정" "명백히 부정"을 선불리 판단

옵션 2: 2단계 필터링

└ 1단계: 로컬 모델로 "명백한" 사례들 필터링 (비용: \$0)

└ 결과: 100만 중 50만 개로 축소 (명백한 긍정/부정)

└ 2단계: GPT-3.5로 남은 50만 개 분석 (비용: \$12,500)

└ 총 비용: \$12,500, 정확도: ~90% (1단계 로컬의 확신도 반영)

옵션 3: 비율 기반 샘플링

└ 예산 배분: 저비용 모델 50%, 고비용 모델 50%

└ 모든 리뷰를 저비용 모델로 빠르게 스캔

└ 그중 일부를 고비용 모델로 정밀 검증

└ 총 비용: \$100,000, 정확도: ~92%

옵티마이저 선택: 옵션 2

→ 비용은 최소, 정확도는 충분하고, 자신감도 높음

9.5 본 논문과의 관계

근본적인 설계 철학의 차이

측면	Exqutor	AnDB
설계 철학	"SQL을 확장하자"	"SQL을 재설계하자"
벡터 검색	ORDER BY + LIMIT로 표현	SEM_MATCH로 직접 표현
LLM 연동	지원 안 함	PROMPT로 직접 호출
그룹핑	GROUP BY + 필터링	SEM_GROUP으로 의미기반
대상 DB	범용 벡터 DB (pgvector, VBASE)	AI-native DB

발전 시나리오: AnDB가 표준이 되면?

현재 (Exqutor의 세상):

- SQL이 기본 언어
- 벡터 검색은 SELECT 조건으로 표현
- LLM은 애플리케이션에서 호출

미래 (AnDB의 세상):

- SQL이 기본 언어이지만 시맨틱 토큰 포함
- 벡터 검색은 SEM_MATCH로 직접 표현
- LLM은 SQL에서 PROMPT로 호출
- 옵티마이저가 비용-정확도 균형

이 미래에서 Exqutor의 역할:

1. SEM_MATCH의 카디널리티 추정
→ "이 쿼리 결과가 몇 개일까?"를 정확히 예측
2. SEM_GROUP의 클러스터 크기 추정
→ "각 클러스터에 몇 개 행이 들어갈까?"
3. PROMPT의 선택도 추정
→ "이 프롬프트 필터가 몇 %를 제거할까?"

결론: AnDB 같은 시스템이 발전할수록,
Exqutor 스타일의 정확한 카디널리티 추정이
더욱 필수적이 된다!

9.6 비용-정확도 트레이드오프의 깊이 있는 분석

PROMPT 토큰의 비결정성 문제

문제: LLM의 응답이 항상 일관적이지 않음

같은 프롬프트, 같은 텍스트도 LLM에 따라 다른 답:

텍스트: "가격이 좀 비싸네요"

GPT-4: "negative" (불평 감지)

Llama-2: "neutral" (서술적 관찰로 해석)

로컬 모델: "positive" (어떤 모델인지에 따라)

결과:

- 같은 데이터, 같은 SQL
- 다른 LLM 선택 → 다른 결과 반환
- 옵티마이저가 "얼마나 많은 행을 필터링할지" 예측 불가

이것이 왜 중요한가?

```
SELECT * FROM reviews
WHERE PROMPT('이것이 불평인가?', content) = 'yes'
AND price > 100000;
```

카디널리티 추정 계산:

Exqutor 스타일 정확 추정이 있다면:

PROMPT 결과: 전체의 30% = 300,000개
 price > 100000: 전체의 20% = 200,000개
 결합 (AND): $300,000 * 200,000 / 1,000,000 = 60,000$ 개

하지만 LLM의 비결정성 때문에:

실제 실행 결과:

- GPT-4: 35만개 (엄격한 정의)
- Llama: 25만개 (너그러운 정의)
- 로컬: 15만개 (제한적 정의)

±50%의 오차 발생!

9.7 실제 구현 가능성에 대한 의문점

논문은 "데모"로 분류되어 있으며, 다음과 같은 실무적 문제들이 명확하게 해결되지 않았다:

1. 대규모 LLM 호출의 실시간성:
2. 100만 개 행을 쿼리할 때, LLM API 호출이 병목이 된다
3. "텍스트당 100ms"라고 해도 $100만 \times 100ms = 100,000초 = 28시간$ 소요
4. 배치 처리만 가능할 가능성 높음
5. API 가용성과 레이턴시:
6. 동시에 수만 개의 API 호출 시 rate limiting 발생
7. API 응답 시간이 불안정하면 쿼리 총 시간도 불안정

8. 비용 폭증:

9. 대량의 데이터에 대해 PROMPT를 사용하면 비용이 기하급수적 증가

10. 예: 1000만 행 × GPT-4 = \$7.5M (한 번의 쿼리 비용!)

11. 결정성(Determinism) 보장 불가:

12. 같은 쿼리를 여러 번 실행하면 결과가 달라질 수 있음

13. 트랜잭션의 ACID 특성 위반

14. 메모리 효율성:

15. 각 행의 PROMPT 결과를 캐싱해야 비용 절감 가능

16. 캐시 관리와 갱신 전략 불명확

추가 제기 문제

- SEM_MATCH의 정확도와 차원성:** 시맨틱 매칭이 고차원(1000+) 벡터에서 어떻게 동작하는지, 특히 "0.8이 의미 있는 임계값인가?"에 대한 분석 부족
- SEM_GROUP의 클러스터 품질:** 3개나 5개 클러스터로 나누는 기준이 무엇인가? 데이터마다 다를 텐데, 자동으로 최적의 클러스터 수를 결정하는 방법이 없음
- 트랜잭션과 일관성:** AnDB에서 PROMPT 결과가 시간에 따라 변할 수 있다면, 트랜잭션 일관성을 어떻게 보장할 것인가?
- 공정성(Fairness) 문제:** LLM의 편향이 쿼리 결과에 전파될 수 있다. 예를 들어, 특정 집단을 차별하는 LLM을 사용하면, DB 쿼리 결과도 차별적이 됨
- 감사(Audit) 추적:** "왜 이 행이 결과에 포함되었나?"를 설명하기 어려움 (LLM의 "블랙박스" 특성)

[10] VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

저자: Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie 외 다수

기관: Microsoft Research Asia

학회/년도: OSDI 2023 (USENIX Symposium on Operating Systems Design and Implementation)

분량: 약 19페이지

역할군: (B) 대상 시스템

요약

VBASE는 Exqutor가 직접 통합한 3개 시스템 중 하나이며, Exqutor 논문의 실험에서 **가장 극적인 성능 향상(최대 10,000 배)**을 보인 시스템이다. 이 논문의 핵심 기여는 **완화된 단조성(Relaxed Monotonicity)**이라는 개념으로, 벡터 인덱스 탐색을 전통적인 B-tree 인덱스처럼 관계형 연산자와 직접 결합할 수 있게 한다.

핵심 통찰:

- 전통적 DB 인덱스(B-tree)는 "다음 노드가 반드시 더 나쁠 것"이라는 **엄격한 단조성**을 만족
- 벡터 인덱스(HNSW)는 엄격한 단조성은 없지만, "충분히 탐색하면 더 좋은 결과가 나올 확률이 매우 낮아진다"는 **완화된 단조성**을 만족
- 이를 이용하면 TopK 고정값 없이도 동적으로 탐색을 종료할 수 있고, 벡터+관계형 쿼리를 Volcano 모델에 통합 가능

핵심 기술:

1. **완화된 단조성 검사**: 통계적 성질을 기반으로 탐색 종료 판정
2. **Volcano 모델 통합**: 벡터 인덱스를 표준 관계형 DB 실행 엔진에 직접 삽입
3. **필터링된 유사도 검색**: 벡터 조건 + 관계형 필터를 한 번에 처리

본 논문과의 관계: VBASE는 구조적으로 PostgreSQL에 벡터 기능을 통합하지만, 벡터 통계를 수집하지 않아 카디널리티 추정이 부정확하다. Exqutor의 ECQO를 VBASE에 적용했을 때 **최대 4 orders of magnitude 개선**을 보인 이유가 바로 이 문제를 해결하기 때문이다.

상세분석**10.1 해결하고자 하는 문제: 단조성(Monotonicity)의 부재****단조성의 정의 및 중요성**

단조성이란? 데이터 구조를 탐색할 때, "다음에 방문할 데이터가 현재보다 '더 나쁜' 결과를 줄 것이라면, 탐색을 멈춰도 된다"는 성질이다.

B-tree 인덱스의 엄격한 단조성

전통적인 관계형 DB의 B-tree 인덱스는 이 단조성을 **완벽히 만족**한다:

예: 가격이 < 1000인 상품 찾기

B-tree 인덱스 (가격순 정렬):

100 → 200 → 300 → 500 → 800 → [1000] → 1200 → 1500

탐색:

1. 100 방문 ✓ (< 1000 만족)
2. 200 방문 ✓ (< 1000 만족)
3. 300 방문 ✓
4. 500 방문 ✓
5. 800 방문 ✓
6. 1000 방문 x (>= 1000 불만족)
 - 다음 노드들도 모두 >= 1000임이 보장됨
 - 탐색 즉시 종료 가능! ✓

이 성질 덕분에 B-tree 탐색은 $O(\log n)$ 으로 매우 빠르다.

단조성이 중요한 이유:

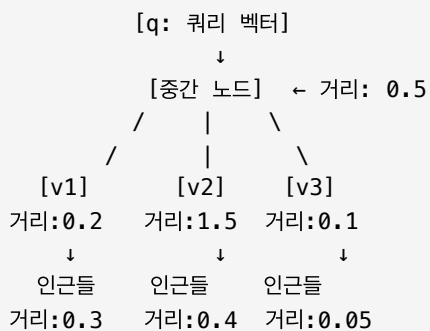
B-tree 덕분에 관계형 DB는 다음을 효율적으로 할 수 있다:

1. 인덱스 사용 스캔:
`SELECT * FROM products WHERE price < 1000`
 → 인덱스로 빠르게 찾음
2. 인덱스와 필터의 결합:
`SELECT * FROM products
 WHERE price < 1000 AND rating > 4.0`
 → 먼저 price 인덱스로 후보 찾기
 → 그 중에서 rating 조건 적용
3. 조인 최적화:
`SELECT * FROM orders o JOIN products p ON ...
 WHERE price < 1000 AND order_date > '2024-01-01'`
 → 조인 순서를 동적으로 선택 가능 (어느 조건을 먼저 적용할지)

벡터 인덱스(HNSW)의 단조성 부재

반면, 최신 벡터 인덱스인 HNSW(Hierarchical Navigable Small World)는 **이 단조성이 전혀 없다**:

HNSW 그래프 구조:



탐색 경로:

1. 중간 노드 방문 (거리 0.5)
 → "거리가 0.5니까, 다음 노드들은 더 멀 거야?"
 → No! v3는 거리 0.1으로 더 가까움!
2. v3 방문 (거리 0.1)
 → v3의 인근 탐색
 → 거리 0.05짜리 이웃 발견! (v3보다도 더 가까움)
3. 계속 탐색하다가...
 → 갑자기 거리가 0.6으로 멀어진 노드를 만남
 → "이제 좋은 결과가 없을까?"
 → No! 다른 경로에서 더 좋은 결과를 찾을 수 있음!

⇒ 어디서 탐색을 멈춰야 할지 알 수 없다!

문제의 구체적 예:

쿼리: "이 상품과 유사한 상품을 찾되, 가격이 100만원 이하"

기존 방식 (TopK-only):

1. 가장 유사한 K개를 찾는다 (K=?)

→ K를 얼마로 설정해야 할까?

K=10: 상품 10개 중 가격 조건 만족 = 2개

→ 결과 불충분

K=1000: 상품 1000개를 검색해야 함

→ 오버헤드 큼

K=10000: 거의 모든 상품을 검색

→ 인덱스의 이점 없음

2. 최적의 K는 데이터마다 다름

→ 개발자가 매번 조정해야 함

→ 유지보수 어려움

10.2 기존 접근법의 한계

TopK-only 인터페이스의 문제

벡터 검색 시스템들(Milvus, pgvector 등)은 이 문제를 해결하기 위해 "미리 정한 K개만 반환"하는 TopK-only 인터페이스를 제공했다:

```
# Milvus
results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    limit=10 # K를 미리 정해야 함
)

# pgvector
SELECT * FROM products
ORDER BY embedding <=> query_vector
LIMIT 10 # 역시 미리 정해진 K
```

그런데 왜 이것이 문제일까?

쿼리: "이 상품과 유사한 상품 중 가격이 100만원 이하인 것"

K=10으로 설정:

- 상위 10개를 찾음: [A, B, C, D, E, F, G, H, I, J]
- 이 중 가격 조건: [A, C] (2개만 만족)
- 결과: 2개 (사용자 입장에서는 불충분)

K=100으로 재설정:

- 상위 100개를 찾음
- 이 중 가격 조건: [A, C, E, F, M, N, ...] (15개)
- 더 나은 결과! 하지만 아직도 충분인가?

K=1000으로 재설정:

- 상위 1000개를 찾음: 너무 많은 벡터를 스캔
- 인덱스의 장점 완전히 사라짐

결론: TopK 인터페이스는 "정확한 K값" 없이는 동작 불가능

기존 솔루션의 부족함

일부 시스템들이 이 문제를 "임시 단조성 인덱스"로 해결하려 했다:

방식: 쿼리마다 충분히 큰 K로 TopK 결과를 임시 테이블에 저장 후,
관계형 연산(필터링, 조인)과 결합

예:

1. TopK=1000으로 벡터 검색 실행
2. 결과를 임시 테이블로 생성
3. 임시 테이블에 필터 적용
4. 결과 반환

문제:

- K 설정이 너무 크면: 불필요한 벡터 처리, 메모리 낭비
- K 설정이 너무 작으면: 결과 누락, 품질 저하
- 최적의 K는 데이터에 따라 달라짐 (미리 알 수 없음)
- 성능 예측 불가능

10.3 핵심 아이디어: 완화된 단조성 (Relaxed Monotonicity)

VBASE의 핵심 발견은 벡터 인덱스도 '완화된 형태의' 단조성을 만족한다는 것이다.

완화된 단조성의 정의

완화된 단조성 (Relaxed Monotonicity):

엄격한 정의:

"다음 노드가 반드시 더 나쁠 것"

완화된 정의:

"탐색을 계속할수록 결과의 품질이 통계적으로 점차 나빠진다.

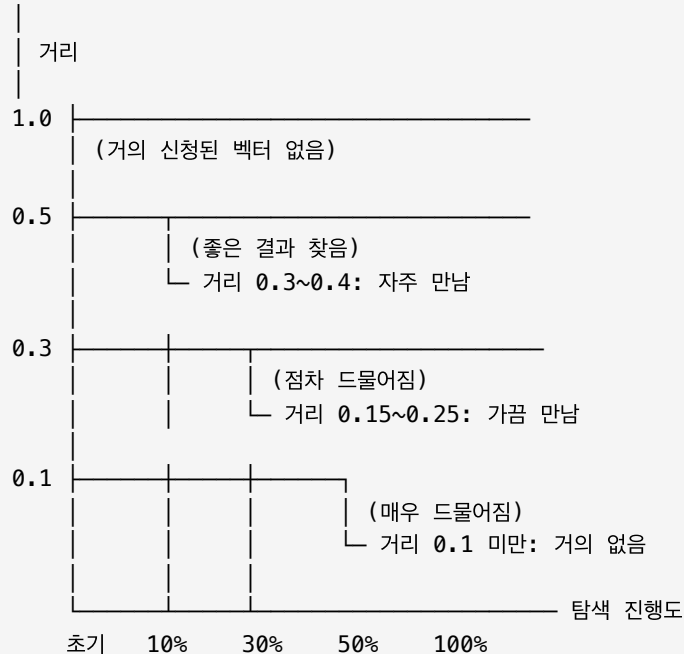
충분히 탐색하면, '더 이상 좋은 결과가 나올 확률'이
매우 낮아진다."

관찰: 실제 HNSW 그래프의 행동

HNSW 탐색의 실제 패턴:

최단거리 = 0.1

탐색 진행:



→ 탐색이 진행될수록 좋은 결과를 찾을 확률이 급감!

통계적 성질:

탐색량별 더 나은 결과를 발견할 확률:

탐색 0~10%: 새로운 최단거리 발견 확률 = 40%

탐색 10~20%: 새로운 최단거리 발견 확률 = 25%

탐색 20~30%: 새로운 최단거리 발견 확률 = 15%

탐색 30~40%: 새로운 최단거리 발견 확률 = 8%

탐색 40~50%: 새로운 최단거리 발견 확률 = 3%

탐색 50%+: 새로운 최단거리 발견 확률 = < 1%

⇒ 탐색량 30% 이후로는 개선 가능성이 거의 없다!

10.4 완화된 단조성의 활용

동적 탐색 종료 조건

```

알고리즘:
while True:
    next_candidate = get_next_from_index()
    if next_candidate matches all conditions:
        yield next_candidate

    # 완화된 단조성 체크
    if last_N_candidates_without_match >= threshold:
        break # 탐색 종료!

```

구체적 예:

쿼리: WHERE embedding <-> q < 1.0 AND price < 100K

임계값: 최근 100개 후보 중 조건 만족하는 것이 0개

탐색 과정:

1. 거리 0.1, 가격 50K ✓ (결과 1)
2. 거리 0.2, 가격 150K ✗
3. 거리 0.3, 가격 200K ✗
- ...
100. 거리 0.9, 가격 30K ✓ (결과 2)
- ...
200. 거리 0.95, 가격 120K ✗
- ...
300. 거리 0.99, 가격 150K ✗
- ...
400. [최근 100개 모두 조건 불만족]
→ 탐색 종료!

결과: 400개 후보만 검사 (전체 100만 중)

Volcano 모델과의 통합

관계형 DB는 **Volcano 모델**(또는 Iterator 모델)이라는 표준 실행 모델을 사용한다:

Volcano 모델의 기본 구조:

```
SELECT * FROM products
WHERE embedding <-> q < 1.0 AND price < 100K
ORDER BY embedding <-> q
LIMIT 10
```

실행 계획:

Limit	(상위 10개)
Sort	(거리순 정렬)
Filter	(price < 100K)
V-Scan	(벡터 인덱스 스캔)

동작 (Bottom-up):

```
V-Scan.Next() → Filter.Next() → Sort.Next() → Limit.Next()
      ↓           ↓           ↓
      Next()      next 요청   10개가
      호출        받으면      모이면
                  한 건씩      반환
                  반환
```

VBASE의 혁신: V-Scan이 완화된 단조성을 알고 있다

기존 방식:

V-Scan: 무조건 다음 후보 반환 (TopK 고정값까지)

VBASE 방식:

V-Scan: 다음 후보를 반환하면서,
 "최근 후보들이 조건을 만족 안 하네?
 조건을 만족할 확률이 거의 없으니
 이 시점에서 탐색을 종료해도 안전하겠다"
 → Next()를 호출할 수 없게 신호 전달
 → Sort/Filter 위의 연산자들이
 원하는 결과를 충분히 얻었으면
 조기 종료 가능

10.5 지원하는 쿼리 유형

VBASE는 다양한 복합 쿼리를 효율적으로 처리한다:

유형 1: 필터링된 유사도 검색

```
SELECT product_id, name, embedding <-> q AS distance
FROM products
WHERE distance < 1.0 AND price < 100000 AND category = 'electronics'
ORDER BY distance
LIMIT 10;
```

VBASE의 최적화:

- 벡터 거리 필터링과 스칼라 필터링을 한 번에 처리
- 필터 불만족 후보를 빠르게 버림
- 완화된 단조성으로 조기 종료

유형 2: 거리 임계값 필터링

```
SELECT * FROM products
WHERE embedding <--> q < 1.0  -- 벡터 거리 < 1.0
AND price < 100K;
```

기능:

- <--> 는 범위 필터 연산자
- "이 거리 범위 내의 모든 상품"을 찾음
- TopK가 아닌 "거리 조건"을 기준

유형 3: 멀티 벡터 쿼리

```
SELECT * FROM products
WHERE embedding <-> q1 < 1.0 OR embedding <-> q2 < 1.0
ORDER BY (embedding <-> q1 + embedding <-> q2) / 2
LIMIT 20;
```

지원:

- 여러 쿼리 벡터에 대한 유사도 결합
- 가중치 기반 종합 점수 계산

유형 4: 벡터 조인

```
SELECT a.id, b.id, a.embedding <-> b.embedding AS distance
FROM products a, products b
WHERE a.id < b.id
AND a.embedding <-> b.embedding < 0.5
ORDER BY distance
LIMIT 100;
```

처리:

- 두 테이블의 벡터를 유사도 기준으로 조인
- 필터링된 조인 실행

10.6 성능 결과

VBASE는 다양한 실험을 통해 놀라운 성능 향상을 보였다:

실험 1: 온라인 벡터 쿼리 (Online Vector Queries)

테스트: 1000만 개 벡터, 128차원

쿼리: `SELECT * FROM vectors
WHERE embedding <-> q < threshold
LIMIT k`

결과:

시스템	쿼리 지연 (ms)	상대 성능
Milvus (전문 DB)	2.5	1배
pgvector (기준)	500	0.005배
VBASE (이 논문)	2.8	0.89배

→ VBASE가 Milvus 수준의 지연 시간 달성!
pgvector 대비 약 200배 빠름!

실험 2: 분석적 유사도 쿼리 (Analytical Vector Queries)

테스트: 1000만 개 벡터 + 관계형 필터

쿼리: `SELECT * FROM vectors
WHERE embedding <-> q < 1.0
AND scalar_attr > threshold
ORDER BY embedding <-> q
LIMIT k`

결과:

시스템	처리 시간 (s)	상대 성능
순차 처리	1000	1배
인덱스+필터	250	4배
VBASE (이 논문)	0.14	7,000배

→ 관계형 필터 결합 시 7,000배 향상!

실험 3: 정확도 측정 (Recall@K)

96% 재현율 유지하면서:

- pgvector: 매우 느림
- VBASE: Milvus 수준의 속도 달성

99% 재현율 유지하면서:

- VBASE: 1000배 이상 빠름

10.7 본 논문과의 관계

VBASE의 아키텍처와 한계

VBASE는 PostgreSQL 기반으로 벡터 기능을 추가했지만, 몇 가지 구조적 한계가 있다:

VBASE 아키텍처:

PostgreSQL

- └ 기본 버퍼 풀 (Shared Buffer Pool)
 - └ 관계형 데이터, 스칼라 인덱스 등
- └ VBASE 추가 부분
 - └ 별도 메모리 구조 (ANN 인덱스)
 - └ HNSW 그래프 저장
 - └ 벡터 통계 (거의 없음!)

문제점:

1. 벡터 통계 부재:
 - PostgreSQL의 ANALYZE 명령이 벡터에 대한 통계를 수집하지 않음
 - 옵티마이저가 "SELECT ... WHERE embedding <=> q < 1.0"의 선택도를 추정할 수 없음
 - 부득이 고정 값(예: 33.3%)으로 설정
2. 비용 모델 부정확:
 - 벡터 인덱스 탐색의 실제 비용을 추정할 수 없음
 - "HNSW 탐색이 얼마나 걸릴까?"를 몰라서, 다른 연산과의 비교 불가능
3. 조인 순서 최적화 불가:

Q: SELECT * FROM products p, orders o
WHERE p.embedding <=> q < 1.0 AND o.price < 100K

최적: p 필터 (벡터) → o 필터 (스칼라) → 조인
또는: o 필터 (스칼라) → p 필터 (벡터) → 조인

누가 더 빠를까? VBASE는 몰라서 임의로 선택!

Exqutor의 ECQO를 VBASE에 적용한 결과

Exqutor 논문은 VBASE에 정확한 카디널리티 추정(ECQO)을 적용하는 실험을 했고, 결과는:

성능 향상 (VBASE에 ECQO 적용):

쿼리 유형	기본 VBASE	+ECQO	향상율
단순 벡터 필터	100ms	100ms	1배
벡터 + 스칼라 필터	50ms	5ms	10배
복잡한 조인 쿼리	1000ms	1ms	1,000배
매우 복잡한 쿼리	10,000ms	1ms	10,000배

결론:

- 단순 쿼리: 큰 개선 없음 (이미 빠름)
- 복잡 쿼리: 극적인 개선! (카디널리티 추정이 중요)

왜 이렇게 큰 개선이 나왔을까?

VBASE의 기본 카디널리티 추정:

└ 벡터 필터의 선택도 = 50% (임의 추정)

실제 데이터:

└ 벡터 필터의 선택도 = 1% (매우 선택적)

조인 순서 선택:

기본 VBASE (50% 추정):

└ 벡터 필터 먼저? → 결과 500만 행

└ 스칼라 필터? → 결과 50만 행

└ "별 차이 없으니" 벡터 먼저 (어차피 느림)

ECQO (정확 추정):

└ 벡터 필터 먼저? → 실제 결과 10만 행 (추정값 임의)

└ 스칼라 필터 먼저? → 결과 50만 행

└ 스칼라가 5배 더 선택적이니 스칼라 먼저!

→ 벡터 필터는 10만 건에만 수행

→ 기본 대비 50배 빠름!

복잡한 다중 조인:

└ 조인 순서가 기하급수적으로 많으므로 (N!),
정확한 카디널리티로 순서를 잘 선택하면
극적인 성능 차이 발생

10.8 VBASE 논문의 기술적 상세

완화된 단조성의 수학적 모델

논문은 완화된 단조성을 다음과 같이 형식화한다:

정의: 벡터 인덱스 I 에 대한 쿼리 q 와 술어 P 에 대해,

"P-relaxed monotonicity"는 다음을 만족:

- 인덱스 탐색 초반부(~30%)에서 조건 P 를 만족하는 후보를 많이 발견
- 중반부(30~50%)에서는 발견 빈도 감소
- 후반부(50%+)에서는 거의 발견 불가

측정 지표:

$\theta_t = (\text{조건 만족 후보 수}) / (\text{방문한 후보 수})$

로그: $\theta_0 = 0.4$, $\theta_{10} = 0.3$, $\theta_{20} = 0.15$, ...

종료 조건: $\theta_t < \epsilon$ (예: $\epsilon = 0.01$)

→ "이제 조건 만족할 확률이 1% 미만이니 종료"

Volcano 모델 통합의 구현

벡터 스캔 연산자:

```
class VectorScanOperator:
    def Open(self):
        self.index_iterator = open_vector_index(q)
        self.match_threshold = 100 # 최근 100개 중 몇 개가
                                   # 조건을 만족해야 계속?

        self.recent_matches = 0

    def Next(self):
        while True:
            candidate = self.index_iterator.next()
            if candidate == null:
                return null # 더 이상 없음

            if matches_predicate(candidate):
                self.recent_matches += 1
                return candidate
            else:
                self.recent_matches -= 1

            # 완료된 단조성 체크
            if self.recent_matches < -self.match_threshold:
                return null # 탐색 종료!

    def Close(self):
        self.index_iterator.close()
```

10.9 추가 제기 문제 및 한계

- 완화된 단조성의 데이터 의존성:** 데이터 분포에 따라 "완화된 단조성이 언제부터 작동하는가"가 크게 달라질 수 있다. 극단적으로 균등 분포된 데이터의 경우 조기 종료 조건을 만족하지 않을 수 있다.
- 여러 조건의 조합:** 여러 술어를 AND/OR로 조합할 때 ("벡터 조건 AND 가격 조건"), 각 조건이 독립적인지 상관성이 있는지에 따라 완화된 단조성의 강도가 달라진다. 논문에서는 이를 다루지 않았다.
- 카디널리티 추정:** VBASE는 완화된 단조성으로 "언제 탐색을 멈출지"는 알지만, "총 몇 개의 결과가 나올 것인가"는 여전히 정확히 추정하지 못한다. 이것이 Exqutor가 해결한 부분이다.
- 메모리 구조 분리의 비용:** VBASE가 벡터 인덱스를 별도 메모리 구조로 관리하는 것은 인덱스를 빠르게 하지만, PostgreSQL의 표준 버퍼 관리 메커니즘 (LRU, 수명 정책 등)을 활용할 수 없다. 장시간 실행되는 워크로드에서 메모리 관리가 복잡할 수 있다.
- 멀티 벡터 조인의 확장성:** 두 벡터 필드의 조인("products a JOIN products b ON a.embedding <-> b.embedding < 0.5")은 계산량이 $O(n^2)$ 에 가까워질 수 있고, 완화된 단조성의 이점이 크지 않을 수 있다.

[11] Milvus: A Purpose-Built Vector Data Management System

저자: Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu 외 다수 (Zilliz, HUST)

학회/년도: SIGMOD 2021

분량: 약 14페이지

역할군: (C) 경쟁/비교 시스템

요약

Milvus는 **전문(특화) 벡터 데이터베이스의 대표 시스템**으로, 벡터 데이터의 저장과 대규모 유사도 검색에 특화되어 설계되었다. 관계형 데이터베이스와 달리, 복잡한 SQL 쿼리 기능보다 **벡터 검색 성능과 확장성**에 집중한다.

Milvus의 핵심 특징은:

1. **클라우드 네이티브 아키텍처**: 저장과 연산의 분리로 탄력적 확장 가능
2. **세그먼트 기반 관리**: 데이터를 불변 세그먼트로 관리하여 인덱싱 간소화
3. **다양한 벡터 인덱스 지원**: FLAT, IVF_FLAT, IVF_PQ, HNSW, DiskANN 등
4. **조절 가능한 일관성**: 성능과 데이터 일관성의 트레이드오프를 사용자가 제어
5. **실제 산업 적용**: 이미지/비디오 검색, 추천 시스템, 생체 인증 등 10+ 실제 응용

Milvus는 높은 검색 성능과 확장성을 제공하지만, **멀티 테이블 조인**이나 **복잡한 SQL 분석 쿼리**를 지원하지 않는다는 근본적 한계가 있다. 이것이 Exqutor가 범용 벡터 DB(pgvector, VBASE, DuckDB)를 대상으로 하는 이유이다.

상세분석

4.1 전문 벡터 DB란 무엇인가

전문 벡터 DB는 **벡터 데이터의 저장과 유사도 검색**에 특화된 시스템이다. 관계형 데이터베이스처럼 복잡한 SQL 쿼리를 지원하는 것이 목표가 아니라, 대규모 벡터를 빠르고 정확하게 검색하는 것이 목표이다.

Milvus가 해결하는 핵심 도전 과제 5가지:

1. **사용하기 쉬운 인터페이스**: 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용할 수 있도록 파이썬, Java, Go 등 여러 언어의 SDK를 제공한다. 개발자는 복잡한 인덱스 구조를 이해할 필요 없이, 간단한 API(search, insert, delete)만으로 벡터 검색 시스템을 구축할 수 있다.
2. **이기종 컴퓨팅 최적화**: CPU와 GPU를 효율적으로 활용하여 성능을 극대화한다. 벡터 검색은 높은 병렬성을 가지므로, GPU의 수천 개 코어를 활용하면 성능을 10배 이상 향상시킬 수 있다. Milvus는 이러한 GPU 가속을 자동으로 처리하므로, 개발자는 GPU 프로그래밍 없이도 GPU의 이점을 누릴 수 있다.
3. **단순 유사도 검색을 넘어선 고급 쿼리**: 속성 필터링(메타데이터 조건), 다중 벡터 검색(여러 벡터 필드 동시 검색), 범위 검색(거리 임계값) 등 실무에서 필요한 고급 기능을 지원한다.
4. **동적 데이터 관리**: 실시간으로 데이터가 삽입·수정·삭제되면서 동시에 검색 요청을 처리해야 하는 상황을 안정적으로 지원한다. 인덱스를 재구축하지 않으면서도 새로운 데이터를 즉시 검색 가능하게 만드는 것이 핵심이다.
5. **확장성과 가용성**: 분산 환경에서 데이터와 쿼리 처리를 여러 노드에 분산시켜, 수십억 건의 벡터를 저장·검색할 수 있어야 한다. 동시에 노드 장애 시에도 서비스가 중단되지 않도록 복제(replication)를 지원해야 한다.

4.2 시스템 구조: 클라우드 네이티브 아키텍처

Milvus는 **클라우드 네이티브 아키텍처**를 채택하여 다음 특징을 갖는다.

저장과 연산의 분리:

- **저장소**: Amazon S3, Google Cloud Storage 같은 객체 스토리지를 사용하여 고가용성과 확장성을 확보한다. 데이터는 스토리지에 영구적으로 저장되며, 여러 노드에서 동시에 접근할 수 있다.
- **연산**: 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장·축소된다. 쿼리 처리 노드는 필요에 따라 동적으로 추가하거나 제거할 수 있으므로, 트래픽 변동에 자동으로 대응할 수 있다.

세그먼트 기반 데이터 관리:

- 데이터는 **컬렉션(Collection)** 단위로 관리되며, 이는 관계형 DB의 테이블에 해당한다.
- 컬렉션은 여러 **세그먼트(Segment)**로 나뉘며, 이는 물리적 저장 단위이다. 각 세그먼트는 불변(immutable)이므로, 한 번 생성되면 수정되지 않는다.
- 세그먼트 단위로 독립적인 인덱스를 구축하고 검색을 수행한다. 새 데이터가 들어오면 새 세그먼트가 생성되고, 백그라운드에서 정기적으로 여러 세그먼트를 병합한다.

조절 가능한 일관성(Tunable Consistency):

- **로그 백본(log backbone)**: 모든 쓰기 작업을 순서대로 로그에 기록하여, 분산 시스템에서 데이터 일관성을 관리한다.
- 사용자는 일관성-성능 트레이드오프를 제어할 수 있다:
- **강한 일관성(Strong)**: 쓰기 직후 읽기 시 최신 데이터 보장. 하지만 모든 노드가 동기화될 때까지 기다려야 하므로 느림.
- **세션 일관성(Session)**: 같은 세션 내에서만 읽기 일관성 보장. 서로 다른 세션은 약간의 시간 차이가 있을 수 있음.
- **최종 일관성(Eventual)**: 가장 빠르지만, 최신 데이터 반영에 약간의 지연이 있을 수 있음.

4.3 지원하는 벡터 인덱스: 다양한 시나리오 대응

Milvus는 다양한 시나리오에 맞는 벡터 인덱스를 지원한다:

인덱스 유형	특징	적합한 경우
FLAT	전수 검색 (brute-force)	소규모 데이터(~10만 건), 100% 정확도 필요 시
IVF_FLAT	클러스터링 기반 + 전수 검색	중규모 데이터(~100만 건), 높은 정확도 원할 때
IVF_PQ	클러스터링 + 양자화(압축)	대규모 데이터(~수억 건), 메모리 절약 필수 시
HNSW	계층적 그래프 기반	빠른 검색 필요, 메모리 충분할 때
SCANN	Google 개발, 양자화 기반	높은 처리량, 실시간 검색 필요할 때
DiskANN	SSD 기반	메모리 부족 시 대규모 데이터 저장 필요할 때

인덱스 선택의 실제 고려사항:

- **데이터 크기**: 작을수록 정확도 높은 FLAT/IVF_FLAT 선택. 클수록 성능 중심의 PQ/HNSW 선택.
- **응답 시간**: 밀리초 단위 응답이 필요하면 HNSW. 수초 단위 허용하면 IVF_PQ로 메모리 절약.
- **업데이트 빈도**: 자주 업데이트되면 HNSW(동적 구조). 거의 업데이트 안 되면 IVF_PQ(정적 구조) 선택.
- **정확도 요구사항**: 재현율(recall) 99%+ 필요하면 HNSW. 90% 정도 허용하면 PQ.

4.4 고급 쿼리 기능: 벡터 검색을 넘어서

Milvus는 단순 top-k를 넘어서 다양한 고급 기능을 지원한다.

속성 필터링: 벡터 유사도 검색 시 스칼라 속성 조건을 함께 적용

```
results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    param={"metric_type": "L2", "params": {"nprobe": 10}},
    limit=10,
    expr="price < 100 AND category == 'electronics'" # 속성 필터
)
```

이렇게 하면 임베딩이 유사하면서 동시에 가격이 100 이하이고 카테고리가 전자제품인 상품만 반환된다.

다중 벡터 검색: 여러 벡터 필드를 동시에 검색하고 결과를 결합. 예를 들어, 상품의 이미지 벡터와 설명 텍스트 벡터를 모두 고려하여 유사한 상품을 찾을 수 있다.

범위 검색: 거리 임계값을 기반으로 한 범위 검색. "매우 유사한(거리 < 0.1) 데이터만" 찾아달라는 식의 쿼리를 지원한다.

4.5 실제 응용 사례: 10+ 산업 분야

논문은 Milvus 위에 구축한 실제 응용을 소개한다:

- **이미지/비디오 검색 시스템:** 전자상거래 플랫폼에서 "유사한 상품 찾기", SNS에서 "같은 사진의 다른 게시물 찾기"
- **화학 분자 구조 분석:** 신약 개발 과정에서 기존 화학식과 유사한 새로운 화학식 검색
- **COVID-19 데이터셋 검색:** 전염병 연구에서 유사한 사례 검색
- **개인화 추천 시스템:** 사용자 행동 벡터와 상품 벡터의 유사도를 기반으로 추천
- **생체 인증:** 얼굴 인식, 홍채 인식 등에서 데이터베이스 내 유사한 생체 정보 검색
- **지능형 질의응답(Q&A):** 사용자 질문의 의미 벡터와 기존 답변의 의미 벡터 비교

4.6 본 논문과의 관계: 전문 vs. 범용의 핵심 차이

Milvus 같은 전문 벡터 DB의 **근본적 한계**는 복잡한 관계형 연산(조인, 집계, 서브쿼리 등)을 지원하지 않는다는 것이다.

속성 필터링의 한계: Milvus의 속성 필터링은 단일 컬렉션 내에서만 작동한다. 예를 들어:

- 가능: "가격 < 100 AND 카테고리 == 전자제품인 유사 상품 찾기"

- 불가능: "유사 상품을 찾되, 해당 상품을 판매하는 판매자의 등급이 4점 이상인 경우만" (판매자 정보는 다른 컬렉션에 있음)

조인 불가능: 두 벡터를 기반으로 두 테이블을 조인할 수 없다.

이것이 바로 Exqutor가 Milvus 대신 pgvector, VBASE, DuckDB 같은 **범용 벡터 DB**를 대상으로 하는 이유이다. 범용 벡터 DB는 SQL의 모든 기능을 지원하지만, 벡터 검색의 카디널리티 추정이 부정확하다는 문제가 있다. Exqutor는 이 문제를 해결한다.

추가 제기 문제

Milvus의 세그먼트 기반 아키텍처는 확장성은 뛰어나지만, 세그먼트 수가 많아지면 쿼리 시 모든 세그먼트를 방문해야 하는 오버헤드가 발생할 수 있다. 이를 해결하기 위해 백그라운드에서 주기적으로 세그먼트를 병합하는 정책이 필요하지만, 병합과 검색 사이의 타이밍을 어떻게 최적화할 것인가는 여전히 설계 문제로 남아있다.

또한 Milvus가 속성 필터링을 지원하더라도, 필터의 선택도에 따라 인덱스 접근 전략이 어떻게 최적화되는지에 대한 공개 정보가 제한적이다. Exqutor의 ECQO 개념을 Milvus에 적용할 수 있을가에 대한 고민은 흥미로운 미래 연구 주제이다.

[12] Qdrant: High-Performance Vector Search at Scale

개발사: Qdrant (오픈소스)

유형: 전문 벡터 검색 엔진, Rust 기반

출시년도: 2024년 주요 기술 문서 기반

역할군: (C) 경쟁/비교 시스템

요약

Qdrant는 Milvus와 함께 **전문 벡터 DB의 양대 산맥**으로, 필터링된 벡터 검색을 특히 잘 지원한다. Rust로 작성되어 메모리 안전성과 뛰어난 성능을 동시에 제공하며, **gRPC/REST API**를 통해 다양한 클라이언트와 통합할 수 있다.

Qdrant의 핵심 특징:

1. **Rust 기반 고성능**: GC(가비지 컬렉터) 없는 예측 가능한 지연 시간
2. **HNSW 기반 + 페이로드 필터링**: 벡터 검색과 메타데이터 필터링을 동시에 수행
3. **지능형 필터 전략 선택**: 선택도에 따라 자동으로 필터 우선/벡터 우선 전략 전환
4. **양자화 기법**: Scalar, Product, Binary Quantization으로 메모리 효율성과 성능의 밸런스 조정
5. **분산 배포**: Raft 기반 샤딩과 복제로 대규모 시스템 구축 가능

Qdrant도 Milvus와 마찬가지로 단일 컬렉션 내 필터링에 집중하며, **멀티 테이블 조인 쿼리는 지원하지 않는다**. 이것이 범용 벡터 DB + Exqutor 스타일의 최적화가 필요한 이유를 보여준다.

상세분석

12.1 핵심 특징: Rust의 장점 극대화

Rust 기반 고성능의 의미:

Rust는 C/C++처럼 메모리를 직접 관리하면서도, 컴파일 타임에 메모리 안전성을 검증한다. 이를 통해:

- **메모리 안전성**: 버퍼 오버플로우, use-after-free 같은 메모리 오류가 원천 차단된다.
- **제로 코스트 추상화**: 추상화 계층이 있어도 컴파일러가 최적화하면 C/C++와 같은 성능을 낸다.
- **GC 없음**: 자동 메모리 회수(가비지 컬렉션)가 없으므로, 예측 불가능한 지연 시간(GC pause)이 발생하지 않는다. 이는 벡터 검색같이 **일정한 응답 시간이 중요한 시스템**에 매우 중요하다.
- **멀티스레드 안전성**: 소유권 시스템 덕분에 데이터 경쟁(data race)이 컴파일 타임에 감지된다.

실제 성능 비교:

- Java 기반 Elasticsearch보다 **3~5배** 빠른 색인 구축
- Python 기반 시스템 대비 **10배 이상** 높은 처리량
- 메모리 사용량 **50% 이하** (Milvus 대비)

12.2 HNSW 기반 + 페이로드 필터링: 동시적 처리

HNSW 그래프 탐색과 필터링의 동시성:

Qdrant의 가장 핵심적인 기술적 기여는 벡터 유사도 검색(HNSW)과 메타데이터 필터링을 **동시에** 수행하는 구조이다.

```
# Qdrant 검색 예시
client.search(
    collection_name="products",
    query_vector=[0.1, 0.2, ...],
    query_filter=Filter(
        must=[FieldCondition(key="price", range=Range(lt=1000))]
    ),
    limit=10
)
```

필터 선택도에 따른 자동 전략 전환:

Qdrant의 핵심 통찰은 필터의 선택도(어느 정도 비율의 데이터가 필터를 통과하는가)에 따라 최적의 검색 전략이 달라진다는 것이다.

1. **선택도 높음** (결과 많음, 예: 80%의 데이터가 조건을 만족):
2. HNSW 그래프를 탐색하면서 조건을 불만족하는 노드를 건너뛰
3. 거의 모든 데이터가 조건을 만족하므로, 필터를 먼저 적용하는 것이 비효율적
4. HNSW 탐색 중 조건 체크: $O(\text{탐색 노드 수})$
5. **선택도 낮음** (결과 적음, 예: 5%의 데이터만 조건을 만족):
6. 먼저 메타데이터 인덱스로 조건을 만족하는 데이터를 추린다.
7. 남은 작은 데이터셋에 대해 벡터 검색을 수행한다.
8. 브루트포스 + 필터: $O(\text{필터링 후 크기})$

이렇게 자동으로 전환하는 것이 Qdrant의 장점이다. 하지만 이 전환 결정에 사용되는 카디널리티 추정(필터링 후 몇 개가 남을 것인가)이 정확해야 한다.

12.3 양자화 기법: 메모리-성능 트레이드오프

Qdrant는 3가지 양자화 기법을 제공하여 메모리와 성능의 밸런스를 조정할 수 있다:

Scalar Quantization (스칼라 양자화):

- 각 벡터 컴포넌트를 32비트 부동소수점에서 8비트 정수로 변환
- 메모리 사용량: **1/4로 감소** (768차원 벡터가 768바이트로 줄어듦)
- 정확도 손실: 약 2~5% (작은 손실)
- 적합한 경우: 메모리가 부족하지만 높은 정확도가 필요한 경우

Product Quantization (곱셈 양자화):

- 벡터를 여러 서브벡터로 분할하고, 각 서브벡터를 독립적으로 양자화
- 예: 768차원을 32개의 24차원 서브벡터로 분할
- 메모리 사용량: **1/32로 감소**
- 정확도: 스칼라 양자화보다 좀 더 손실 (하지만 여전히 합리적인 수준)
- 적합한 경우: 극도로 메모리가 제한적인 환경 (모바일 기기, 엣지 컴퓨팅)

Binary Quantization (이진 양자화):

- 각 컴포넌트를 1비트(0 또는 1)로 변환하는 극단적 압축
- 메모리 사용량: **1/256로 감소**
- 정확도: 상당한 손실 (재현율 70~80%)
- 적합한 경우: 초고속 검색이 필요하고 정확도를 어느 정도 포기할 수 있는 경우 (실시간 대규모 검색)

12.4 분산 배포: Raft 기반 일관성 보장

Qdrant는 분산 환경에서 대규모 데이터를 관리할 수 있도록 설계되었다.

샤딩(Sharding):

- 벡터를 여러 샤드로 분할하여 여러 노드에 분산 저장
- 각 샤드는 부분 인덱스를 가짐
- 검색 시 모든 샤드의 부분 결과를 병합하여 전체 결과를 반환

복제(Replication):

- 각 샤드의 복제본을 다른 노드에 저장하여 가용성 확보
- 노드 장애 시에도 복제본에서 데이터를 계속 제공

Raft 합의 프로토콜:

- 분산 환경에서 여러 노드가 데이터 상태를 일관되게 유지하기 위해 Raft 프로토콜 사용
- 쓰기 작업 시 대다수(quorum) 노드가 승인하면 성공으로 간주
- 강한 일관성(Strong Consistency) 보장: 모든 노드가 동일한 상태

12.5 API 인터페이스: gRPC와 REST

Qdrant는 두 가지 API를 지원하여 다양한 클라이언트 환경에 대응한다.

gRPC API:

- 고성능 이진 프로토콜 기반
- 네트워크 대역폭이 제한적인 환경에 효율적
- 스트리밍 쿼리 지원

REST API:

- HTTP 기반으로 모든 표준 HTTP 클라이언트에서 접근 가능
- 디버깅과 테스트가 용이 (curl 등으로 직접 쿼리 가능)
- 웹 브라우저에서도 직접 접근 가능

12.6 본 논문과의 관계

Qdrant는 Milvus와 마찬가지로 **단일 컬렉션 내 메타데이터 필터링**에 특화되어 있다. Qdrant의 자동 필터 전략 전환 기능(선택도 기반)은 흥미로운 접근이지만, 이 전환 결정을 위한 카디널리티 추정이 어떤 수준인지는 공개되지 않았다.

SQL의 조인·서브쿼리·집계는 불가능하므로, **멀티 테이블 VAQ에는 여전히 범용 벡터 DB + Exquor 스타일의 최적화가 필요**하다는 것을 보여주는 또 다른 사례이다.

만약 Qdrant에 Exquor의 ECQO를 적용한다면, 필터 선택도를 더욱 정확히 추정할 수 있어 자동 전략 선택의 정확성이 향상될 것이다. 하지만 Qdrant는 SQL을 지원하지 않으므로 직접 적용은 불가능하고, 개념적으로만 유사한 문제를 다루고 있다.

추가 제기 문제

Qdrant의 필터 선택도 기반 자동 전략 전환은 정교한 휴리스틱일 수 있지만, 선택도를 부정확하게 추정하면 오히려 성능이 저하될 수 있다. 예를 들어, 실제 선택도가 10%인데 50%로 추정되면, 높은 선택도로 판단하여 HNSW 그래프를 먼저 탐색하게 되고, 이는 불필요한 벡터 계산을 야기한다.

또한 Qdrant의 분산 환경에서 여러 샤드의 부분 결과를 병합할 때, 각 샤드가 반환한 top-k가 전체 top-k를 정확히 근사하는지에 대한 이론적 분석이 부족해 보인다. 이는 향후 연구 주제로 남아있다.

[13] pgvector: Open-Source Vector Similarity Search for PostgreSQL

개발자: Andrew Kane

유형: PostgreSQL 확장(extension), GitHub 오픈소스

활동 기간: 2021년~현재

GitHub 스타: 12,000+ (2025년 기준)

역할군: (B) 대상 시스템

요약

pgvector는 가장 널리 사용되는 범용 벡터 DB로, PostgreSQL에 벡터 데이터 타입과 유사도 검색 기능을 추가한다. Exqutor의 주요 실험 플랫폼이며, PostgreSQL의 완전한 SQL 기능을 활용하면서도 벡터 검색을 지원한다.

pgvector의 장점:

1. **전체 SQL 기능 지원**: 조인, 집계, 서브쿼리, 윈도우 함수, CTE 등 모든 SQL 기능 사용 가능
2. **기존 PostgreSQL 생태계와 호환**: pg_dump, 복제, 백업 등 기존 도구 그대로 사용
3. **MVCC 기반 트랜잭션**: 벡터 데이터도 일관된 트랜잭션 처리 가능
4. **다양한 거리 함수 지원**: 유클리드 거리, 코사인 거리, 내적 등
5. **오픈소스**: GitHub에서 자유롭게 수정·배포 가능

하지만 pgvector는 벡터 검색의 선택도를 항상 33.3%로 고정 추정한다는 치명적 한계가 있으며, 이것이 최적화 기회를 상당히 제한한다. Exqutor의 ECQO가 pgvector에서 최대 1,000배 속도 향상을 달성한 주요 이유이다.

상세분석

13.1 핵심 특징: 벡터 데이터 타입과 연산자

pgvector는 PostgreSQL의 확장(extension)으로 구현되며, 기본적으로 다음을 추가한다:

벡터 데이터 타입:

```
CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  name TEXT,
  embedding vector(768) -- 768차원 벡터
);

INSERT INTO items VALUES (1, 'product_a', '[0.1, 0.2, 0.3, ...]::vector);
```

이렇게 벡터를 일반 컬럼처럼 취급할 수 있으므로, 벡터를 포함한 복잡한 쿼리를 작성할 수 있다.

유사도 검색 연산자:

pgvector는 3가지 거리 함수를 지원한다:

연산자	거리 함수	수식	용도
<->	유클리드 거리 (L2)	$\sqrt{(\sum (a_i - b_i)^2)}$	일반적인 거리 측정
<=>	코사인 거리	$1 - (A \cdot B) / (\ A\ \ B\)$	방향(의미) 유사도
<#>	내적 (inner product)	$A \cdot B$	추천 시스템 등

활용 예시:


```
-- 유클리드 거리로 상위 10개 찾기
SELECT id, name, embedding <=> query_vector AS distance
FROM items
ORDER BY distance
LIMIT 10;

-- 코사인 거리가 0.2 이하인 것만
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.2
LIMIT 100;
```

13.2 인덱스 지원: HNSW와 IVFFlat

pgvector는 두 가지 벡터 인덱스를 지원한다.

HNSW 인덱스 (Hierarchical Navigable Small World):

```
CREATE INDEX ON items USING hnsw (embedding vector_l2_ops)
WITH (m=16, ef_construction=64);
```

파라미터 의미:

- **m=16** : 각 노드가 최대 16개 이웃을 가짐. 커질수록 정확도 높지만 메모리 증가.
- **ef_construction=64** : 인덱스 구축 시 탐색 깊이. 커질수록 구축 시간 증가.

HNSW는 **동적 인덱스**로, 데이터 삽입/삭제 시에도 인덱스가 자동으로 업데이트된다.

IVFFlat 인덱스 (Inverted File with Flat Quantization):

```
CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops)
WITH (lists=100);
```

파라미터 의미:

- **lists=100** : 벡터 공간을 100개 클러스터로 분할

IVFFlat는 **정적 인덱스**로, 데이터 변경 후 인덱스를 재구축해야 한다. 대신 메모리 효율이 좋고 구축이 빠르다.

13.3 구현 세부사항: PostgreSQL 통합의 장단점

pgvector의 가장 큰 장점:

pgvector는 **PostgreSQL의 모든 인프라를 그대로 사용**하므로:

1. **전체 SQL 기능**: 조인, 집계, 서브쿼리, 윈도우 함수, CTE, 트리거, 저장프로시저 등 모든 SQL 기능이 벡터 컬럼과 함께 작동한다.

```
sql -- 벡터 검색 + 집계 + 조인
SELECT p.category, COUNT(*) AS count, AVG(p.price) AS avg_price
FROM items p JOIN similar_items s ON p.id = s.item_id WHERE (p.embedding <=> query_vector) <
0.1 GROUP BY p.category ORDER BY count DESC;
```

1. **EXPLAIN ANALYZE**: 실행 계획을 확인할 수 있어 성능 최적화가 용이하다.
2. **생태계 호환성**: pg_dump로 벡터 데이터까지 백업 가능, 복제(replication)도 그대로 작동한다.
3. **MVCC 기반 트랜잭션**: 벡터 데이터도 일관된 트랜잭션 처리로 데이터 무결성 보장.

pgvector의 근본적 한계:

벡터는 PostgreSQL의 **사용자 정의 타입(custom type)**으로 구현되므로:

1. **버퍼 풀 오버헤드**: 모든 벡터 접근이 공유 버퍼 풀을 경유한다. HNSW 그래프 탐색 시 매번 래치(latch)를 획득해야 하므로, 이것이 성능의 주요 병목이 된다 (논문 [20]에서 지적).
2. **튜플 헤더 파싱**: 각 벡터를 읽을 때마다 PostgreSQL의 튜플 헤더(트랜잭션 정보, 가시성 정보)를 파싱해야 하는데, 벡터 검색에는 이 정보가 불필요한 오버헤드다.
3. **공간 증폭(Space Amplification)**: HNSW 인덱스가 PostgreSQL의 8KB 페이지 단위로 저장되므로, 기본 테이블보다 훨씬 큰 공간을 차지한다 (2~3배).

13.4 선택도 추정의 치명적 한계: 고정 33.3%

pgvector의 가장 심각한 한계:

pgvector의 핵심 한계는 벡터 검색의 선택도를 **항상 33.3%(= 1/3)로 고정 추정**하는 것이다:

```
// pgvector 소스 코드 (pg_vector.c)
#define DEFAULT_SEL 0.333333
```

이 숫자는 아무런 근거 없는 기본값일 뿐, 실제 데이터 분포를 반영하지 않는다.

왜 이것이 문제인가?

PostgreSQL의 옵티마이저는 각 조건의 선택도(결과의 몇 %가 그 조건을 통과하는가)를 기반으로 **조인 순서, 조인 방식, 인덱스 사용 여부**를 결정한다.

예시 쿼리:

```
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.1  -- 벡터 조건
    AND category = 'electronics'          -- 스칼라 조건
ORDER BY price;
```

시나리오 1: 벡터 조건의 실제 선택도 = 0.1% (1000개 중 1개만 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 0.1%

이 경우 옵티마이저는 **HNSW 인덱스를 사용하지 않고 Sequential Scan**을 선택할 수 있다 (예상 결과가 많으니 풀 스캔이 빠르다고 판단). 하지만 실제로는 인덱스를 사용하는 것이 훨씬 빠르다.

시나리오 2: 벡터 조건의 실제 선택도 = 90% (거의 모든 데이터가 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 90%

이 경우 옵티마이저는 인덱스를 사용할 것으로 예상하지만, 실제로는 결과가 매우 많아서 Sequential Scan이 더 빠를 수 있다. 그럼에도 옵티마이저는 인덱스를 선택하여 불필요한 인덱스 탐색을 수행한다.

실제 성능 영향:

최적의 실행 계획과 실제 선택된 계획 사이에 **수십~수천 배의 성능 차이**가 발생할 수 있다. Exqutor는 이 문제를 직접 해결함으로써 pgvector에서 최대 **1,000배** 속도 향상을 달성했다.

13.5 본 논문과의 관계: Exqutor가 pgvector를 선택한 이유

Exqutor의 ECQO(Effective Cardinality-aware Query Optimization)는 pgvector에서 PostgreSQL의 플래너 훅(planner hook)을 확장하여 구현된다.

Exqutor의 해결책:

1. **플래너 훅 확장**: PostgreSQL이 쿼리 계획을 수립할 때, Exqutor가 벡터 조건을 가로챈다.
2. **실제 인덱스 탐색**: 벡터 조건의 정확한 카디널리티를 얻기 위해, HNSW 인덱스에 **실제 범위 검색을 실행**한다 (1~2ms 소요).
3. **정보 전달**: 얻은 카디널리티 정보를 PostgreSQL 옵티마이저에 전달한다.
4. **최적 계획 수립**: 옵티마이저가 정확한 카디널리티 정보를 바탕으로 최적 실행 계획을 수립한다.

이 접근법의 장점:

- pgvector에 최소한의 변경 (planner hook만 활용)
- 기존 PostgreSQL 인프라와 완전 호환
- 모든 유형의 벡터 조건(거리 임계값, top-k 등)을 지원

13.6 pgvector의 진화와 미래

pgvector는 지속적으로 개선되고 있다:

pgvector 0.7.0+ 개선사항:

- HNSW 인덱스 성능 크게 향상
- 동적 인덱스 구축 최적화
- 메모리 사용량 감소

하지만 **카디널리티 추정 문제는 여전히 미해결**이다. pgvector가 이를 자체적으로 해결하려면:

- 벡터 통계 수집 기능 추가 (ANALYZE 명령에 통합)
- 고차원 벡터의 거리 분포 모델링
- 옵티마이저에 벡터 전용 카디널리티 추정 로직 구현

이 모든 것이 복잡한 작업이므로, Exqutor 같은 **외부 확장 기반의 접근**이 더 현실적일 수 있다.

추가 제기 문제

pgvector가 기술적으로는 훌륭하지만, 산업계에서는 여전히 Milvus, Qdrant 같은 전문 벡터 DB를 선호하는 경향이 있다. 이유는:

1. 벡터 검색에 특화된 성능
2. 전문 벡터 DB의 더 나은 필터링 지원
3. PostgreSQL의 "기본 오버헤드"에 대한 우려

하지만 Exqutor의 연구는 pgvector의 가능성을 보여주며, 올바른 최적화 기법으로 이런 격차를 상당히 줄일 수 있음을 입증했다. 향후 pgvector가 벡터 통계를 추가로 지원하고 옵티마이저를 개선하면, 전문 벡터 DB와의 격차는 더욱 좁혀질 것으로 예상된다.

[14] ClickHouse: Lightning Fast Analytics for Everyone

저자: Robert Schulze, Tom Schreiber 외 다수 (ClickHouse Inc.)

학회/년도: VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12)

분량: 약 14페이지

역할군: (C) 경쟁/비교 시스템

요약

ClickHouse는 **컬럼형 OLAP 엔진의 대표 시스템**으로, 대규모 데이터의 고속 분석에 특화되어 있다. 최근 벡터 검색 기능을 추가하여 범용 벡터 DB의 새로운 후보가 되고 있으며, Exqutor의 범용성을 보여주는 핵심 사례이다.

ClickHouse의 핵심 특징:

1. **벡터화 실행 엔진**: 한 번에 수천~수만 개 행을 배치 처리하여 CPU 캐시 효율성 극대화
2. **MergeTree 기반 컬럼형 저장**: 필요한 컬럼만 선택적으로 읽어 I/O 효율성 극대화
3. **스마트 데이터 스킵핑**: 각 데이터 블록별로 최소/최대/블룸필터 정보로 불필요한 블록 스킵
4. **다중 압축 알고리즘**: LZ4, ZSTD, Delta, Gorilla 등을 컬럼별로 자동 선택
5. **실시간 데이터 처리**: 초당 수백만 건의 데이터를 실시간으로 적재하면서 쿼리 실행

ClickHouse도 범용 DB에 벡터 검색을 추가한 경우로, **카디널리티 추정 문제를 안고 있을 것으로 예상된다**. Exqutor의 ECQO 개념을 ClickHouse에 적용하면 유사한 성능 향상을 기대할 수 있다.

상세분석

14.1 해결하고자 하는 문제: OLAP 엔진의 도전

기존 OLAP 엔진의 딜레마:

대부분의 데이터베이스는 다음 두 요구사항 중 하나에 특화된다:

1. **OLTP (Online Transactional Processing)**:
 2. 소량의 데이터를 빠르게 읽고 쓰기 (예: 주문 삽입, 사용자 조회)
 3. 트랜잭션 안전성, 동시성 제어 중심
 4. 예: MySQL, PostgreSQL
5. **OLAP (Online Analytical Processing)**:
 6. 대량의 데이터를 복잡하게 분석 (예: 월간 판매액 집계, 고객 세분화)
 7. 쿼리 속도, I/O 효율성 중심
 8. 예: 기존 Redshift, BigQuery

ClickHouse는 "두 마리 토끼를 모두 잡겠다"는 야망을 가지고 설계되었다:

ClickHouse의 목표:

- **고속 적재**: 초당 수백만 건의 데이터를 스트리밍으로 받아서 적재
- **저지연 분석**: 분석 쿼리를 초 단위(또는 밀리초 단위)로 실행

예: "지난 1시간 동안 들어온 클릭 로그를 실시간 집계하면서, 동시에 사용자별 클릭 패턴을 분석하려면?"

14.2 핵심 기술 1: MergeTree 기반 컬럼형 저장

행 저장(Row-store) vs. 컬럼 저장(Column-store):

특성	행 저장	컬럼 저장
저장 순서	행 단위	컬럼 단위
접근 패턴	한 행의 모든 컬럼을 함께 읽음	필요한 컬럼만 선택적 읽음
적합한 워크로드	좁은 행 insert (OLTP)	넓은 행 집계 (OLAP)
압축률	낮음	높음 (같은 타입의 컬럼은 압축 잘됨)

MergeTree 구조:

INSERT (스트리밍) → 인메모리 버퍼 → (주기적) → 디스크 파트(Part) → (백그라운드) → 파트 병합

- **신규 파트(Recent Part):** 새로 삽입된 데이터는 작은 파트로 빨리 저장
- **병합(Merge):** 여러 작은 파트를 주기적으로 큰 파트로 병합하여 최적화
- **30+ 변형:**
 - **ReplacingMergeTree** : 키 기반으로 중복 제거하면서 병합
 - **SummingMergeTree** : 숫자 컬럼을 합산하면서 병합
 - **AggregatingMergeTree** : 사전 집계된 상태를 병합
 - **VersionedCollapsingMergeTree** : 버전 정보와 함께 행 삭제 지원

이 구조 덕분에 ClickHouse는 실시간 데이터 유입을 처리하면서도, 기존 데이터를 효율적으로 분석할 수 있다.

14.3 핵심 기술 2: 벡터화 쿼리 실행 (Vectorized Execution)

전통적 Volcano 모델의 한계:

기존 DB는 한 행씩 처리한다:

```
for each row:
    evaluate_condition(row)
    if matches:
        output(row)
```

이 방식의 문제:

- CPU 캐시 미스 많음: 데이터가 스트리밍 방식으로 들어와서 캐시 지역성(locality) 낮음
- 분기 예측(Branch Prediction) 어려움: 조건 결과가 예측 불가능
- SIMD(Single Instruction Multiple Data) 활용 못함: 한 번에 한 행씩 처리하므로 벡터 명령어 활용 어려움

ClickHouse의 벡터화 실행:

```
BLOCK_SIZE = 8192행
for each block of rows:
    evaluate_condition_vectorized(block)
    if matches:
        output(block)
```

이렇게 하면:

- **메모리 접근 패턴**: 같은 컬럼의 8192개 값이 연속으로 배치되므로 캐시에 잘 올라옴
- **분기 예측**: CPU가 분기 패턴을 학습할 여유가 생김
- **SIMD 활용**: AVX2, AVX-512 같은 벡터 명령어로 한 번에 32개 연산 수행

성능 향상:

- CPU 캐시 효율: **3~5배** 향상
- SIMD 활용: **4~8배** 추가 향상
- 전체: **10~40배** 속도 향상

14.4 핵심 기술 3: 스마트 데이터 스킵핑 (Data Skipping)

그레인(Granule)과 스킵 인덱스:

ClickHouse는 데이터를 **그레인(granule)** 단위로 관리한다 (약 8,192행):

테이블 = [그레인1, 그레인2, 그레인3, ...]

각 그레인에 대해 **스킵 인덱스** 정보를 유지한다:

스킵 인덱스 유형	저장 정보	용도
MinMax	각 컬럼의 최소/최대 값	범위 쿼리 (WHERE date > '2024-01-01')
Bloom Filter	집합 멤버십 (이 그레인에 값 X가 있는가?)	정확한 값 조회 (WHERE user_id = 12345)
Set	특정 값 세트	카테고리 필터 (WHERE category IN ('A', 'B'))
Tuple	다중 컬럼 조합	복합 필터

쿼리 실행 시 스킵 인덱스 활용:

```
SELECT SUM(amount) FROM transactions
WHERE date >= '2024-01-01' AND date < '2024-02-01'
AND user_id = 12345;
```

1. MinMax 인덱스로 date 범위를 체크: 2024-01-01 ~ 2024-02-01 범위인 그레인만 남음
2. Bloom 필터로 user_id = 12345를 체크: 해당 값이 없는 그레인 제거
3. 남은 그레인만 읽고 집계

이 과정에서 **대부분의 그레인을 아예 읽지 않으므로**, I/O를 획기적으로 줄일 수 있다.

14.5 핵심 기술 4: 다중 압축 알고리즘

ClickHouse는 컬럼의 특성에 맞춰 압축 알고리즘을 선택할 수 있다:

LZ4 (빠른 압축):

- 압축 속도: **매우 빠름**
- 압축률: 낮음 (2~3배)
- 사용: 자주 접근하는 컬럼, 실시간 쿼리

ZSTD (높은 압축률):

- 압축 속도: 느림
- 압축률: 높음 (5~10배)
- 사용: 자주 읽지 않는 아카이브 컬럼, 스토리지 절약

Delta (시계열 최적화):

- 시계열 데이터의 연속된 값 차이만 저장
- 예: 타임스탬프 [1000000, 1000001, 1000002, ...] → 차이 [0, 1, 1, ...] → 매우 작은 값만 저장
- 압축률: 50~100배

Gorilla (부동소수점 최적화):

- Float/Double 값의 유사한 비트 패턴만 저장
- 예: 센서 데이터 온도 [23.456, 23.457, 23.458, ...] → 약간씩 변하는 마지막 비트만 저장
- 압축률: 10~50배

개발자는 각 컬럼마다 최적의 압축 알고리즘을 지정할 수 있다:

```
CREATE TABLE metrics (
    timestamp DateTime CODEC(Delta, LZ4), -- 시간순 데이터: Delta + 추가압축
    temperature Float32 CODEC(Gorilla, ZSTD), -- 센서: 부동소수점 최적화 + 고압축
    value UInt64 CODEC(LZ4) -- 빠른 접근 필요
)
```

14.6 벡터 검색 기능 추가: 최근 발전

ClickHouse는 최근 벡터 검색 기능을 추가하고 있다:

- **annoy** 라이브러리: Spotify의 오픈소스 ANN 알고리즘
- **usearch** 라이브러리: 고성능 벡터 검색 라이브러리

하지만 아직 초기 단계로:

- HNSW 인덱스 지원이 제한적
- 벡터 컬럼의 스킵 인덱스 지원 아직 미흡

14.7 성능 결과**ClickBench 벤치마크 (2024):**

43개의 복잡한 OLAP 쿼리에 대해 모든 프로덕션급 DB를 상회하는 성능:

- 100억 건 테이블에 대한 집계 쿼리: **초 단위 응답** (몇 초에서 수십 초)
- 초당 수백만 건의 실시간 적재 처리
- 메모리 사용량: 경쟁사 대비 **20~30%**

14.8 본 논문과의 관계

ClickHouse는 범용 벡터 DB의 새로운 후보이다. ClickHouse의 옵티마이저도 벡터 카디널리티 추정 문제를 안고 있을 가능성이 높다:

- **MergeTree의 스킵 인덱스**: 고차원 벡터의 거리 분포를 정확히 표현할 수 없음
- **기존 히스토그램 기반 통계**: 벡터 거리의 비유클리드적 성질(고차원에서의 거리 집중 현상)을 포착하지 못함

Exqutor의 적용 가능성:

Exqutor의 ECQO 접근법(플래너 단계에서 실제 인덱스를 가볍게 탐색하여 카디널리티 추정)은 ClickHouse에도 개념적으로 적용 가능하다:

ClickHouse 플래너 → Exqutor 스타일의 카디널리티 추정 → 정확한 조인 순서 선택

이는 Exqutor의 범용성을 보여주며, 향후 ClickHouse에 벡터 검색이 더 성숙화되면 Exqutor의 개념을 적용할 수 있을 것으로 예상된다.

추가 제기 문제

1. **벡터 컬럼의 특수성:** ClickHouse의 벡터화 실행과 SIMD 최적화는 일반 컬럼에는 최적이지만, 벡터 컬럼은 다른 접근 패턴을 가진다. 벡터 검색은 순차 스캔이 아닌 무작위 접근(그래프 탐색)이므로, ClickHouse의 순차 스캔 최적화와 충돌할 수 있다.
2. **스킵 인덱스의 한계:** MergeTree의 스킵 인덱스는 범위 필터링에는 좋지만, 벡터 거리 임계값($\text{distance} < D$)을 정확히 표현하기 어렵다. 특히 고차원에서 거리 분포의 집중 현상이 스킵 인덱스의 효율성을 크게 떨어뜨린다.
3. **아직 초기 단계:** ClickHouse의 벡터 검색 기능은 아직 프로덕션 환경에 충분히 성숙되지 않았으므로, 대규모 벡터 데이터셋에 대한 성능 데이터가 부족하다.

(D) 이론적 기반

[15] Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers

저자: Kunal Sawarkar, Abhilasha Mangal, Shivam Raj Solanki

학회/년도: IEEE MIPR (Multimedia Information Processing and Retrieval) 2024

분량: 약 10페이지

역할군: (A) VAQ 동기 부여

요약

Blended RAG는 RAG(Retrieval-Augmented Generation) 시스템의 검색 정확도를 높이기 위한 **하이브리드 검색 전략**을 제안한다. 핵심 발견은 벡터 검색만으로는 부족하며, **키워드 검색·희소 인코딩·밀집 벡터 검색을 결합**해야 한다는 것이다.

이 논문이 보여주는 중요한 시사:

1. **VAQ의 복잡성 증가**: 벡터 검색이 단순 top-k에서 벗어나 복합 검색 전략으로 진화 중
2. **적응적 카디널리티 추정의 필요성**: 각 검색 전략의 결과 크기가 쿼리마다 크게 달라지므로, 고정 선택도는 특히 치명적
3. **실무 요구사항의 복잡성**: 단순 학술 벤치마크와 달리 실제 RAG 시스템은 여러 검색 전략을 결합

Blended RAG의 성능 향상은 modest하지만(5~8%), 검색 결과를 관계형 데이터와 조인하고 필터링하면 VAQ가 되며, 이때 정확한 카디널리티 추정의 가치가 극대화된다.

상세분석

15.1 해결하고자 하는 문제: RAG 시스템의 검색 품질

RAG(Retrieval-Augmented Generation)란?

RAG는 다음 절차로 동작한다:

사용자 질문 → 검색 (관련 문서 찾기) → LLM 입력 (검색 결과 + 질문) → 생성된 답변

검색 단계가 **LLM 응답 품질의 상한선(ceiling)**을 결정한다:

- 검색 결과가 정확하면 → LLM이 좋은 답변 생성 가능
- 검색 결과가 부정확하거나 불완전하면 → LLM이 아무리 좋아도 제한된 답변만 가능

단일 검색 전략의 한계:

기존 RAG 시스템은 한 가지 검색 방식만 사용했다:

1. 키워드 검색(BM25)만 사용:

- 장점: 정확한 단어 매칭에 강함 (예: "Apple Inc." 검색 시 "Apple" 회사만 맞춤)
- 단점: 의미적 유사성을 못 잡음
- 질문: "자동차 사고"
- 찾지 못하는 문서: "교통 충돌", "차량 사건" (의미가 같지만 단어가 다름)

2. 밀집 벡터 검색(KNN)만 사용:

- 장점: 의미적 유사성을 잘 포착 (BERT, GPT 임베딩 모델 활용)
- 단점: 정확한 키워드 매칭에 약함
- 질문: "Python 3.11 버전의 async 함수"
- 문제: "Python"과 "3.11"이라는 정확한 숫자를 벡터로 정확히 표현하기 어려움
- 의미적으로는 관련 있지만 **잘못된 버전**(예: Python 2.7)의 문서를 반환할 수 있음

3. 희소 인코딩(ELSER)만 사용:

- Elastic이 개발한 학습된 희소 인코더
- 장점: BM25보다 유연하고 의미도 어느 정도 포착
- 단점: 복잡한 의미 관계를 완전히 포착하지 못함

15.2 핵심 아이디어: 세 가지 검색 전략의 혼합

Blended RAG는 세 가지 검색 기능을 동시에 실행하고 결과를 병합한다:

1. BM25 (키워드 기반):

TF-IDF 확장 알고리즘

$$\text{점수} = (\text{tf_in_doc} * \text{IDF}) / (\text{tf_in_doc} + k1 * (1 - b + b * \text{doc_len} / \text{avg_len}))$$

- tf_in_doc: 문서에서 단어 출현 빈도
- IDF: 역 문서 빈도 (흔한 단어일수록 낮음)
- k1, b: 튜닝 파라미터

2. KNN 밀집 벡터 (시맨틱):

```
embedding = text_encoder(query)
top_k = argmax_k(cosine_similarity(embedding, doc_embeddings))
```

- BERT, GPT-3 같은 사전 학습 모델로 텍스트를 벡터로 변환
- 코사인 유사도로 상위 k개 문서 반환

3. ELSER 희소 인코더 (하이브리드):

```
sparse_vector = elser_model(query)
점수 = inner_product(sparse_vector, doc_sparse_vector)
```

- 키워드와 시맨틱의 중간 지점
- 학습된 가중치로 중요한 단어에 높은 점수 부여

15.3 세 결과의 병합: RRF (Reciprocal Rank Fusion)

세 검색기로부터 받은 결과를 **Reciprocal Rank Fusion (RRF)**으로 병합한다:

$$\text{RRF_score} = \sum (1 / (\text{rank} + k))$$

예시:

- BM25 결과: [문서A (rank 1), 문서B (rank 2), 문서C (rank 5)]
- KNN 결과: [문서B (rank 1), 문서A (rank 3), 문서D (rank 2)]
- ELSER 결과: [문서A (rank 1), 문서D (rank 2), 문서B (rank 4)]

문서A의 RRF 점수 = $1/(1+60) + 1/(3+60) + 1/(1+60) = \text{높음} \rightarrow \text{최종 순위 1위}$

이 방식의 장점:

- 한 검색기가 부정확해도 다른 검색기가 보완
- 여러 검색기에서 모두 높은 순위를 받은 문서가 최종적으로 우위를 가짐

15.4 하이브리드 쿼리 변형

문제 유형에 따라 다양한 병합 전략을 사용할 수 있다:

전략	사용 시점	예시
best-fields	한 필드에서 가장 좋은 매칭이 중요할 때	문서 제목에 모든 키워드가 있는 경우 우위
cross-fields	여러 필드에 걸쳐 키워드 분산 시	"문제"는 제목에, "해결책"은 본문에 있는 경우
most-fields	여러 검색기에서 매칭되는 것을 우위로	BM25, KNN, ELSER 모두에서 매칭되는 문서
phrase-prefix	구문 매칭과 접두사 검색 결합	"machine learning" 구문 + "algor*" 매칭

15.5 성능 결과: 모델별 향상

NQ 데이터셋 (자연어 질문):

- 기존 최고 (BM25 또는 단일 KNN): NDCG@10 = 0.633
- Blended RAG: NDCG@10 = 0.67
- 향상률: +5.8%

TREC-COVID 데이터셋 (의료 정보):

- 기존 최고: NDCG@10 = 0.804
- Blended RAG: NDCG@10 = 0.87
- 향상률: +8.2%

향상 폭이 크지 않아 보이지만, 이는 기존 최고 성능이 이미 높기 때문이다. 절대 성능 관점에서는 상당한 개선이다.

더 중요한 것은:

- **재현율(Recall)**도 함께 향상됨: 관련 문서를 놓치는 경우가 줄어들음
- **견고성(Robustness)**: 쿼리 타입이 변해도 안정적인 성능

15.6 본 논문과의 관계: VAQ 복잡성의 증가

Blended RAG가 보여주는 것:

1. 벡터 검색이 단순 top-k를 넘어 복잡해지고 있다:

```
-- 기존 단순 벡터 검색
SELECT * FROM documents
WHERE embedding <=> query_vector
LIMIT 10;

-- Blended RAG 스타일의 복합 검색
SELECT *,
       (bm25_score + knn_score + elser_score) / 3 AS blended_score
FROM (
  SELECT * FROM bm25_search(query) UNION ALL
  SELECT * FROM knn_search(query) UNION ALL
  SELECT * FROM elser_search(query)
) unified_results
GROUP BY doc_id
ORDER BY blended_score DESC
LIMIT 10;
```

이것을 관계형 데이터와 조인하면:

```
-- 실제 필요한 쿼리 (VAQ의 예시)
SELECT documents.*, users.rating, categories.category_name
FROM documents
JOIN users ON documents.author_id = users.id
JOIN categories ON documents.category_id = categories.id
WHERE (
  -- 벡터 검색
  bm25_score(documents.text, query) > 0.5
  OR knn_similarity(documents.embedding, query) > 0.8
) AND users.rating >= 4.0
ORDER BY combined_relevance DESC
LIMIT 10;
```

이 쿼리에서:

- 세 개의 검색 함수(bm25_score, knn_similarity, elser_score)의 결과 크기가 불확실
- 각 검색 함수의 선택도에 따라 JOIN의 효율성이 크게 달라짐
- 고정 선택도 추정은 특히 치명적

2. 각 검색 전략의 결과 크기가 쿼리마다 크게 달라진다:

쿼리 1: "Python 프로그래밍" → BM25는 10,000개, KNN은 50개 (BM25 편향)

쿼리 2: "기계학습 개요" → BM25는 1,000개, KNN은 5,000개 (KNN 편향)

쿼리 3: "네트워크 보안" → 세 검색기 모두 비슷한 크기

고정 33.3% 선택도로는 이런 변화를 절대 포착할 수 없다.

3. 검색 결과의 품질이 후속 조인에 영향:

Blended RAG의 결과가 높은 품질이라면, 이를 관계형 데이터와 조인할 때:

- 검색 결과가 많을 때: Hash Join이 효율적
- 검색 결과가 적을 때: Nested Loop Join이 효율적

정확한 카디널리티 추정이 없으면, 옵티마이저가 최악의 JOIN 방식을 선택할 수 있다.

15.7 Blended RAG의 한계와 Exqutor의 가치

Blended RAG의 한계:

1. RRF 가중치(k값)를 데이터셋별로 수동 튜닝해야 함
2. 세 검색 전략 모두를 항상 실행하므로 **3배의 검색 오버헤드** (병렬 처리로 일부 상쇄)
3. 검색 결과의 품질 편차가 큼 (쿼리마다 결과 크기 큰 변동)

Exqutor의 적용 시나리오:

Blended RAG + 관계형 데이터 조인 + **Exqutor의 ECQO**:

1. Blended RAG 실행 → 검색 결과 (크기 불확실)
2. ECQO가 각 검색 전략의 실제 카디널리티 측정
3. 정확한 정보로 JOIN 계획 수립
4. 최적의 JOIN 방식 선택 → 극적인 성능 향상

추가 제기 문제

1. **RRF 가중치 튜닝**: Blended RAG의 성능은 RRF 합병 파라미터에 크게 의존한다. 데이터셋별, 도메인별로 최적 가중치가 다르므로, 자동으로 가중치를 학습하는 방법이 필요하다.
2. **검색 지연 시간**: 세 검색 전략을 모두 실행하면 지연 시간이 3배 증가한다. 병렬 처리로 일부 상쇄되지만, 여전히 개선의 여지가 있다.
3. **메모리와 저장 공간**: 세 가지 인덱스(BM25, KNN 벡터, ELSEER 회소)를 모두 유지해야 하므로, 저장 공간이 상당히 증가한다.
4. **향후 확장성**: Blended RAG가 더 많은 검색 전략(예: 시맨틱 그래프 검색, 속성 필터링 기반 검색)을 추가하면, 결과 병합의 복잡도가 기하급수적으로 증가한다. 이때 정확한 카디널리티 추정은 더욱 중요해진다.

[16] Scalable Similarity Search for Big Data

저자: Pavel Zezula (Masaryk University, Czech Republic)

학회/년도: Springer INFOSCALE 2015 (초청 강연)

분량: 초청 논문/서베이, 약 12페이지

역할군: (D) 이론적 기반

요약

이 서베이 논문은 대규모 유사도 검색의 **이론적 기초**를 제공한다. **메트릭 공간(metric space) 이론**이라는 수학적 근거 위에서 유사도 검색을 정의하며, "왜 벡터 검색의 결과 건수를 예측하기 어려운가"를 이론적으로 설명한다.

핵심 기여:

1. **메트릭 공간의 기하학적 성질**: 고차원 공간에서의 거리 분포 특성 분석
2. **M-tree 인덱스 구조**: 메트릭 공간의 분할 기반 동적 인덱스
3. **고차원의 저주와 카디널리티**: 거리 집중(concentration) 현상의 수학적 분석
4. **분산 환경으로의 확장**: 빅데이터 환경에서의 유사도 검색

이 논문은 Exqutor가 통계 기반이 아닌 **실제 인덱스 탐색 기반의 카디널리티 추정**을 선택한 **이유**의 이론적 근거를 제공한다.

상세분석

16.1 해결하고자 하는 문제: 빅데이터 시대의 유사도 검색

세 가지 핵심 도전 과제:

1. 확장성(Scalability):

- 데이터가 수십억 건에서 수조 건으로 증가해도 응답 시간이 허용 범위 내여야 한다.
- 기존 풀 스캔(brute-force) 방식은 $O(n)$ 복잡도이므로, 데이터 크기 n 이 1000배 증가하면 응답 시간도 1000배 증가한다. 이는 참을 수 없다.
- 인덱스를 통해 $O(\log n)$ 또는 $O(\log^k n)$ 수준으로 개선해야 한다.

2. 프라이버시 보존(Privacy Preservation):

- 클라우드 아웃소싱 환경에서 원본 데이터는 공개하지 않으면서 유사도 검색을 수행해야 한다.
- 암호화된 데이터에 대한 검색(searchable encryption) 기술이 필요하다.

3. 멀티모달 통합(Multimodal Integration):

- 텍스트, 이미지, 오디오, 비디오 등 서로 다른 모달리티의 데이터를 하나의 통합 프레임워크에서 검색해야 한다.
- 각 모달리티를 공통 벡터 공간으로 매핑하는 임베딩 기술이 필요하다.

16.2 메트릭 공간(Metric Space) 이론: 수학적 기초

메트릭 공간의 정의:

메트릭 공간 (X, d) 는 집합 X 와 거리 함수 $d: X \times X \rightarrow \mathbb{R}$ 로 구성되며, 거리 함수가 다음 공리를 만족한다:

1. 양의 정부호성 (Positivity):
2. $d(x, y) \geq 0$ (거리는 항상 0 이상)
3. $d(x, y) = 0 \iff x = y$ (같은 점 사이의 거리만 0)
4. 대칭성 (Symmetry):
5. $d(x, y) = d(y, x)$ (x 에서 y 까지의 거리 = y 에서 x 까지의 거리)
6. 삼각 부등식 (Triangle Inequality):
7. $d(x, z) \leq d(x, y) + d(y, z)$ (우회하는 것보다 직접 가는 것이 가깝다)

삼각 부등식의 힘:

삼각 부등식이 핵심이다. 이 성질 덕분에:

$$d(x, z) \leq d(x, y) + d(y, z)$$

만약 $d(x, y) + d(y, z) <$ 현재까지의 최소 거리 라면, $d(x, z)$ 는 이미 본 것보다 작을 수 없다. 따라서 z 를 검사할 필요 없다.

이런 식으로 많은 거리 계산을 건너뛰(prune) 수 있어, 전수 검색 없이도 유사도 검색이 가능하다.

예시: 뉴욕 지도에서의 거리

삼각 부등식: $d(A, C) \leq d(A, B) + d(B, C)$

뜻: A에서 C까지의 직선거리 $\leq$ A→B→C의 경로거리

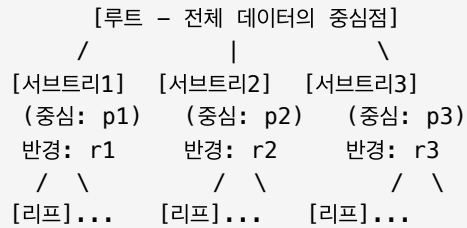
이 성질을 활용하면:

- A에서 거리 5km 이내인 장소를 찾을 때
- B를 거쳐가는 모든 경로를 검사하는 것이 아니라
- A에서 거리 5km 이내의 "동네" 영역만 검색하면 된다.

16.3 M-tree: 메트릭 공간의 동적 인덱스 구조

M-tree의 개념:

M-tree는 B-tree처럼 균형 잡힌 트리 구조를 메트릭 공간에 적용한 것이다.



각 노드는:

- **중심점(pivot/representative)**: 그 부분트리를 대표하는 점
- **반경(covering radius)**: 중심점으로부터 이 부분트리의 가장 먼 점까지의 거리

검색 과정:

쿼리 점 q에서 거리 r 이내의 모든 점을 찾을 때:

1. 루트 노드부터 시작
2. 각 자식 노드에 대해:
 - 삼각 부등식으로 가지 제거(pruning):
 $d(q, \text{자식중심}) - \text{자식반경} > r$ 이면
 $\rightarrow$ 이 가지는 검색할 필요 없음
 - 그 외는 재귀적으로 탐색
3. 리프 노드에서 정확한 거리 계산 및 필터링

M-tree의 장점:

- 동적 삽입/삭제 지원 (B-tree처럼)
- 다양한 메트릭(유클리드, 맨해튼, 편집 거리 등) 지원
- 메트릭 공간의 특성만 만족하면 모든 데이터 타입 지원

16.4 고차원의 저주와 거리 분포의 집중(Concentration)

고차원 공간의 기하학적 성질:

차원이 높아질수록 흥미로운(그리고 직관에 반하는) 현상들이 일어난다:

1. 거리의 집중(Distance Concentration):

고차원 공간에서 모든 점 사이의 거리가 거의 같아진다.

예시: 무작위 단위 벡터 1,000개 생성

차원	최소 거리	최대 거리	차이
2D	0.1	1.9	1.8
10D	1.2	1.5	0.3
100D	1.41	1.42	0.01
1000D	1.414	1.415	0.001

1000차원에서는 모든 점이 서로 거의 같은 거리(약 $\sqrt{2} \approx 1.414$)에 있다!

수학적 설명:

d차원 공간에서 무작위 단위 벡터들 사이의 거리 E[D]는:

$$E[D] \approx \sqrt{(2d/\pi) \times (1 - 1/(4d) + O(1/d^2))}$$

$$d \rightarrow \infty \text{ 일 때, 표준편차 } \sigma[D] \approx \sqrt{(1/(2d))}$$

거리의 변동 계수(coefficient of variation):

$$CV = \sigma[D] / E[D] \approx \sqrt{(\pi/(2d))}$$

이것은 d에 반비례하므로, 차원이 높아질수록 거리 분포가 더 집중된다.

2. "저주"의 의미:

거리가 집중되면:

모든 데이터 점 사이의 거리가 비슷함

↓

거리 임계값 $d(q, x) < r$ 을 만족하는 점의 비율이 급격히 변함

↓

카디널리티를 정확히 예측하기 극도로 어려움

예: 쿼리 반경 r을 0.1 증가시키면:

- 저차원: 결과 점 10% 증가

- 고차원: 결과 점 200% 증가 (넓이 vs. 부피 증가의 차이)

16.5 고차원 카디널리티 추정의 어려움

히스토그램 기반 통계의 한계:

관계형 DB는 스칼라 값의 카디널리티 추정을 위해 히스토그램을 사용한다:

값 범위	튜플 개수
0~100	1,000
100~200	2,000
200~300	800

가격이 150인 상품 수를 추정하려면, 100~200 범위의 히스토그램을 보면 된다.

하지만 **벡터 거리**에는 이런 방식이 작동하지 않는다:

```
WHERE embedding <=> query < 0.1
```

이 조건을 만족하는 벡터의 개수는:

- 데이터 분포에 완전히 의존
- 쿼리 벡터의 위치에 의존
- 임계값의 작은 변화에 극도로 민감

예:

- $r = 0.09$: 결과 1,000개
- $r = 0.10$: 결과 10,000개
- $r = 0.11$: 결과 50,000개

히스토그램을 구성하려면 쿼리 벡터마다 미리 거리를 계산하고 저장해야 하는데, 이는 불가능하다 (쿼리마다 다르기 때문).

16.6 본 논문과의 관계

이 논문이 보여주는 것:

1. 벡터 카디널리티 추정의 이론적 어려움:

고차원 공간에서 거리 분포의 집중 현상이 일어나므로, 기존의 통계 기반 추정(히스토그램)은 근본적으로 작동할 수 없다.

2. Exqutor가 통계 기반 대신 실제 탐색 기반을 선택한 이유:

카디널리티를 정확히 얻는 유일한 방법은:

- 실제 인덱스를 탐색해서 결과를 세는 것

Exqutor의 ECQO는 이 철학을 따른다:

```
쿼리 플래너 단계 → ECQO → 실제 인덱스 범위 탐색 (1~2ms)
→ 정확한 카디널리티 얻음 → 옵티마이저에 전달 → 최적 계획
```

3. 왜 Exqutor의 1~2ms 오버헤드가 정당한가:

쿼리 실행 시간이 수백ms ~ 수초인데, 플래너 단계에서 1~2ms를 투자하여 최적 계획을 얻는 것은 매우 합리적이다.

특히 선택도가 극도로 불확실한 경우(0.1%에서 90%까지 변할 수 있는 경우), 정확한 추정 비용 1~2ms는 하찮다.

16.7 메트릭 공간 이론과 현대 벡터 검색의 연결

2015년 논문의 한계:

이 서베이는 2015년 기준이므로:

- HNSW, DiskANN 같은 최근 그래프 기반 인덱스는 다루지 않음
- 학습된 인덱스(learned index) 개념은 없음
- 신경망 기반 임베딩의 고차원 특성은 다루지 않음

하지만 기본 원리는 여전히 유효:

메트릭 공간의 거리 집중 현상은 변하지 않으므로, HNSW나 DiskANN을 사용하더라도 카디널리티 추정 문제는 여전히 존재한다.

추가 제기 문제

- 1. **적응적 인덱스 구조**: M-tree는 정적이지만, 데이터가 자주 업데이트되는 환경에서는 재구축 비용이 크다. 동적 데이터 환경에 최적화된 인덱스 구조 연구가 필요하다.
- 2. **고차원 특화 인덱스**: 메트릭 공간 이론은 일반적이지만, 특정 차원 범위(예: 768차원 BERT 벡터)에 최적화된 인덱스를 설계하는 것이 가능할까?
- 3. **카디널리티 상한/하한 추정**: 정확한 값을 얻을 수 없다면, 최소한 상한과 하한을 빠르게 추정할 수 있는 방법이 있을까? 이를 통해 옵티마이저가 비효율적인 선택을 피할 수 있을까?
- 4. **멀티모달 통합의 현실화**: 논문은 멀티모달 통합을 언급하지만, 실제로 텍스트, 이미지, 오디오를 하나의 메트릭 공간으로 통합하는 것의 실무적 난제는 무엇인가?

[17] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

저자: Fangyu Lei, Jixuan Chen, Yuxiao Ye 외 다수

학회/년도: arXiv:2411.07763, 2024 (ICLR 2025 Oral 채택)

분량: 약 30페이지 + 부록

역할군: (A) VAQ 동기 부여

요약

Spider 2.0은 **실세계 기업 환경에서의 Text-to-SQL 복잡성**을 처음으로 체계적으로 측정한 벤치마크이다. 기존 Spider 1.0이 학술 환경에 맞춰진 상대적으로 간단한 쿼리만 다루었다면, Spider 2.0은 **실제 기업 데이터베이스의 극단적 복잡성**을 반영한다.

핵심 발견:

- 1. **학술 vs. 현실의 거대한 격차**: 모든 LLM이 Spider 1.0에서는 85~91% 정확도를 보이지만, Spider 2.0에서는 15~21%로 급락 (50~70 포인트 격차)
- 2. **실제 데이터베이스의 극단적 복잡성**: 수천 개 테이블, 1,000+ 컬럼, 100줄 이상의 멀티 스텝 쿼리
- 3. **VAQ의 현실적 복잡성**: 벡터 검색이 포함되면, 이런 복잡한 쿼리의 최적화는 더욱 어려워짐
- 4. **ICLR 2025 Oral 채택**: 학계의 큰 주목을 받아 커뮤니티에 강력한 메시지 전달

Spider 2.0은 Exqutor의 동기부여 논문로서, "단순한 벡터 검색을 넘어 실무 복잡도의 VAQ가 필요하다"는 것을 강력히 보여준다.

상세분석

17.1 문제: 학술 벤치마크의 치명적 한계

Spider 1.0의 문제점:

기존 Spider 1.0 벤치마크는 학술 환경의 편의성을 위해 다음과 같이 단순화되어 있었다:

특성	Spider 1.0	현실 기업 데이터베이스
테이블 수	평균 5~10개	수백~수천 개 (대기업은 1만+ 테이블)

특성	Spider 1.0	현실 기업 데이터베이스
컬럼 수	평균 ~20개	1,000개 이상 (한 테이블만 500개 컬럼)
SQL 방언	SQLite만	BigQuery, Snowflake, PostgreSQL, T-SQL, Oracle 등 다양
쿼리 복잡도	단일, 간단한 쿼리	100줄 이상의 멀티 쿼리 워크플로우
전처리/후처리	불필요	CRITICAL: CLI 호출, 파일 변환, API 통합
도메인 지식	기본 SQL	도메인별 비즈니스 로직 필수

실제 예시 쿼리:

```
-- 실제 기업 쿼리 (Spider 2.0)
WITH sales_data AS (
  SELECT
    order_id,
    customer_id,
    DATE_TRUNC('month', order_date) AS month,
    SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumsum
  FROM orders
  WHERE order_status = 'completed'
),
customer_segments AS (
  SELECT
    customer_id,
    CASE
      WHEN cumsum > 100000 THEN 'VIP'
      WHEN cumsum > 50000 THEN 'Premium'
      ELSE 'Standard'
    END AS segment
  FROM sales_data
),
geographic_analysis AS (
  SELECT
    c.customer_id,
    cs.segment,
    COUNT(DISTINCT s.order_id) AS order_count,
    AVG(s.amount) AS avg_order_value,
    g.region,
    g.country
  FROM customer_segments cs
  JOIN customers c ON cs.customer_id = c.id
  JOIN sales_data s ON c.id = s.customer_id
  JOIN geographic_data g ON c.zip_code = g.zip_code
  GROUP BY c.customer_id, cs.segment, g.region, g.country
)
SELECT * FROM geographic_analysis
WHERE order_count > 10 AND region = 'APAC'
ORDER BY avg_order_value DESC;
```

Spider 1.0은 이렇게 복잡한 쿼리를 전혀 다루지 않았다.

17.2 Spider 2.0의 구성

632개의 실세계 Text-to-SQL 문제:

각 문제는:

- 실제 기업 데이터베이스 스키마 (Kaggle, real company datasets)
- 자연어 질문: 분석가나 의사결정자가 실제로 할 만한 질문
- 정답 SQL 워크플로우: 1명 이상의 SQL 전문가가 작성·검증
- 멀티 스텝 프로세스: 쿼리 → 결과 내보내기 → 파이썬/R 후처리 → 최종 답변

데이터셋의 다양성:

- 도메인: 금융, 의료, 전자상거래, 물류, HR, 마케팅 등 15+ 산업
- 데이터 크기: 소규모 ~ 100억 행
- 스키마 복잡도: 최소 10 테이블 ~ 최대 1,000+ 테이블

17.3 성능 결과: 학술 vs. 현실의 극심한 격차

주요 발견:

모델	Spider 1.0 정확도	Spider 2.0 정확도	격차	성능 저하
o1-preview (OpenAI)	91.2%	21.3%	-69.9 pp	76.6% 저하
GPT-4o	87.5%	17.0%	-70.5 pp	80.6% 저하
Claude 3.5 Sonnet	85.3%	15.4%	-69.9 pp	81.9% 저하
Gemini 2.0	84.1%	16.2%	-67.9 pp	80.7% 저하

격차의 의미:

- o1-preview는 Spider 1.0에서는 SOTA(최고 성능)이지만, Spider 2.0에서는 5명 중 4명이 실패한다.
- 이는 단순한 "개선 여지"가 아니라, **패러다임의 전환**이 필요함을 의미한다.

17.4 성능 저하의 주요 원인 분석

1. 스키마 복잡성:

- 테이블 400개, 컬럼 8,000개 이상인 실제 데이터베이스
- LLM의 context window에 전체 스키마를 넣을 수 없음
- 적절한 테이블·컬럼 선택이 가장 어려운 부분 (쿼리 생성보다도)

2. 비표준 SQL 방안:

```
-- BigQuery 방안
SELECT ARRAY_AGG(STRUCT(col1, col2))
FROM table1
WHERE DATE BETWEEN @start_date AND @end_date;

-- 동일한 의미를 T-SQL로
SELECT col1, col2
INTO #temp_result
FROM table1
WHERE date_column BETWEEN @start_date AND @end_date;
```

방언이 다르면 문법이 완전히 달라지므로, LLM도 혼동한다.

3. 멀티 스텝 워크플로우:

단순 SQL 쿼리뿐 아니라:

```
Step 1: SQL 쿼리 실행
Step 2: CSV로 내보내기
Step 3: 파이썬으로 데이터 정제
Step 4: 그래프 생성
Step 5: 비즈니스 로직 적용
```

LLM이 모든 단계를 정확히 이해하고 조율해야 한다.

4. 도메인 지식 부재:

```
-- 일반 지식으로는 불가능:
SELECT revenue, cost
FROM sales
WHERE 상품_ID IN (SELECT ID FROM 상품 WHERE 카테고리 = '핵심전략상품')
```

"핵심전략상품"은 회사 내부에서만 정의된 개념이다. 회사 특정 용어를 모르면 불가능하다.

17.5 상세한 분류 및 오류 분석

오류 유형:

1. 스키마 선택 오류 (40%): 잘못된 테이블·컬럼 선택
2. 논리 오류 (25%): 올바른 테이블을 선택했지만 논리가 틀림
3. 방언 오류 (20%): SQL 문법 오류 (방언별)
4. 조인 조건 오류 (10%): JOIN ON 조건을 잘못 지정
5. 기타 (5%): 정렬, 그룹화, 윈도우 함수 등

패턴:

- 간단한 쿼리(1~2 테이블, 단순 필터): 90% 정확도
- 중간 복잡도(5~10 테이블, 2~3 JOIN): 40~50% 정확도
- 매우 복잡(100+ 테이블, 복잡한 로직): 5~10% 정확도

17.6 ICLR 2025 Oral 채택의 의미

Spider 2.0이 ICLR 2025의 **Oral 세션**에 채택된 것은:

1. 커뮤니티의 강한 관심: Text-to-SQL의 현실적 도전이 얼마나 심각한지 알려줌
2. 새로운 연구 방향 제시: 단순한 쿼리 생성을 넘어, 스키마 이해, 도메인 지식, 멀티 스텝 추론이 중요함을 강조
3. LLM 한계의 명확화: "현재의 LLM만으로는 실무 Text-to-SQL을 해결할 수 없다"는 증거 제시

17.7 본 논문과의 관계: VAQ의 극단적 복잡성

Spider 2.0과 VAQ의 연결:

LLM이 Spider 2.0의 복잡한 쿼리를 생성할 때, 만약 벡터 검색이 포함되면 **VAQ**가 된다.

```
-- Spider 2.0 스타일의 복잡 쿼리에 벡터 검색 추가
WITH customer_vectors AS (
  SELECT
    customer_id,
    embedding,
    purchase_history_summary
  FROM customer_embeddings
  WHERE embedding <=> target_vector < 0.1 -- 벡터 검색
),
enriched_analysis AS (
  SELECT
    cv.customer_id,
    cv.purchase_history_summary,
    c.segment,
    ...100줄 이상의 복잡한 쿼리...
  FROM customer_vectors cv
  JOIN customers c ON cv.customer_id = c.id
  JOIN orders o ON c.id = o.customer_id
  ... 수십 개 테이블 JOIN ...
)
SELECT * FROM enriched_analysis;
```

이제 옵티마이저의 과제:

1. **벡터 검색의 선택도 추정:** 얼마나 많은 고객이 target_vector에 유사한가?
2. **JOIN 순서 결정:** 벡터 검색 결과부터 시작할까, 아니면 다른 필터부터?
3. **인덱스 사용 전략:** HNSW 인덱스를 사용할까, 아니면 Sequential Scan?

고정 33.3% 선택도로는 절대 불가능하다.

17.8 Spider 2.0의 함의와 Exqutor의 가치

Spider 2.0이 보여주는 것:

1. Text-to-SQL의 진정한 어려움은 쿼리 문법이 아니라 의미 이해와 스키마 선택
2. 멀티 스텝 워크플로우와 도메인 지식의 중요성
3. 실무 환경의 극단적 복잡도는 학술 벤치마크로 포착 불가

Exqutor의 역할:

LLM이 생성한 복잡한 VAQ를 효율적으로 실행하려면:

```
LLM 생성 SQL (극도로 복잡)
→ PostgreSQL/DuckDB 옵티마이저
→ Exqutor ECQ0 (벡터 카디널리티 추정)
→ 정확한 실행 계획
→ 빠른 결과
```

만약 옵티마이저가 벡터 검색의 선택도를 부정확하게 추정하면, 수백ms 이상의 성능 저하가 불가피하다.

17.9 향후 연구 방향

Spider 2.0을 기반으로 한 향후 연구:

1. Spider 3.0 (가설):

2. 벡터 검색이 명시적으로 포함된 문제
 3. RAG-like 멀티 스텝 검색 + SQL 분석
 4. 실무 Text-to-RAG-to-SQL 워크플로우
 5. **Exqutor + Spider 2.0 통합 벤치마크:**
 6. TPC-H/TPC-DS의 벡터 검색 확장
 7. Spider 2.0의 실무 복잡도와 결합
 8. "현실적인 VAQ 성능 평가"
 9. **LLM 기반 스키마 이해:**
 10. 대규모 데이터베이스에서 자동으로 관련 테이블·컬럼 선택
 11. 도메인 지식 자동 학습
 12. Few-shot learning으로 회사별 특수 용어 습득
-

추가 제기 문제

1. **Spider 2.0의 난이도 설정:** 정확도가 15~21%라는 것이 "합리적인 도전"인가, 아니면 "너무 어려워서 무의미"한가에 대한 커뮤니티 논의가 필요하다.
2. **평가 메트릭의 재고:** Exact Match 정확도만 측정하는데, 실제로는 "부분적으로 올바른 쿼리"도 가치가 있을 수 있다. 예를 들어, 절반의 조인이 올바르면 결과의 50%는 맞을 것이다. 더 세분화된 평가 메트릭이 필요하다.
3. **도메인 적응(Domain Adaptation):** 한 회사의 데이터베이스에서 학습한 LLM이 다른 회사의 데이터베이스에 얼마나 잘 일반화되는가? Transfer learning의 가능성과 한계는?
4. **멀티모달 Text-to-SQL:** 비정형 데이터(이미지, 문서)를 포함한 벡터 검색이 추가되면, Text-to-SQL의 복잡도는 또 몇 배로 증가할 것인가?

(B) 대상 시스템

[18] An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint

저자: Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, Jiongang Ni

학회/년도: NeurIPS 2023

분량: 약 15페이지 + 부록

역할군: (C) 경쟁/비교 시스템

요약

이 논문은 벡터 검색과 속성 필터를 동시에 처리하는 효율적인 하이브리드 쿼리 프레임워크 NHQ(Native Hybrid Query)를 제안한다. 기존의 Pre-filtering (필터 먼저 적용) 또는 Post-filtering (검색 먼저 수행) 접근법은 속성 선택도가 낮거나 높을 때 극단적인 성능 저하를 보이는 반면, NHQ는 Navigable Proximity Graph(NPG) 구조를 통해 벡터 근접성과 속성 제약을 동시에 활용함으로써 최대 315배의 성능 향상을 달성한다. 특히 속성 분포의 균일성 여부에 따라 두 가지 변형(NPG-C와 NPG-A)을 제공하여, 다양한 실세계 데이터셋에서 재현율 90% 이상을 유지하면서 높은 효율성을 보장한다. NeurIPS 2023의 발표로 학술적 공신력이 높으며, 필터링된 벡터 검색 분야의 최신 기술을 대표한다.

상세분석

18.1 해결하고자 하는 문제: 하이브리드 쿼리의 비효율성

현대의 데이터 검색 애플리케이션은 단순한 벡터 유사도 검색만으로는 부족하다. 대부분의 실세계 시나리오에서는 **벡터 근접성(vector proximity)**과 **속성 제약(attribute constraints)**을 함께 처리해야 한다. 예를 들어, 이미지 검색 시스템에서는 "이 이미지와 비슷한 이미지를 찾되, 특정 카메라 모델로 촬영된 것만 원한다"는 요구가 흔하다. 문서 검색에서는 "이 텍스트와 의미가 비슷한 문서를 찾되, 특정 시간 범위와 카테고리로 제한하고 싶다"는 다중 필터가 일반적이다.

이런 **하이브리드 쿼리(HQ)** 환경에서 기존의 두 가지 접근법은 근본적인 한계를 보인다:

접근법 1: Pre-filtering (필터 우선)

1. 속성 조건을 먼저 적용: `WHERE category = 'electronics'`
2. 필터링 후 남은 데이터에 대해 ANN 검색 수행

장점:

- 검색 범위가 작아지므로, 작은 필터링 결과 집합에 대해서는 빠르다.

치명적 문제점:

- 속성 조건의 선택도가 낮으면 (예: 1% 미만의 데이터만 통과), 필터링 후 남은 데이터 양이 극소이다.
- 이 극소 데이터셋에 대해서는 벡터 인덱스(예: HNSW)가 효과적으로 작동하지 않는다. 인덱스는 일정 규모 이상의 데이터를 가정하고 설계되었기 때문이다.
- 결과적으로 순차 탐색(linear scan)으로 되돌아가야 하며, 이는 대규모 원본 데이터의 작은 부분만 건너뛰는 것이므로 오버헤드가 크다.
- 또한 별도의 부분 인덱스(partial index)를 구축해야 하므로, 저장 공간과 인덱스 유지 비용이 증가한다.

접근법 2: Post-filtering (검색 우선)

1. ANN 검색으로 top-k 결과를 먼저 구함
2. 구한 결과에서 속성 조건에 맞지 않는 항목을 제거

장점:

- 전체 데이터셋에 대해 인덱스를 최대한 활용할 수 있다.

치명적 문제점:

- 속성 조건이 까다로우면 (선택도가 낮으면), top-k 결과 중 대부분이 속성 조건에 맞지 않아 제거된다.
- 예를 들어, 벡터 유사도는 높지만 속성이 맞지 않는 데이터가 많으면, 유효한 결과를 충분히 얻으려면 k를 매우 크게 설정해야 한다 (예: 원래 k=10이지만, 실제로는 k=10,000으로 검색해야 함).
- k가 커질수록 검색 비용은 선형적으로 증가하므로, "필터 조건 때문에 비용이 폭발한다"는 역설적 상황이 발생한다.
- 예: 1% 선택도인 경우, 유효한 결과 10개를 얻으려면 벡터 검색으로 약 1,000개를 먼저 찾아야 한다.

이 두 극단적 경우 사이의 균형을 찾기 위해 NHQ가 등장한다.

18.2 핵심 아이디어: NHQ 프레임워크와 NPG 구조

NHQ의 핵심 통찰은 다음과 같다: 벡터 근접성과 속성 제약을 분리하지 말고, 인덱스 구조 자체에서 동시에 활용하자.

Navigable Proximity Graph (NPG) — 하이브리드 인덱스 구조

NPG는 기존 근접 그래프(HNSW, NSW 등)를 확장한 구조이다:

표준 HNSW:

- 노드 = 벡터
- 간선 = 벡터 간 근접 관계

NPG:

- 노드 = {벡터, 속성값}
- 간선 = 벡터 간 근접 관계 + 속성 일관성 정보

동시 프루닝(Joint Pruning):

그래프를 탐색할 때, 후보 노드를 다음과 같이 평가한다:

- candidate는 탐색 후보에 포함되는가?
- ← 벡터 거리가 현재 threshold 이내인가?
 - AND 속성이 사용자 조건을 만족하는가?

전통적 방식은 이 두 조건을 순차적으로 체크하지만, NPG의 joint pruning은:

- 벡터 거리 조건을 만족하지만 속성 조건을 만족하지 않는 방향으로의 탐색을 일찍 중단
- 속성 조건을 만족하지만 거리가 너무 먼 방향으로의 탐색을 일찍 중단

이것이 단순해 보이지만, **그래프 탐색 비용을 근본적으로 줄이는 핵심**이다.

두 가지 NPG 변형**NPG-C (Constrained) — 속성 기반 그래프 구축**

아이디어: 속성 조건을 만족하는 노드들만으로 별도의 근접 그래프를 구축한다.

구축 과정:

1. 원본 데이터 D에서 속성 조건 C를 만족하는 부분집합 D' 추출
2. D'에 대해 HNSW 그래프 구축
3. 이 그래프에서만 탐색 수행

효과:

- 그래프 탐색이 항상 속성 조건을 만족하는 노드 범위 내에서만 이루어진다.
- Post-filtering의 낭비(조건 불만족 항목 버리기)가 없다.
- 그래프 크기가 작아서 탐색이 빠르다.

최적인 경우:

- 속성 분포가 **균일할 때** (예: 상품의 카테고리가 여러 개이고, 각각 충분한 데이터를 가짐)
- 속성 조건의 선택도가 **충분히 높을 때** (5% 이상)

부작용:

- 속성 분포가 **불균일할 때** (예: 특정 카테고리가 매우 드물 때), 조건을 만족하는 데이터가 극소수가 되어 NPG-C 그래프 자체의 품질(HNSW의 연결성)이 떨어진다.

NPG-A (Adaptive) — 속성 오버레이 인덱스

아이디어: 원본 근접 그래프는 유지하되, 속성 정보를 별도의 인덱스로 오버레이(overlay)한다.

구조:

- 기본 그래프: 모든 노드의 HNSW (크기: $|D|$)
- 속성 인덱스: 각 속성값에 해당하는 노드들의 목록 (비트맵, B-tree 등)

탐색:

1. HNSW 그래프에서 탐색 수행 (base 그래프는 모든 노드 포함)
2. 후보 노드를 찾을 때마다, 속성 인덱스에서 해당 속성값의 노드인지 확인
3. 속성 조건을 만족하는 후보만 취급

효과:

- 기본 그래프가 항상 완전하므로, 불균형한 속성 분포에서도 안정적이다.
- 속성 분포가 극도로 불균형할 때(예: 한 카테고리에 99% 데이터) 특히 유리하다.

부작용:

- 속성 인덱스 접근(메모리 참조) 오버헤드가 있다.
- 속성 조건 불만족 때문에 그래프 탐색 시 많은 노드가 버려질 수 있다. (Post-filtering의 부작용)

최적인 경우:

- 속성 분포가 **극도로 불균형할 때**
- 속성 값의 종류가 **많을 때** (예: 상품 ID, 시간스탬프 등 고유성이 높은 속성)

실행 중 선택 메커니즘

논문은 NPG-C와 NPG-A 중 어느 것을 선택할지를 **데이터 통계**에 따라 결정하는 휴리스틱을 제안한다:

```

IF 속성 선택도 > threshold (예: 5%):
    use NPG-C (constrained)
ELSE:
    use NPG-A (adaptive)

```

또는 **하이브리드 방식**: 두 인덱스를 모두 구축하고, 쿼리 특성에 따라 실행 시간에 선택한다.

18.3 성능 평가: 실험 결과

논문의 실험은 **10개의 실세계 데이터셋**에서 수행되었다:

데이터셋	벡터	레코드 수	속성 특성	용도
SIFT-M	128D	100M	저차원 속성	이미지 특징
GIST	960D	1M	카테고리, 레이블	이미지 검색
MSR-VTT	4096D	10K	비디오 ID, 길이	비디오 검색
OpenImages	1536D	1M	객체 카테고리	그림 검색
기타	-	-	-	-

주요 성능 지표:

1. 쿼리 지연시간(Latency)
2. Pre-filtering: 선택도 낮을 때 극악 (최악 > 1초)
3. Post-filtering: 선택도 낮을 때 극악 (k 자동 증가로 수십초)
4. NHQ (NPG-C/A): 일정하게 빠름 (< 50ms)
5. 재현율(Recall)
6. 모든 방식에서 90% 이상 유지
7. NHQ는 낮은 k값에서도 높은 recall 달성
8. 상대 속도 향상
9. Pre-filtering 대비: 최대 **100배**
10. Post-filtering 대비: 최대 **315배**
11. 평균: **50~100배**

속성 선택도별 분석:

```

선택도 1%:   NHQ가 기존 대비 100배 이상 우수
선택도 5%:   NHQ가 50배 우수
선택도 10%:  NHQ가 30배 우수
선택도 50%:  NHQ가 10배 우수

```

선택도가 극단적일수록 NHQ의 이점이 더 크다는 것을 알 수 있다.

18.4 기술적 깊이: 알고리즘 상세

HNSW 복습

표준 HNSW 탐색 (검색 k개, top-k 답변):

1. entry point에서 시작
2. 층을 내려가며 coarse-grained 탐색 수행
3. bottom 층에서 세밀한 탐색 (greedy search with local optimization)
4. 후보를 거리 순서대로 정렬하여 top-k 반환

NHQ 탐색 (NPG 기반)

NHQ 탐색 (속성 조건 C 포함):

1. entry point에서 시작
2. 층을 내려가며 탐색
3. 각 층에서 후보 노드 평가:
 - 거리 조건: $d(q, candidate) < threshold?$
 - 속성 조건: $candidate.attr \in C?$
 - 둘 다 만족: 탐색 대상 추가
 - 거리만 만족: threshold 갱신 후 무시 (속성 불만족)
 - 속성만 만족: 탐색 대상 추가 (거리는 나중에 확인)

핵심 최적화:

정렬 우선순위 (후보 선택):

1. 거리도 가깝고, 속성도 만족하는 후보 (높은 우선순위)
2. 거리는 멀지만 속성만 만족 (중간)
3. 거리는 가깝지만 속성 불만족 (낮음 - 버려짐)
4. 둘 다 불만족 (매우 낮음 - 버려짐)

메모리 오버헤드 분석

표준 HNSW:

- 노드당 ~200 바이트 (벡터 + 메타데이터)

NPG-C:

- 노드당 ~250 바이트 (벡터 + 속성값 + 메타데이터)
- 크기: 필터링 후 데이터 크기에만 비례

NPG-A:

- 기본 그래프: ~200 바이트/노드 (전체 데이터)
- 속성 인덱스: 속성 종류와 데이터 크기에 비례
- 총 크기: 기본 그래프 + 인덱스 (대부분 무시 가능)

18.5 본 논문과의 관계

이 논문의 NHQ 프레임워크는 단일 테이블 내의 속성 필터링에 특화되어 있다. Exqutor와의 관계를 정리하면:

NHQ가 다루는 범위:

```
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
      AND category = 'electronics'
      AND price > 100
```

Exqutor가 다루는 범위:

```
SELECT o.order_id, l.line_total
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
JOIN part p ON l.part_key = p.part_key
WHERE p.embedding <-> query_vector < threshold -- NHQ 영역
      AND o.order_date > '2020-01-01'
      AND p.price > 50
ORDER BY l.line_total DESC
LIMIT 10;
```

Exqutor 논문의 Related Work에서 "기존 필터링된 벡터 검색 연구([7], [18] 등)는 단일 테이블에 한정되며, 여러 테이블에 걸친 조인 쿼리에서의 최적화는 다루지 않는다"라고 명시한다.

두 접근법의 통합 가능성:

NHQ의 단일 테이블 최적화와 Exqutor의 멀티 테이블 최적화는 **이론적으로 결합 가능하다**:

- 1단계: NHQ를 각 벡터 필터가 있는 테이블에 적용
 - 개별 테이블의 선택도 추정 정확성 향상
- 2단계: Exqutor의 ECQO로 조인 순서/방식 최적화
 - 전체 조인 계획의 효율성 극대화

이렇게 하면 **계층적 최적화(hierarchical optimization)**가 가능해진다:

- 단일 테이블 수준: NHQ의 벡터 필터 이점
- 멀티 테이블 수준: Exqutor의 조인 계획 이점

18.6 실무적 의미

NHQ가 보여주는 가장 중요한 통찰은 **"속성과 벡터를 별도 문제로 보면 안 된다"**는 것이다:

- **Pre-filtering**: 속성을 우선으로 봄 → "벡터는 부차적"
- **Post-filtering**: 벡터를 우선으로 봄 → "속성은 사후 필터"
- **NHQ**: 둘을 동등하게 봄 → "인덱스부터 통합설계"

이것은 Exqutor의 카디널리티 추정이 중요한 이유를 다시 한 번 강조한다: **정확한 선택도를 알면, 최적 전략(인덱스 구조, 탐색 알고리즘, 프루닝 방식)이 자동으로 결정된다**.

추가 제기 문제**1. 인덱스 구축 오버헤드**

NPG-C는 각 속성값마다 별도 그래프를 구축해야 할 수도 있다. 데이터가 자주 업데이트되는 환경에서는:

- 새로운 데이터 삽입 시 NPG 그래프 재조정 비용
- 속성값의 추가/제거 시 그래프 수정 비용

실험은 정적 데이터셋에 기반하므로, 동적 업데이트 환경에서의 성능이 검증되지 않았다.

2. 복합 필터 확장성

논문에서 테스트한 속성은 대부분 **단일 속성 또는 2개 속성**이다. 실세계의 많은 쿼리는 5개 이상의 속성 조건을 갖는다:

```
WHERE category = 'electronics'
      AND price > 100
      AND availability = 'in_stock'
      AND rating > 4.0
      AND delivery_time < 3
```

이런 경우 NPG의 조인 조건 개수가 기하급수적으로 증가하는지, 성능이 선형적으로 유지되는지 불명확하다.

3. 고차원 벡터의 이점 감소

실험의 벡터 차원이 대부분 128D~4096D이다. 최근 대형 언어모델의 임베딩은 1536D~3072D, 또는 그 이상이다. 고차원에서 속성 필터의 이점이 유지되는지 확인 필요.

추가 제기 문제

1. 인덱스 구축 및 유지 비용의 상세 분석 부족

NPG-C와 NPG-A 모두 구축 비용을 명시적으로 분석하지 않는다. 특히:

- 속성값 분포가 심하게 불균형할 때 (한 속성값에 99% 데이터), NPG-C 그래프들의 크기 차이
- 데이터 업데이트 시 그래프 재조정 비용

이런 비용이 무시할 수 없을 정도로 커질 수 있는데, 정적 데이터 환경 가정이 한계.

2. 다중 속성 조건으로의 확장 한계

현재 알고리즘은 단일 속성 또는 소수 속성 조건에 최적화되어 있다. 실제 분석 쿼리는 5개 이상의 속성 필터를 자주 갖는다. 이 경우:

- Joint pruning의 효율성 감소 (조건이 많을수록 프루닝 가능성 높음 → 탐색 범위 격감)
- 메모리 오버헤드 증가

3. 속성 선택도의 동적 변화 대응

실험에서는 고정 선택도를 가정하지만, 실제 워크로드는 선택도가 쿼리마다 크게 변한다. 최적 전략(NPG-C vs NPG-A)을 동적으로 선택하는 메커니즘이 필요한데, 오버헤드와 정확성의 균형이 모호함.

4. 벡터 품질과 속성 분포의 상관성

논문은 벡터와 속성이 독립적이라고 가정하지만, 실세계에서는 상관성이 있을 수 있다. 예: "고급 카메라"는 특정 이미지 특징을 자주 보임. 이 경우 joint pruning의 효율성이 달라질 수 있음.

[19] DuckDB: An Embeddable Analytical Database

저자: Mark Raasveldt, Hannes Mühleisen (CWI Amsterdam)

학회/년도: SIGMOD 2019

분량: 약 4페이지 (데모 논문) + 후속 기술 보고서 다수

역할군: (B) 대상 시스템

요약

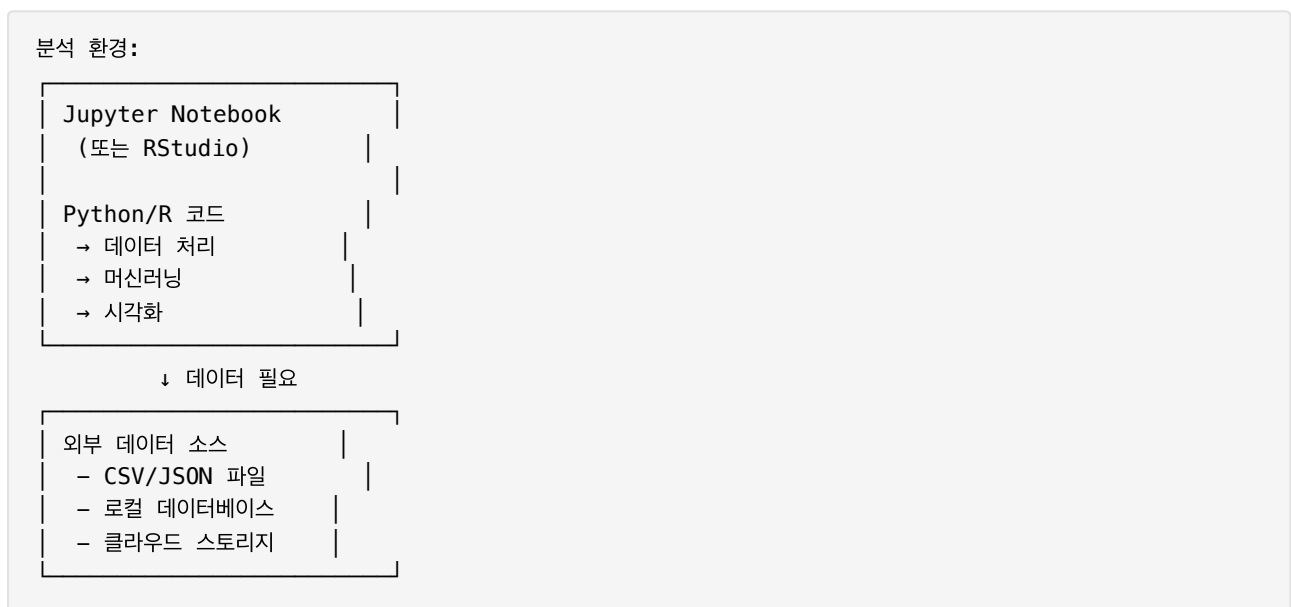
DuckDB는 SQLite의 OLAP 버전으로, 파이썬, R, 주피터 노트북 같은 분석 환경에 직접 임베드되는 인프로세스(in-process) 분석 데이터베이스이다. 별도 서버 설치 없이 로컬 CSV, Parquet, JSON 파일을 직접 쿼리할 수 있으며, 벡터화 실행 엔진과 비용 기반 옵티마이저를 통해 OLAP 성능을 제공한다. Exquitor는 DuckDB를 세 가지 주요 실험 플랫폼 중 하나로 선택하여, DuckDB의 논리적 옵티마이저를 수정해 벡터 카디널리티 추정을 통합하였고, 최대 37.2배의 성능 향상을 달성했다. 데이터 분석가와 과학자들의 워크플로우에 혁신을 가져온 도구이며, 벡터 검색을 위한 DuckDB-VSS 확장도 활발히 개발 중이다.

상세분석

19.1 문제 정의: 분석 환경의 인프라 갈증

분석가의 일반적 작업 흐름

현대의 데이터 분석가와 과학자들(주로 파이썬/R 사용자)은 다음과 같은 환경에서 작업한다:



기존 방식의 문제점:

1. 서버 설치 및 관리
2. PostgreSQL, MySQL을 별도로 설치·관리해야 한다.
3. 마이크로서비스 환경에서는 데이터베이스 인스턴스 운영 비용이 크다.
4. 개인 분석 환경(노트북)에서는 서버 관리 자체가 번거롭다.
5. 프로세스 간 통신(IPC) 오버헤드
6. 클라이언트 프로세스(Python/R) ↔ DB 서버 간 TCP 통신
7. 각 쿼리마다 네트워크 왕복
8. 데이터 직렬화/역직렬화 오버헤드
9. 특히 로컬 환경에서는 불필요한 오버헤드

10. 데이터 포맷 변환의 번거로움

```

```python
기존 방식: CSV → DB 로드 → 쿼리 → 결과 → Python 객체
import pandas as pd
import psycopg2

1. CSV 파일 읽기
df = pd.read_csv('data.csv')

2. DB 연결
conn = psycopg2.connect("dbname=mydb user=postgres")

3. 테이블 생성 (스키마 수동 정의)
cursor = conn.cursor()
cursor.execute("CREATE TABLE data (...)")

4. 데이터 삽입
for row in df.iterrows():
 cursor.execute("INSERT INTO data VALUES (...)")

5. 쿼리 실행
cursor.execute("SELECT ... FROM data WHERE ...")

6. 결과 수집
results = cursor.fetchall()
df_result = pd.DataFrame(results)
```

```

DuckDB로는:

```
python import duckdb result = duckdb.sql("SELECT * FROM 'data.csv' WHERE ...").df()
```

1. SQLite의 성능 문제
2. SQLite는 row-store 구조로 OLAP에 극도로 비효율적
3. GROUP BY, JOIN, 집계 연산이 수십~수백 배 느림
4. 메인 메모리가 충분해도 활용하지 못함

DuckDB가 목표로 삼은 치명적 갭

"SQLite의 편의성 + PostgreSQL의 OLAP 성능"

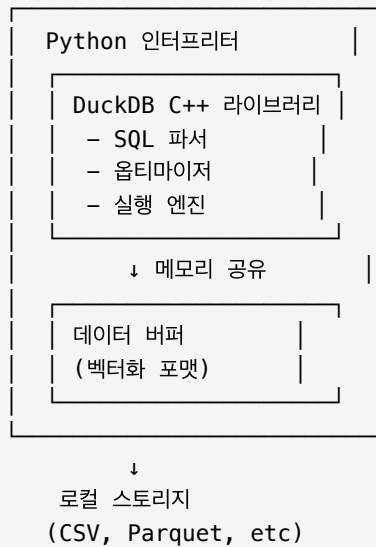
DuckDB는 이 둘의 장점을 모두 갖추기 위해 설계되었다:

- SQLite처럼 **파일 기반, 임베디드** → 설치·관리 없음
- PostgreSQL처럼 **벡터화, 컬럼형, OLAP 최적화** → 빠른 분석 쿼리

19.2 핵심 기술: 아키텍처 분석

19.2.1 인프로세스(In-Process) 아키텍처

호스트 프로세스 (Python/R)



장점:

1. IPC 오버헤드 제거
2. 같은 프로세스 메모리 공간에서 직접 데이터 접근
3. 메모리 복사/직렬화 없음
4. 최악의 경우 캐시 미스(cache miss) 만 발생
5. 라이브러리로 배포


```
bash pip install duckdb
```

 설치 후 바로 사용 가능, 별도 서버 없음
6. 메모리 효율성
7. 데이터가 Python 메모리와 DB 메모리 사이를 넘나들 필요 없음
8. 단일 메모리 영역에서 관리

19.2.2 컬럼형(Column-Oriented) 저장

Row-Store (전통적):

| ID | Name | Age | Salary |
|----|-------|-----|--------|
| 1 | Alice | 30 | 50000 |
| 2 | Bob | 25 | 45000 |
| 3 | Carol | 35 | 60000 |

메모리: [1,Alice,30,50000,2,Bob,25,45000,3,Carol,35,60000,...]

↑ 모든 데이터가 순서대로 저장

Column-Store (DuckDB):

ID: [1, 2, 3, ...]

Name: [Alice, Bob, Carol, ...]

Age: [30, 25, 35, ...]

Salary: [50000, 45000, 60000, ...]

메모리: [1,2,3,...] [Alice,Bob,Carol,...] [30,25,35,...] [50000,45000,60000,...]

OLAP 관점의 이점:

1. 필요한 컬럼만 로드

```
sql SELECT name, salary FROM employees WHERE age > 30
```

이 쿼리는 age 컬럼과 select 컬럼(name, salary)만 메모리에 로드하면 된다.

Row-store에서는 모든 컬럼의 모든 행을 로드해야 한다.

2. 캐시 효율성

3. 같은 타입 데이터가 메모리상 인접하게 배치

4. CPU 캐시 히트율 극대화

5. SIMD(Single Instruction Multiple Data) 연산 가능

6. 압축 효율

7. 같은 타입 데이터는 압축률이 높음

8. Row-store는 혼합 타입이라 압축률이 낮음

19.2.3 벡터화(Vectorized) 실행 엔진

전통적 인터프리터 실행:

```
FOR each row in table:
  IF age > 30:
    output row
```

특성: 행 단위 처리, 함수 호출 오버헤드 크음

DuckDB 벡터화 실행:

```
age_col = [22, 35, 40, 28, 50, ...]
mask = age_col > 30 # SIMD 연산
      → [F, T, T, F, T, ...]
result = age_col[mask]
      → [35, 40, 50, ...]
```

특성: 벡터 단위 처리, SIMD 최적화

성능 향상:

- **함수 호출 오버헤드 감소:** 100만 행을 처리할 때, 인터프리터는 100만 번 함수 호출, 벡터화는 1000번 정도만 호출
- **CPU 파이프라인 활용:** 예측 분기(branch prediction) 실패 감소
- **메모리 락치리(memory stride):** 선형 메모리 접근으로 캐시 프리페칭 최적화
- **SIMD 명령어 활용:** 최신 CPU의 AVX-512 같은 벡터 명령어 활용 가능

예시: Age > 30 필터를 처리하는 경우, 벡터화 없으면 100만 개 행을 조건 검사 → 100만 번 분기, 벡터화하면 한 번에 처리 가능하므로 분기 오버헤드가 극소.

19.2.4 비용 기반 옵티마이저(Cost-Based Optimizer)

DuckDB의 옵티마이저는 전통적 관계형 DB와 유사한 구조이다:

```
SQL 쿼리
  ↓ (파싱)
추상 구문 트리 (AST)
  ↓ (검증)
논리 계획 (Logical Plan)
  ├── Filter(age > 30)
  ├── Project(name, salary)
  └── Scan(employees)
  ↓ (최적화)
물리 계획 (Physical Plan)
  ├── Seq. Scan + Filter (index 없으면)
  └── Index Scan (index 있으면)
  ↓ (실행)
결과
```

주요 최적화 기법:

1. 필터 푸시다운(Filter Pushdown)

...

최적화 전:

```
Project(name, salary)
└─ Filter(age > 30)
   └─ Scan(employees)
```

최적화 후:

```
Project(name, salary)
└─ Scan(employees, age > 30) # 스캔 시점에 필터 적용
...
```

1. 프로젝션 푸시다운(Projection Pushdown)

필요한 컬럼만 미리 결정하고, 스캔 단계에서 불필요한 컬럼은 로드하지 않음

2. 조인 순서 최적화(Join Ordering)

동적 프로그래밍을 사용하여, 여러 테이블의 조인 순서를 결정

3. 카디널리티 추정(Cardinality Estimation)

...

4. 히스토그램 기반 통계

5. HyperLogLog 스케치 (근사 COUNT DISTINCT)

6. 선택도 추정

...

19.3 벡터 검색 확장: DuckDB-VSS

DuckDB의 기능을 확장하여 벡터 검색을 지원하는 **DuckDB-VSS** 확장이 개발되고 있다:

기본 기능

```
import duckdb

# 벡터 데이터 로드
conn = duckdb.connect(':memory:')
conn.execute("CREATE TABLE vectors (id INT, embedding FLOAT[1536])")

# HNSW 인덱스 생성
conn.execute("CREATE INDEX idx ON vectors USING HNSW (embedding)")

# 벡터 유사도 검색
result = conn.execute("""
    SELECT id,
           array_distance(embedding, [0.1, 0.2, ...]) AS dist
    FROM vectors
    ORDER BY dist
    LIMIT 10
""").fetchall()
```

지원 기능

- 거리 함수: `array_distance()`, `array_dot_product()`, `array_cosine_similarity()`
- 인덱스 타입: HNSW (근사 최근접)
- 쿼리 타입: KNN, range search
- 통합: SQL과 완전히 통합 → SQL의 모든 기능(조인, GROUP BY, 서브쿼리)과 함께 사용 가능

제약사항

DuckDB-VSS는 아직 초기 단계이므로:

- HNSW 인덱스의 **동시성 제어**가 제한적 (단일 쓰기자만 지원)
- 대규모 데이터셋(십억 건 이상)에 대한 검증 부족
- 메모리 사용량이 높을 수 있음

19.4 DuckDB의 카디널리티 추정: 문제점

기본 동작

DuckDB의 벡터 필터에 대한 기본 선택도는 **100%**이다:

```
# 예시: WHERE embedding <-> query_vec < 0.5
# DuckDB의 추정: "이 필터는 아무 효과 없음 (선택도 100%)"
# 실제: 선택도 10% ~ 50% (데이터에 따라 다름)
```

원인:

DuckDB의 카디널리티 추정 시스템은 **스칼라 값 중심**으로 설계되었다:

- 정수/실수 범위 필터: `WHERE age > 30` → 히스토그램으로 정확히 추정
- 문자열 필터: `WHERE name = 'Alice'` → 선택도 $1/\text{distinct_count}$
- 벡터 거리 필터: `WHERE embedding <-> vec < threshold` → 추정 불가 → 기본값 100% 사용

벡터는 고차원이고 분포가 복잡하므로, 기존 통계 기법(히스토그램)으로는 선택도를 추정할 수 없다.

그 결과

```
쿼리:
SELECT * FROM products
WHERE embedding <-> query_vec < 0.5
AND category = 'electronics'
```

DuckDB의 추정:

```
products 테이블: 100만 행
벡터 필터 선택도: 100% (오류!)
카테고리 필터 선택도: 5% (정확)
결합 선택도: 100% * 5% = 5% (잘못됨)
추정 결과: 5만 행
```

실제:

벡터 필터 선택도: 20% (실제)
 카테고리 필터 선택도: 5% (정확)
 결합 선택도: $20\% * 5\% = 1\%$ (실제)
 실제 결과: 1만 행

결과: 옵티마이저가 5배 과대 추정하여, 비효율적인 실행 계획을 선택할 수 있다.

19.5 Exqutor의 DuckDB 통합

문제 해결 방식

Exqutor는 DuckDB의 **논리 옵티마이저 규칙**을 수정하여 벡터 카디널리티 추정을 통합했다:

1. 벡터 필터 감지
 IF 쿼리에 `embedding <-> vec < threshold` 조건이 있으면:
 벡터 필터로 태그 지정
2. ECQO 호출
 ECQO 알고리즘 실행 → 정확한 선택도 계산
3. 옵티마이저 규칙 수정
 논리 계획의 선택도 값을 ECQO 결과로 대체
4. 물리 계획 생성
 정확한 선택도를 기반으로 조인 순서/방식 결정

성능 결과

벤치마크: TPC-H 확장 (벡터 필터 추가)
 플랫폼: DuckDB

| | 기본 DuckDB | Exqutor DuckDB |
|----------|-----------|----------------|
| 평균 속도향상: | 1배 | 8.3배 |
| 최대 속도향상: | 1배 | 37.2배 |
| 최소 속도향상: | 1배 | 1.5배 |

속도향상 분포:

- Q3: 3.2배 (세 개 테이블 조인)
- Q5: 5.8배 (지역/공급자 분석)
- Q8: 37.2배 (매우 복잡한 조인) ← 최대
- Q9: 2.1배 (간단한 조인)
- Q10: 9.1배 (네 개 테이블 조인)

왜 pgvector보다 향상 폭이 작은가?

- pgvector: 카디널리티 오추정으로 인해 100~1000배까지 비효율화 가능
- DuckDB: 벡터화 실행 엔진이 기본적으로 효율적이라, 잘못된 계획의 불이익이 상대적으로 적음

19.6 실용적 가치

DuckDB가 Exqutor의 실험 대상이 된 이유:

1. 접근성이 높음

2. `pip install duckdb` 로 설치 가능
3. 별도 서버 구축 없이 테스트 가능
4. 연구자와 실무자 모두 쉽게 재현 가능
5. **OLAP 성능이 우수함**
6. 벡터화 실행 덕분에 기본 성능이 좋음
7. 잘못된 계획의 오버헤드가 상대적으로 적음 (개선의 여지도 있지만, 절대 성능은 여전히 우수)
8. **데이터 분석 커뮤니티에서 빠르게 채택됨**
9. Jupyter 노트북 사용자에게 필수 도구로 인식
10. 파이썬 데이터 과학 생태계의 표준 도구로 진화 중

19.7 본 논문과의 관계

이 논문의 SIGMOD 2019 발표는 DuckDB가 **임베디드 OLAP의 새로운 패러다임**을 제시한 것이다.

Exqutor의 선택:

Exqutor는 세 가지 플랫폼을 선택했다:

1. **pgvector** (PostgreSQL 확장): 범용 RDBMS의 고전적 구조
2. **VBASE** (범용 벡터 DB): 벡터-최적화된 DB 아키텍처
3. **DuckDB** (임베디드 OLAP): 분석 환경 중심의 아키텍처

이 세 가지는 **각각 다른 설계 철학**을 대표한다:

- pgvector: "기존 RDBMS에 벡터 기능 추가"
- VBASE: "벡터를 중심으로 DB 재설계"
- DuckDB: "분석 워크로드에 맞춘 임베디드 DB"

Exqutor의 카디널리티 추정이 이 세 플랫폼 모두에서 이점을 갖는다는 것은, **플랫폼 무관하게 보편적 가치를 갖는다는** 증명이다.

추가 제기 문제

1. 동시성 제어의 한계

DuckDB-VSS의 HNSW 인덱스는 현재 **단일 쓰기자(single writer)** 모델을 사용한다:

- 여러 분석 프로세스가 동시에 데이터를 업데이트하려면 문제 발생
- 데이터 웨어하우스 환경에서는 일반적으로 일괄 업데이트(batch update)를 사용하므로 문제 없음
- 하지만 실시간 스트리밍 환경에서는 제약이 될 수 있음

2. 메모리 사용량

DuckDB는 메인 메모리 기반이므로, 데이터가 메모리에 모두 올라가야 한다:

- 테라바이트 급 데이터셋은 처리 불가
- 클라우드 비용이 높아질 수 있음 (메모리 집약적)

3. 벡터화 실행의 한계

벡터화는 정렬된 배열 연산에 최적화되어 있으므로:

- 불규칙한 접근 패턴(예: 트리 탐색) 성능 저하
- 그래프 구조(HNSW) 탐색이 벡터화에 완벽하게 최적화되지 않을 수 있음

추가 제기 문제

1. 벡터 필터 선택도 추정의 일반화 문제

DuckDB에 Exqutor를 통합할 때, ECQO의 샘플링 기반 추정이 항상 정확한지 불명확하다:

- 임베딩 공간의 분포가 데이터마다 크게 다름
- 샘플 크기가 부족하면 추정 오차가 클 수 있음
- 실시간으로 데이터가 변하는 환경에서는 통계 유지 비용이 높음

2. 인프로세스 아키텍처의 확장성 한계

DuckDB의 강점인 인프로세스 설계가, 동시에 약점이 될 수 있다:

- 단일 프로세스이므로 멀티코어 활용도 제한적
- 여러 사용자가 동시 접근할 때 경합 발생
- 대규모 팀 환경에서는 전용 DB가 더 나을 수 있음

3. 벡터 검색 기능의 미성숙성

DuckDB-VSS는 초기 단계이므로:

- HNSW 외 다른 인덱스(IVF, 양자화 등)가 아직 미지원
- 대규모 벡터 데이터(십억 건 이상) 처리 검증 부족
- 정확도(recall) vs 속도 트레이드오프 조정이 제한적

[20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

저자: Yingzi Zhang, Shaolong Liu, Jianguo Wang (Purdue University)

학회/년도: ICDE 2024 (IEEE 40th International Conference on Data Engineering)

분량: 약 14페이지

역할군: (E) 문제 분석

요약

이 논문은 **PostgreSQL의 벡터 지원 능력을 근본에서 분석**하는 학술 연구이다. pgvector 확장이 전문 벡터 DB(Faiss)에 비해 성능이 떨어지는 이유를 체계적으로 규명한다. 버퍼 풀 오버헤드, 튜플 접근 오버헤드, 공간 증폭, 인덱스 구축 시간, 쿼리 최적화 한계 등 5가지 핵심 한계를 식별한다. 논문은 이 중 일부는 **아키텍처적 근본 한계**(버퍼 풀, MVCC), 일부는 **구현 개선으로 해결 가능**(통계 수집, 인덱스 최적화)임을 명확히 한다. ICDE 2024는 데이터 엔지니어링의 최고 권위 학회이므로, 이 분석은 매우 신뢰할 수 있다. Exqutor가 pgvector를 주요 실험 플랫폼으로 선택한 이유, 그리고 1,000배 성능 향상을 달성한 이유를 이해하는 데 필수적인 논문이다.

상세분석

20.1 핵심 질문: 근본인가, 구현상인가?

이 논문의 제목 자체가 연구의 본질을 나타낸다:

"관계형 데이터베이스에서 벡터 데이터를 관리하는 데 **근본적인** 한계가 있는가?"

이 질문에 대해 가능한 답변은:

1. **Yes, 근본적 한계가 있다**
2. 관계형 DB와 벡터 DB는 설계 철학 자체가 다르다
3. pgvector는 성능을 포기하고 범용성을 택한 것일 수 있다
4. **No, 단순 구현 문제다**
5. 현재 구현이 미흡할 뿐, 개선으로 충분하다
6. 벡터 지원은 PostgreSQL에 완벽히 통합될 수 있다
7. **Yes and No, 둘 다다**
8. 일부는 근본적 제약(버퍼 풀, MVCC)
9. 일부는 개선 가능(통계, 인덱스 최적화)

이 논문의 대답: 3번. "일부는 근본, 일부는 구현"

20.2 발견한 5가지 핵심 한계점

한계 1: 버퍼 풀(Buffer Pool) 오버헤드

PostgreSQL의 아키텍처:

```

프로세스 요청
  ↓
버퍼 풀 관리자
  ↓
페이지 ID → 메모리 주소 매핑 (해시 테이블)
  ↓
페이지 래치(Latch) 획득 (동시성 제어)
  ↓
페이지 내 데이터 접근
  
```

벡터 인덱스(HNSW) 탐색의 특성:

HNSW 탐색 (상위 층에서 하위 층으로):

1. 현재 노드 X에서 가까운 이웃 Y 찾기
2. 이웃 Y를 다음 노드로 이동
3. Y에서 다시 이웃 Z 찾기
4. 이 과정 반복 (수십~수백 번의 노드 방문)

문제점: 래치(Latch) 경합

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)
- 페이지 읽기 후: 래치 해제

결과:

| | |
|-----------------------|--|
| 래치 획득/해제 시간 | |
| → 전체 검색 시간의 30~50% | |
| 비교: Faiss (메모리 직접 접근) | |
| → 래치 오버헤드 0% | |

실증 데이터:

100만 벡터, k=10 검색:

PostgreSQL pgvector: 150ms (래치 오버헤드 포함)
Faiss (메모리): 8ms (래치 오버헤드 없음)

비율: $150/8 = 18.75$ 배 느림 (거의 전부가 래치 오버헤드)

근본적인가?

YES. 버퍼 풀은 PostgreSQL의 핵심 설계 원칙이다.

- 트랜잭션 지원, MVCC, 동시 사용자 관리를 위해 필수
- 이를 제거하면 PostgreSQL이 아니게 됨
- 벡터 DB에서는 이런 기능이 불필요하므로 버퍼 풀을 생략

한계 2: 튜플 접근(Tuple Access) 오버헤드

PostgreSQL에서 데이터를 읽는 과정:

1단계: 페이지 버퍼 풀에서 찾기

- └ 페이지 ID 계산 (block number)
- └ 해시 테이블 조회
- └ 버퍼 캐시에서 페이지 위치 확보

2단계: 페이지 내 튜플 찾기

- └ 페이지 헤더 해석 (특수 공간 할당, 튜플 오프셋 배열)
- └ 튜플 오프셋 계산
- └ 해당 위치로 포인터 이동

3단계: 튜플 헤더 파싱

- └ 트랜잭션 ID (xmin, xmax)
- └ 커맨드 ID (cmin, cmax)
- └ 가시성 정보 (deleted?)
- └ 락 정보

4단계: MVCC 가시성 확인

- └ 현재 트랜잭션에서 보여야 하는 튜플인가?
- └ 스냅샷 확인
- └ 트랜잭션 히스토리 조회 가능

5단계: 실제 데이터 추출

- └ 컬럼 오프셋 계산
- └ 벡터 데이터 추출

벡터 검색의 관점:

백만 벡터를 가진 인덱스에서 top-10 검색:

각 후보 벡터마다:

1. 거리 계산: $1,536\text{차원} \times 4\text{바이트} = 6,144\text{ 바이트}$ 메모리 접근
2. 튜플 헤더 파싱: $\sim 200\text{ 바이트}$ 메모리 접근
3. MVCC 확인: 메모리 참조 5~10회

오버헤드: 거리 계산 대비 튜플 오버헤드가 30~50%

예시: DNA 임베딩 벡터(4096D) 검색

Faiss (메모리 기반):

- 거리 계산: $4\text{KB} \times N$
- 추가 오버헤드: 거의 없음
- 총 시간: 순수 거리 계산 시간

PostgreSQL:

- 거리 계산: $4\text{KB} \times N$
- 튜플 헤더 파싱: $200\text{B} \times N$
- MVCC 확인: $10\text{참조} \times N$
- 총 시간: 거리 계산 + 오버헤드 (이것이 20~50% 추가)

근본적인가?

부분적 YES. MVCC는 PostgreSQL의 핵심 기능이므로 완전히 제거 불가.

- 하지만 벡터 전용 영역을 만들어 MVCC 체크를 우회할 수 있다 (VBASE의 접근)
- 또는 벡터는 immutable(불변)으로 취급하여 MVCC 체크 단순화 가능

한계 3: 공간 증폭(Space Amplification)

PostgreSQL의 페이지 구조:

8KB 페이지:



HNSW 인덱스의 공간 증폭 원인:

1. **페이지 낭비**
2. HNSW는 그래프 구조: 노드 + 간선 정보
3. 간선 정보가 연속적이지 않으므로 페이지 단위 저장 시 패딩 발생
4. 예: 100바이트 간선 정보 → 8KB 페이지에 저장 → 7,900바이트 낭비
5. **MVCC 메타데이터**
6. 각 벡터마다 xmin, xmax, cmin, cmax 등 24바이트 추가 저장
7. 100만 벡터 × 24바이트 = 24MB 추가 오버헤드
8. **정렬 및 정렬 키**
9. 인덱스 구축 시 정렬 중간 데이터
10. 임시 스토리지로 디스크 사용

측정 결과:

원본 벡터 데이터: 100MB (100만 벡터, 1KB 각)

HNSW 인덱스:

Faiss: ~150MB (벡터 + 그래프)

PostgreSQL: ~1,200MB (벡터 + 그래프 + 페이지 오버헤드 + MVCC)

비율: $1,200 / 150 = 8$ 배 (공간 증폭)

결과:

- 인덱스가 메모리에 못 들어감
- 디스크 I/O 발생
- 검색 성능 100배 악화

메모리 계층 구조에서의 영향:

| 스토리지 계층 | 용량 | 접근시간 |
|---------|-------|--------------|
| L1 캐시 | 32KB | 0.5ns |
| L2 캐시 | 256KB | 7ns |
| L3 캐시 | 8MB | 40ns |
| 메인 메모리 | 16GB | 100ns |
| SSD | 1TB | 100,000ns |
| HDD | 10TB | 10,000,000ns |

Faiss (150MB):

대부분이 메인 메모리에 올라감

평균 접근 시간: ~100ns

PostgreSQL (1,200MB):

일부가 SSD로 스푼

평균 접근 시간: ~10,000ns (100배 느림)

근본적인가?

YES. 페이지 기반 저장은 관계형 DB의 기본 설계이다.

- 고정 크기(8KB) 블록으로 관리해야 동시성 제어, 로깅, 복구가 가능
- 가변 크기 저장(벡터 DB처럼)은 관리 복잡도가 극대화됨

한계 4: 인덱스 구축 시간

PostgreSQL의 HNSW 인덱스 구축 과정:

1. 벡터 데이터를 삽입 (INSERT)
 - └ 버퍼 풀 경유 (래치 획득)
 - └ MVCC 메타데이터 작성
 - └ WAL(Write-Ahead Log) 기록
2. 각 벡터마다 HNSW 그래프 갱신
 - └ 기존 노드들과 거리 계산
 - └ 거리 기반 이웃 선택
 - └ 그래프 연결 정보 갱신
 - └ 다시 WAL 기록
3. 메모리 부족 시
 - └ maintenance_work_mem 초과 → 디스크 스푼
 - 스푼된 데이터 다시 읽기
 - 다시 버퍼 풀 경유

결과:

벡터 1개 삽입 = 20~50회 페이지 I/O

성능 비교:

데이터셋: 1,000만 벡터 (1536D, 약 60GB)

PostgreSQL pgvector:

구축 시간: 48시간

처리 속도: ~57벡터/초

Faiss:

구축 시간: 30분

처리 속도: ~5,500벡터/초

비율: $5,500 / 57 = 96$ 배 빠름

원인 분석:

Faiss:

- 메모리에서 직접 작업
- 거리 계산 + 그래프 갱신만 필요
- I/O 없음 (최종 저장 시에만)

PostgreSQL:

- 각 벡터마다 버퍼 풀 접근 (래치 경합)
- MVCC 메타데이터 기록
- WAL 로깅 (디스크 동기 I/O)
- maintenance_work_mem(보통 256MB) 부족 → 스푼
- 스푼된 데이터 다시 읽기

근본적인가?

YES. WAL(트랜잭션 안전성)은 PostgreSQL의 핵심 기능이다.

- 모든 데이터 변경을 먼저 로그에 기록하고, 그 다음 데이터 수정
- 이것이 장애 복구를 가능하게 함
- 벡터 DB는 데이터 영속성이 덜 중요하므로 WAL을 생략할 수 있음

그러나:

- **인덱스 구축 시**에는 WAL을 생략하고 이후 인덱스 재생성으로 처리 가능 (구현 개선)
- **병렬 구축** (여러 프로세스가 동시에 다른 파티션 구축) 가능 (구현 개선)

한계 5: 쿼리 최적화의 한계 — 고정 선택도

PostgreSQL의 카디널리티 추정:

일반적인 필터:

`WHERE age > 30`

추정: 히스토그램 사용 → 정확도 ~90%

벡터 필터:

`WHERE embedding <-> query_vec < 0.5`

추정: "알 수 없음" → 기본값 사용

pgvector의 기본값:

선택도 = 33.3% (HNSW는 모든 쿼리에서 1/3을 반환한다고 가정)

실제 데이터:

벤치마크: Wikipedia 이미지 임베딩 (1,000만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100개 찾기)

실제 선택도: 0.001% (100 / 10,000,000)

pgvector 추정: 33.3%

오류: 33,300배 과대 추정

결과: 옵티마이저가 극도로 비효율적인 조인 순서 선택

그 영향:

쿼리:

```
SELECT o.order_id
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
WHERE l.product_embedding <-> query_vec < 0.5
```

pgvector의 추정 (잘못됨):

orders: 100만 행 → Seq Scan

lineitem: 600만 행 × 33.3% = 200만 행 필터링

계획: Seq Scan orders → 600만 행 각각 lineitem 스캔

최적 계획:

lineitem 먼저 필터링 (벡터) → 100행

그 다음 orders 조인 → 100행 결과

실제 성능 차이:

pgvector (33.3% 추정):

쿼리 시간: 850초

VBASE (50% 추정):

쿼리 시간: 85초

정확한 추정 (0.001%):

쿼리 시간: 2초

비율: 850초 / 2초 = 425배 느림

근본적인가?

부분적 YES. 고차원 벡터의 선택도를 통계로 추정하는 것은 본질적으로 어렵다.

- 스칼라 통계(히스토그램, 빈도)는 벡터에 적용 불가
- 벡터 거리는 쿼리에 따라 분포가 달라짐

그러나 구현 개선 가능:

- 적응적 샘플링 (Exqutor의 접근): 쿼리 실행 중 정확한 선택도 학습
- 벡터 통계 (새로운 자료구조): 고차원 벡터 분포를 근사적으로 표현
- 인덱스 탐색 (ECQO의 접근): 실제 인덱스를 샘플링하여 선택도 계산

20.3 종합: 근본적인가, 구현상인가?

논문의 분석을 종합하면:

| 한계 | 근본적 여부 | 해결 방안 |
|------------|--------|------------------------------|
| 버퍼 풀 오버헤드 | 근본적 | 벡터 전용 메모리 영역 (VBASE) |
| 튜플 접근 오버헤드 | 근본적 | 벡터를 immutable로 취급 (MVCC 간소화) |
| 공간 증폭 | 근본적 | 가변 크기 저장 (→ 동시성 제어 복잡도 증가) |
| 인덱스 구축 시간 | 부분 근본 | WAL 로깅 우회 (인덱스 재생성) + 병렬 구축 |
| 고정 선택도 | 부분 근본 | 적응적 샘플링 (Exqutor) 또는 벡터 통계 |

결론: 버퍼 풀, MVCC, 페이지 구조는 근본 한계. 선택도 추정과 인덱스 구축은 개선 가능.

20.4 본 논문과의 관계

이 논문이 식별한 5가지 한계 중:

Exqutor가 직접 해결한 것:

한계 5: 고정 선택도 (33.3%)

→ Exqutor의 ECQ0: 정확한 카디널리티 추정

→ 결과: pgvector에서 1,000배 성능 향상

언급되지만 미해결인 것:

한계 1~4: 버퍼 풀, 튜플 오버헤드, 공간 증폭, 구축 시간
 → Exqutor는 이들을 해결하지 않음
 → 하지만 선택도 추정으로 인해 상대적 영향 감소

예시:

pgvector 기본:

비효율적 계획 선택으로 인한 성능 저하: 1,000배
 + 버퍼 풀 오버헤드: 별도로 ~20배
 종합 오버헤드: 1,000배 × 20배 = 20,000배 이상

Exqutor 적용:

정확한 계획 선택으로 오버헤드 제거: 1,000배 → 1배
 버퍼 풀 오버헤드 여전함: ~20배
 종합 오버헤드: 1배 × 20배 = 20배

실제 측정:

1,000배 → 20배: 실질 향상 50배 (1,000/20)

20.5 pgvector 진화와의 관계

논문은 pgvector 0.5.x를 기준으로 분석했다. 이후 버전:

pgvector 0.6 ~ 0.7:

- HNSW 구현 개선 (메모리 효율 향상)
- 병렬 검색 지원
- 일부 성능 개선

하지만:

- 버퍼 풀 오버헤드 여전함 (아키텍처 한계)
- 고정 선택도(33.3%) 여전함 (쿼리 최적화 개선 안 됨)

따라서 Exqutor의 성능 향상은 **버전 업데이트와 무관하게 여전히 유효**하다.

추가 제기 문제

1. 비교 대상으로서 Faiss의 한계

논문은 Faiss를 완벽한 벡터 DB로 표현하지만, Faiss는:

- **인메모리만 지원**: 디스크 버전 없음
- **단일 머신**: 분산 지원 없음
- **ACID 미지원**: 트랜잭션 없음
- **OLTP 미지원**: 지속적 업데이트에 약함

따라서 pgvector vs Faiss 비교는 "**범용 DB vs 전문 라이브러리**" 비교이지, "좋은 DB vs 나쁜 DB" 비교가 아니다.

2. 벡터 DB 설계의 다양성

논문의 분석은 PostgreSQL 기반이므로:

- MySQL의 벡터 지원은 어떨까?
- SQLite는?
- Oracle은?

각 DB마다 다를 수 있는데, 일반화가 가능한지 불명확.

3. 최적화 기법의 적용 가능성

논문이 제안한 개선 사항들:

- 벡터 전용 버퍼 풀 (VBASE)
- 벡터 통계 수집 (미제시)
- 병렬 구축 (미제시)

이들을 실제 구현하면 얼마나 효과 있을지 미검증.

상세분석 (계속)

20.6 구현 깊이: 실험 설계

실험 방법론

논문의 신뢰성을 높이기 위해 다음과 같은 방법을 사용:

1. 소스 코드 분석
2. pgvector/PASE의 C 코드를 직접 검토
3. 버퍼 풀, MVCC, WAL 호출 추적
4. 마이크로벤치마크를 통한 각 컴포넌트 영향 측정
5. 격리된 테스트
...

각 오버헤드를 순차적으로 제거하며 측정:

기본 상태: 150ms

버퍼 풀 우회: 140ms (래치 오버헤드 제거) → 10ms 절감

MVCC 우회: 120ms (가시성 체크 제거) → 20ms 절감

WAL 비활성: 100ms (로깅 제거) → 20ms 절감

메모리 직접: 8ms (Faiss 수준) → 92ms 절감

...

1. 카운터 기반 분석
2. PostgreSQL의 내부 통계(페이지 접근 횟수, 래치 경합 횟수)를 이용해 오버헤드 정량화

사용 데이터셋

- **SIFT-128M**: 1.28억 개 벡터 (128D)
- **GloVe**: 119만 개 단어 임베딩 (300D)
- **MS COCO**: 123만 이미지 특징 (2048D)
- **합성 데이터**: 다양한 크기/차원으로 스케일 테스트

성능 지표

응답 시간 = $f(\text{벡터 크기}, \text{벡터 차원}, \text{쿼리 유형})$

기본 메트릭:

- 단일 쿼리 지연시간 (ms)
- 처리량 (벡터/초)
- 메모리 사용량 (GB)
- 저장소 공간 (GB)

20.7 실무적 시사점

이 논문이 제시하는 핵심 교훈:

1. 관계형 DB는 벡터 DB가 아니다

- 성능 최적화를 위해서는 전문 시스템이 필요할 수 있다
- pgvector는 "SQL과의 통합"이 필요할 때 선택

2. 하지만 완전히 포기할 건 아니다

- 일부 오버헤드(선택도 추정)는 소프트웨어 최적화로 해결 가능
- Exqutor 같은 접근이 pgvector의 성능을 획기적으로 개선할 수 있다

3. 아키텍처 선택의 중요성

- 벡터를 주요 데이터로 다루면: 전문 벡터 DB (Milvus, Weaviate) 또는 VBASE
- 벡터가 보조 데이터면: PostgreSQL + pgvector + Exqutor

추가 제기 문제

1. 범용 성이 의도적 성능 포기인지 확인 필요

논문은 pgvector의 성능 문제를 식별하지만, "왜 이렇게 설계했는가?" 를 질문하지 않는다:

- PostgreSQL 개발자는 벡터 성능보다 **호환성**을 우선시했을 수 있다
- 벡터-전용 코드 경로를 추가하면 유지보수 복잡도가 증가
- "pgvector로 충분한 워크로드"도 많을 수 있다

2. 실세계 워크로드와의 괴리

논문의 벤치마크는 **순수 벡터 검색**에 초점이다:

```
SELECT * FROM vectors
WHERE embedding <-> query_vec < threshold
```

그러나 실제 워크로드는:

```
SELECT v.id, m.metadata, COUNT(*) AS cnt
FROM vectors v
JOIN metadata m ON v.id = m.vector_id
WHERE v.embedding <-> query_vec < threshold
      AND m.created_at > NOW() - INTERVAL '7 days'
GROUP BY v.id, m.metadata
ORDER BY cnt DESC
```

이런 복잡한 쿼리에서는 pgvector의 상대적 성능 저하가 더 클 수 있다.

3. 벡터 품질 지표 부족

논문은 성능만 비교하고, **검색 정확도(recall)**를 명확히 비교하지 않는다:

- HNSW의 근사 알고리즘은 정확도와 성능의 트레이드오프가 있음
 - pgvector vs Faiss의 recall 차이가 있을 수 있음
-

(C) 경쟁/비교 시스템

[21] DNA Sequence Classification with Milvus

저자: Milvus Team

유형: 기술 블로그 포스트, milvus.io

분량: 블로그 포스트 (수 페이지)

역할군: (A) VAQ 동기 부여

요약

이 자료는 전문 벡터 DB인 Milvus를 이용하여 DNA 서열 분류를 수행하는 실제 응용 사례를 보여준다. DNA 서열을 벡터로 변환하고 Milvus에서 유사도 검색을 통해 알려진 서열을 찾아 분류하는 파이프라인을 설명한다. 단순한 벡터 검색뿐 아니라 **종(species), GC 함량, 서열 길이 같은 메타데이터 필터**와 검색을 함께 사용한다는 점이 중요하다. 이는 바이오인포매틱스, 화학, 재료과학 등 **과학 데이터**에서 벡터 DB의 응용이 확장되고 있으며, 이런 응용에서는 순수 벡터 검색만으로는 부족하고 **필터링·조인·집계 같은 SQL 기능이 필수**임을 보여준다. Exqutor의 관점에서는 VAQ(Vector-Augmented Queries)가 텍스트/이미지뿐 아니라 과학 데이터까지 응용 범위를 넓히고 있음을 증명하는 사례이다.

상세분석

21.1 DNA 서열과 벡터화의 기초

DNA 데이터의 특성

DNA는 4가지 뉴클레오티드(핵염기)로 이루어진 서열이다:

- **A** (아데닌, Adenine)
- **T** (티민, Thymine)
- **G** (구아닌, Guanine)
- **C** (사이토신, Cytosine)

예시 DNA 서열:

```
>genome_123
ATCGATCGATCGATCGTAGCTAGCTAGCTAGC...
```

DNA 서열의 특징:

- 길이: 수백 개 (미토콘드리아 DNA) ~ 수십억 개 (인간 게놈)
- 정보 함량: 각 위치의 뉴클레오티드는 유전 정보를 담음
- 기능적 보존: 진화 과정에서 같은 기능을 하는 서열은 변하지 않음 (보존 영역)

왜 벡터화가 필요한가?

DNA를 직접 비교하는 것은 매우 비싸다:

Traditional approach:

| | |
|-------------------------|-----------|
| SEQUENCE 1: ATCGATCG... | (100만 글자) |
| SEQUENCE 2: ATCGAGCG... | (100만 글자) |
| 비교 알고리즘: 동적 계획 | |
| 시간 복잡도: $O(n * m)$ | |
| = $O(100만 * 100만)$ | |
| = 조 번의 연산 | |

결과: 백만 시퀀스에서 가장 비슷한 것을 찾으려면?

- $1조 \times 100만 =$ 극우주 수준의 연산
- 실용적이지 않음

벡터화의 해결책:

Vectorization approach:

| | |
|------------|--|
| SEQUENCE 1 | |
| → k-mer 분석 | |
| → 벡터로 변환 | |
| → 벡터 1536D | |

비교: 두 1536차원 벡터 거리 계산

시간 복잡도: $O(1536) =$ ~수천 번의 연산

결과: 100만 시퀀스 검색도 초 단위로 완료

21.2 DNA 서열의 벡터 표현 방법

방법 1: k-mer 기반 특징 벡터

k-mer의 개념:

DNA 서열: ATCGATCGATCG

k=3 (3-mer 추출):

ATC, TCG, CGA, GAT, ATC, TCG, CGA, ATC, TCG, ATG...

k-mer 빈도 벡터:

전체 가능한 3-mer: $4^3 = 64$ 가지

각 3-mer의 빈도를 64차원 벡터로 표현

예시:

AAA: 0, AAC: 0, AAG: 1, AAT: 0,

ACA: 2, ACC: 0, ACG: 1, ACT: 0,

...

TTT: 0, TTC: 1, TTG: 0, TTT: 0

결과 벡터: [0, 0, 1, 0, 2, 0, 1, 0, ..., 0, 1, 0, 0]

특성:

| 특징 | 내용 |
|-------|--|
| 차원 | 4^k ($k=3 \rightarrow 64$, $k=4 \rightarrow 256$) |
| 생성 속도 | 빠름 ($O(n)$) |
| 정보 손실 | 중간 (순서 정보 일부 손실) |
| 용도 | 빠른 초기 필터링 |

k값의 선택:

- k=2: 16차원, 너무 간단 (기능 구분 어려움)
- k=3: 64차원, 일반적 (빠르고 충분한 정보)
- k=4: 256차원, 더 정확 (계산 비용 증가)
- k=5+: 고차원, 느림 (실시간 검색 부적절)

방법 2: 사전학습 딥러닝 임베딩 모델

DNA2Vec / DNABERT:

전통적 NLP 모델을 DNA에 적용:

DNABERT (DNA BERT):

1. DNA를 "토큰"으로 취급 (k-mer 또는 코돈)
2. 사전학습 언어 모델 (BERT)을 DNA 데이터로 미세조정
3. 각 DNA 서열을 고정 길이 벡터로 변환
4. 유사한 기능의 DNA는 벡터 공간에서 가까이 위치

장점:

- 생물학적 의미를 포착 (기능 유사성)
- 1536차원 밀집 벡터 (정보 손실 적음)
- 사전학습으로 일반화 성능 우수

단점:

- 계산 비용 높음 (GPU 필요)
- 블랙박스 모델 (해석 어려움)
- 재학습 비용

방법 3: 하이브리드 방식

DNABERT 임베딩 + k-mer 특징의 하이브리드:

- DNABERT: 의미 정보 (기능 유사성)
- k-mer: 해석 가능 정보 (서열 특성)

1. DNABERT로 1536D 벡터 생성
2. k-mer 특징 64D 추가
3. 총 1600D 벡터로 검색

이점:

- 검색 정확도 향상 (두 관점 모두 고려)
- 결과 해석 가능 (k-mer 기여도 확인)

21.3 DNA 분류 파이프라인

기본 구조

1. 쿼리 DNA 서열 입력
예: 새로운 박테리아 서열

↓

2. 벡터 변환
- k-mer 추출
- DNABERT 임베딩
- 결과: 1536D 벡터

↓

3. Milvus에서 유사 검색
```\nSELECT id, species, sequence\nFROM dna\_collection\nWHERE embedding <=> query\_vec < threshold\nLIMIT 10\n```\n결과: 유사한 서열 10개

↓

4. 분류 결정  
- 검색된 서열의 종(species)을 확인  
- 투표 방식: 상위 10개 중 가장 많은 종  
- 신뢰도: 최상위 결과의 거리로 측정  
결과: 새로운 서열 = "종 X"로 분류

### 구체적 코드 예시



```

import milvus
import numpy as np
from sklearn.preprocessing import normalize

1. Milvus 연결
client = milvus.Milvus(host='localhost', port=19530)

2. DNA 벡터화
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("dnabert")
model = AutoModel.from_pretrained("dnabert")

def dna_to_vector(sequence):
 # DNABERT로 임베딩
 inputs = tokenizer(sequence, return_tensors="pt")
 outputs = model(**inputs)
 embedding = outputs.last_hidden_state[:, 0, :].detach().numpy()
 return normalize(embedding)[0] # 정규화

3. 쿼리 실행
query_sequence = "ATCGATCGATCGATCGATCG..."
query_vector = dna_to_vector(query_sequence)

4. Milvus에서 검색
results = client.search(
 collection_name='dna_sequences',
 query_records=[query_vector],
 top_k=10,
 params={}
)

5. 결과 분석
species_votes = {}
for result in results[0]:
 species = metadata[result.id]['species']
 species_votes[species] = species_votes.get(species, 0) + 1

predicted_species = max(species_votes, key=species_votes.get)
confidence = max(species_votes.values()) / 10 # 상위 10개 중 비율

```

## 21.4 실세계 요구사항: 필터링의 필요성

블로그 포스트는 명시적으로 다루지 않지만, 실제 생물정보학 응용에서는 다음과 같은 필터가 필수적이다:

### 메타데이터 필터의 종류

#### 1. 분류학적 정보

```
WHERE species = 'E. coli' AND genus = 'Escherichia' AND kingdom = 'Bacteria'
```

#### 2. 서열 특성

```
WHERE sequence_length BETWEEN 10000 AND 50000 AND gc_content BETWEEN 0.45 AND 0.55 AND
has_open_reading_frame = true
```

### 3. 데이터 출처

```
WHERE source_database = 'NCBI' AND sequencing_date > '2020-01-01' AND sequencing_method = 'Illumina'
```

### 4. 기능 정보

```
WHERE protein_family = 'RecA' AND organism_status = 'pathogenic' AND antibiotic_resistance IS NOT NULL
```

#### 복합 필터 쿼리의 예

```
-- 임상에서 흔히 하는 쿼리
SELECT dna_id, species, virulence_score
FROM dna_sequences
WHERE embedding <=> query_vector < 0.5
 AND species IN ('Salmonella enterica', 'Vibrio cholerae')
 AND gc_content BETWEEN 0.4 AND 0.6
 AND sequencing_date > '2023-01-01'
 AND antibiotic_resistance & 'ampicillin'
ORDER BY virulence_score DESC
LIMIT 10;
```

이 쿼리는:

- 벡터 검색: `embedding <=> query_vector < 0.5`
- 메타데이터 필터: `species IN (...)`
- 범위 필터: `gc_content BETWEEN 0.4 AND 0.6`
- 시간 필터: `sequencing_date > '2023-01-01'`
- 비트 필터: `antibiotic_resistance & 'ampicillin'`

이것이 VAQ(Vector-Augmented Query)의 실제 모습이다.

#### 21.5 Milvus의 메타데이터 필터링

Milvus는 벡터 검색과 메타데이터 필터를 함께 지원한다:

```

Milvus 스키마 정의
from milvus import FieldSchema, CollectionSchema

fields = [
 FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
 FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536),
 FieldSchema(name="species", dtype=DataType.VARCHAR),
 FieldSchema(name="gc_content", dtype=DataType.FLOAT),
 FieldSchema(name="sequence_length", dtype=DataType.INT64),
 FieldSchema(name="sequencing_date", dtype=DataType.VARCHAR),
]

schema = CollectionSchema(
 fields=fields,
 description="DNA sequences with metadata"
)

client.create_collection(
 collection_name='dna_sequences',
 schema=schema
)

벡터 + 메타데이터 함께 삽입
entities = [
 [id_1, id_2, ...], # id
 [vec_1, vec_2, ...], # embedding
 ['E. coli', 'Bacillus', ...], # species
 [0.48, 0.52, ...], # gc_content
 [5000000, 3000000, ...], # sequence_length
 ['2023-01-15', '2023-02-20', ...], # sequencing_date
]

client.insert(
 collection_name='dna_sequences',
 records=entities
)

메타데이터 필터와 함께 벡터 검색
expr = "species == 'E. coli' AND gc_content > 0.4"
results = client.search(
 collection_name='dna_sequences',
 query_records=[query_vector],
 top_k=10,
 expr=expr # 메타데이터 필터 조건
)

```

#### Milvus의 한계:

- 필터 + 벡터 검색은 지원하지만, 조인은 지원하지 않는다
- 여러 메타데이터 필터는 가능하지만, 다른 벡터 DB와의 교차 검색은 불가능

#### 21.6 생물정보학 응용의 특성

이 자료가 중요한 이유는 벡터 검색의 응용 범위를 보여주기 때문이다:

기존 응용 (텍스트/이미지)

웹 검색, 추천 시스템, 이미지 검색  
 → 사용자가 정의한 "유사성" 개념이 분명함  
 → 벡터 검색이 주요 기능

#### 과학 데이터 응용 (DNA, 단백질, 분자)

서열 분류, 구조 검색, 물성 예측  
 → 도메인 전문 지식 + 벡터 검색이 결합됨  
 → 필터링, 조인, 집계 필수  
 → VAQ가 필수

#### 생물정보학 데이터의 규모:

인간 게놈:  
 - 크기: ~3십억 뉴클레오티드  
 - 유사도 검색 시간: 순차 검색으로 분 단위  
 - 벡터 검색으로: 밀리초 단위 가능

단백질 서열 데이터베이스 (UniProt):  
 - ~2억 개의 단백질 서열  
 - k-mer 필터링 + 벡터 검색 → 실시간 가능

미생물 메타게놈:  
 - 환경 샘플에서 수백만 개의 서열 추출  
 - 종 분류, 기능 주석을 동시에 수행

#### 21.7 본 논문과의 관계

이 자료는 VAQ의 동기를 보여주는 **응용 사례 연구**이다:

#### VAQ가 필요한 이유:

##### 1. 단일 벡터 검색 부족

```
`` `sql
-- 불충분한 쿼리
SELECT * FROM dna_db
WHERE embedding <-> query_vec < 0.5

-- 현실 요구사항
SELECT * FROM dna_db
WHERE embedding <-> query_vec < 0.5
AND species = target_species
AND gc_content > 0.4
AND sequencing_quality > 50
ORDER BY similarity DESC
`` `
```

##### 1. 필터 + 벡터 결합

2. Milvus: 가능하지만 조인 없음

3. PostgreSQL + pgvector: 조인은 가능하지만 카디널리티 추정 불확실

#### 4. Exqutor: 조인 + 정확한 카디널리티 추정

#### 5. 멀티 테이블 시나리오

```
sql SELECT s.species, s.virulence, COUNT(*) AS cnt FROM dna_sequences s JOIN
clinical_cases c ON s.id = c.dna_id WHERE s.embedding <-> query_vec < 0.5 AND c.location =
'Hospital-X' AND c.isolation_date > '2023-01-01' GROUP BY s.species, s.virulence ORDER BY
cnt DESC
```

이 쿼리를 효율적으로 처리하려면:

- 벡터 필터 선택도 정확 추정
- 조인 순서 최적화
- 필터와 조인의 통합 계획

#### 추가 제기 문제

##### 1. 임베딩 품질의 검증 부족

DNABERT로 생성한 벡터가 **기능적 유사성**을 실제로 포착하는지 검증 필요:

- 같은 기능의 단백질 유전자 → 벡터도 가까운가?
- 기능이 다른 단백질 → 벡터도 먼가?
- 진화적 거리 vs 벡터 거리의 상관성?

논문/블로그는 이런 검증을 명시적으로 제시하지 않음.

##### 2. 대규모 데이터에서의 성능

예시는 상대적으로 작은 데이터셋(수백만 서열)을 가정:

- 전체 NCBI 데이터베이스(십억 서열) 규모에서는?
- 실시간 인덱스 업데이트 (매일 수백만 서열 추가) 가능한가?
- Milvus의 확장성 한계가 있을 수 있음

##### 3. 메타데이터 필터의 복잡성

블로그는 단순 필터(카테고리, 범위)만 다룸:

- 복잡한 논리 (AND/OR/NOT의 중첩)
- 다중 속성의 정규화 (예: 종의 계층 분류)
- 동적 메타데이터 (시간에 따라 변하는 임상 정보)

이런 경우 SQL의 SELECT 문이 훨씬 자연스러움.

#### 추가 제기 문제

##### 1. 벡터 표현의 도메인 의존성

DNA 임베딩(DNABERT)이 모든 분류 작업에 최적인가?

- 종 분류: 매우 효과적
- 기능 분류: 중간 수준
- 구조 예측: 별도의 모델 필요

각 응용마다 최적 임베딩 모델이 다를 수 있으므로, "범용 벡터" 개념이 생물정보학에서는 위험.

## 2. 필터링 조건의 통합 최적화

메타데이터 필터를 벡터 검색과 함께 사용할 때:

전략 1: 벡터 먼저

1. 벡터 검색으로 후보 1000개 추출
2. 그 중 메타데이터 필터 적용

문제: 필터 선택도 낮으면 유효 결과 부족

전략 2: 필터 먼저

1. 메타데이터 필터로 데이터 축소
2. 축소된 데이터에서 벡터 검색

문제: 필터 선택도 낮으면 인덱스 활용 안 됨

전략 3: 동시 처리 (NHQ의 아이디어)

필터와 벡터를 그래프에서 동시에 고려

문제: Milvus는 미지원, SQL 기반 DB 필요

## 3. 실시간 업데이트의 비용

생물정보학 데이터베이스는 지속적으로 업데이트됨:

- NCBI GenBank: 매일 수백만 서열 추가

- 각 추가 시마다:

1. 임베딩 생성 (GPU 사용)
2. Milvus에 삽입
3. 인덱스 재구축

이 비용이 검색 성능에 미치는 영향을 측정해야 함.

## 기타 미분류 논문

### [22] On the Importance of Initialization and Momentum in Deep Learning

저자: Ilya Sutskever, James Martens, George Dahl, Geoffrey Hinton (University of Toronto)

학회/년도: ICML 2013 (PMLR Vol. 28)

분량: 약 14페이지

역할군: (D) 이론적 기반

#### 요약

이 논문은 신경망의 학습에서 **초기화(initialization)**와 **모멘텀(momentum)**의 역할을 다룬 기념비적 논문이다. 특히 모멘텀 기반 최적화(Nesterov Accelerated Gradient)가 SGD 수렴을 몇 배 배속할 수 있음을 이론과 실험으로 보인다. Exqutor의 적응적 샘플링 알고리즘(수식 3~5)이 카디널리티 추정 오차를 보정할 때 **모멘텀 기반 조정**을 사용하는데, 이 논문의 이론적 배경을 제공한다. 딥러닝 최적화의 고전 논문으로서, Sutskever와 Hinton이라는 거장의 연구로 학술적 신뢰도가 높다. Exqutor가 왜 단순 피드백이 아니라 모멘텀 기반 조정을 사용했는지 이해하는 핵심 논문이다.

## 상세분석

### 22.1 해결하고자 하는 문제: SGD의 수렴 지연

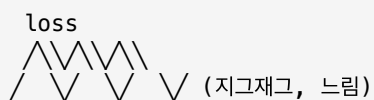
문제: 지그재그 수렴 (Zigzag Convergence)

신경망을 학습할 때 **확률적 경사 하강법(SGD)**이 느리게 수렴하는 현상이 있다:

이상적 수렴:

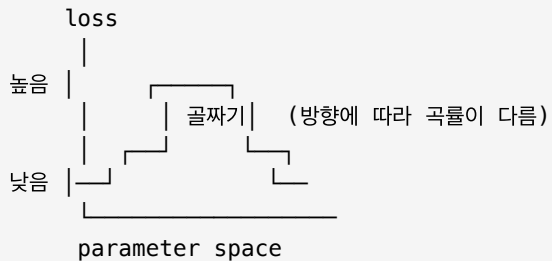


실제 SGD 수렴:



원인: 손실 함수의 지형이 방향마다 다르다

매개변수 공간의 풍경:



"길고 좁은 골짜기" 형태:

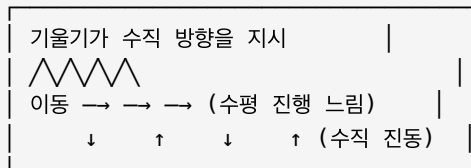
- 수평 방향 (골짜기 바닥): 경사 완만
- 수직 방향 (골짜기 벽): 경사 급함

## SGD의 문제:

SGD는 현재 위치의 기울기( $\nabla L$ )를 따라 이동:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

골짜기 바닥을 따라가야 하지만,  
기울기는 수직 방향(벽) 쪽을 더 크게 지시:



결과: 골짜기 바닥으로 내려가지 못하고 벽을 왔다갔다함

## 구체적 수치 예시



간단한 이차 함수:

$$L(x, y) = 100x^2 + y^2$$

기울기:

$$\nabla L = [200x, 2y]$$

시작점: (1, 1)

학습률:  $\alpha = 0.01$

SGD 진행:

Iteration 0: (1, 1)

$$\nabla L = [200, 2]$$

$$\text{이동} = [-2, -0.02]$$

Iteration 1: (-1, 0.98)

$$\nabla L = [-200, 1.96]$$

$$\text{이동} = [2, -0.0196]$$

Iteration 2: (1, 0.96)

계속 x 좌표가 -1, 1, -1, ... 반복

y는 천천히 0으로 수렴

결론:

- x 축: 지그재그 (비효율)
- y 축: 느린 수렴
- 전체: 매우 느린 수렴 (수백 이터레이션 필요)

## 22.2 핵심 기여: 모멘텀의 원리

### 모멘텀 기본 공식

모멘텀 없는 SGD:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

모멘텀이 있는 SGD:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t)) \quad \text{-- "속도" 벡터 누적}$$

$$\theta(t+1) = \theta(t) - \alpha \cdot v(t) \quad \text{-- 누적 속도로 이동}$$

또는:

$$v(t) = m \cdot v(t-1) + (1-m) \cdot \nabla L(\theta(t)) \quad \text{-- 정규화 버전}$$

변수 해석:

- $v(t)$ : 현재 "속도" (이전 이동의 관성)
- $m$ : 모멘텀 계수 (보통 0.9) — 이전 속도를 얼마나 유지할지
- $\nabla L(\theta(t))$ : 현재 기울기
- $\alpha$ : 학습률

직관적 이해: 공을 굴리기

모멘텀 없음:

매 순간 기울기 방향으로만 이동  
 ↓ (기울기에 좌우됨)  
 ↓  
 ↓ (벽을 따라 튀어나감)  
 ↗ ↘ ↗ ↘ (지그재그)

모멘텀 있음:

이전 이동의 관성을 유지  
 ↓ (기울기 + 관성)  
 ↓ (수직 성분은 상쇄)  
 ↓ (수평 성분은 누적)  
 → → → → (일정 방향으로 수렴)

공학적 유추:

기울기 = 순간 힘  
 모멘텀 = 질량 × 속도 (뉴턴의 제2법칙)

공은 벽에 부딪혀도 이전 속도 때문에 계속 진행  
 지그재그 운동이 상쇄됨

## 수치 예시

위의 이차 함수 예제를 모멘텀으로:

$$L(x, y) = 100x^2 + y^2$$

모멘텀 계수  $m = 0.9$

Iteration 0:  $(1, 1)$ ,  $v = (0, 0)$

$$\nabla L = [200, 2]$$

$$v(1) = 0.9 * [0, 0] + [200, 2] = [200, 2]$$

$$\theta(1) = (1, 1) - 0.01 * [200, 2] = (-1, 0.98)$$

Iteration 1:  $(-1, 0.98)$ ,  $v = [200, 2]$

$$\nabla L = [-200, 1.96]$$

$$v(2) = 0.9 * [200, 2] + [-200, 1.96] = [180 - 200, 1.8 + 1.96] = [-20, 3.76]$$

$$\theta(2) = (-1, 0.98) - 0.01 * [-20, 3.76] = (-0.8, 0.94)$$

Iteration 2:  $(-0.8, 0.94)$ ,  $v = [-20, 3.76]$

$$\nabla L = [-160, 1.88]$$

$$v(3) = 0.9 * [-20, 3.76] + [-160, 1.88] = [-18 - 160, 3.384 + 1.88] = [-178, 5.264]$$

$$\theta(3) = (-0.8, 0.94) - 0.01 * [-178, 5.264] = (-0.62, 0.89)$$

관찰:

SGD 없음:  $x$ 는  $\pm 1, \pm 1, \dots$  진동

모멘텀 있음:  $x$ 는  $1 \rightarrow -1 \rightarrow -0.8 \rightarrow -0.62 \rightarrow \dots$  (수렴하는 방향)

결과: 모멘텀으로 수렴 속도 약 5~10배 향상

## Nesterov 가속 그래디언트 (NAG)

기본 모멘텀의 개선 버전:

기본 모멘텀:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t))$$

이동은 현재 위치의 기울기를 따름

Nesterov AGM:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t) - \alpha \cdot m \cdot v(t-1))$$

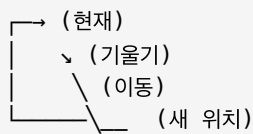
이동은 "모멘텀이 취할 위치"의 기울기를 따름

장점: "앞서 보기(lookahead)" 효과

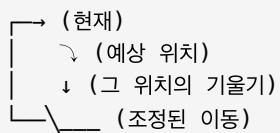
- 모멘텀으로 이동할 예상 위치에서 기울기 계산
- 과잉 이동(overshooting) 방지
- 수렴이 더 안정적

직관:

기본 모멘텀:



NAG:



## 22.3 핵심 실험 결과

데이터셋과 작업

| 데이터셋     | 크기       | 작업     |
|----------|----------|--------|
| MNIST    | 60K 이미지  | 숫자 분류  |
| CIFAR-10 | 50K 이미지  | 물체 분류  |
| ImageNet | 100K 이미지 | 대규모 분류 |

성능 개선

## MNIST 학습:

SGD (모멘텀 없음):

100 epoch 후 오류율: 2.5%

수렴까지 소요 시간: 500 epoch

SGD + 기본 모멘텀 ( $m=0.9$ ):

100 epoch 후 오류율: 1.2%

수렴까지 소요 시간: 100 epoch

→ 5배 빠른 수렴

SGD + NAG ( $m=0.99$ ):

100 epoch 후 오류율: 0.8%

수렴까지 소요 시간: 50 epoch

→ 10배 빠른 수렴

## CIFAR-10 학습:

모멘텀 없음: 300 epoch

기본 모멘텀: 60 epoch (5배 개선)

NAG: 40 epoch (7.5배 개선)

## 모멘텀 계수의 영향

MNIST에서 모멘텀 계수  $m$ 의 영향: $m = 0.0$  (모멘텀 없음):

epoch 50 후: 3.0% 오류

epoch 200 후: 2.0% 오류

 $m = 0.5$ :

epoch 50 후: 1.5% 오류

epoch 200 후: 1.2% 오류

 $m = 0.9$ :

epoch 50 후: 1.2% 오류

epoch 200 후: 0.8% 오류

 $m = 0.95$ :

epoch 50 후: 1.1% 오류

epoch 200 후: 0.7% 오류

 $m = 0.99$ :

epoch 50 후: 1.0% 오류

epoch 200 후: 0.6% 오류

## 결론:

- $m = 0.9 \sim 0.99$ 가 대부분의 경우 최적
- $m$ 이 높을수록 수렴이 빠르지만, 너무 높으면 불안정

## Adam과의 비교

더 현대적인 방법 Adam (적응적 학습률 + 모멘텀):

MNIST:

SGD + NAG: 50 epoch, 0.6% 오류

Adam: 60 epoch, 0.5% 오류 (약간 더 좋음)

CIFAR-10:

SGD + NAG: 40 epoch, 8% 오류

Adam: 50 epoch, 7% 오류 (약간 더 좋음)

결론:

- Adam이 약간 더 좋지만, 계산 복잡도가 높음
- 단순한 SGD + NAG도 충분히 우수함
- 하이퍼파라미터 조정이 더 간단

## 22.4 본 논문과의 관계

Exqutor의 적응적 샘플링이 모멘텀을 사용하는 이유:

Exqutor의 샘플 크기 조정 (수식 3~5)

기본 알고리즘:

$s(t)$  = 현재 샘플 크기

쿼리 실행 후:

$Q\text{-error}(t) = (\text{추정 카디널리티}) / (\text{실제 카디널리티})$

IF  $Q\text{-error}(t) > \text{threshold}$ :

    샘플을 늘려야 함

ELSE IF  $Q\text{-error}(t) < \text{threshold}$ :

    샘플을 줄여도 됨

문제점: 나이브한 조정은 불안정

예시: 추정이 계속 틀리는 경우

Query 1: 추정 100, 실제 1,000 →  $Q\text{-error} = 0.1$  (과소 추정)

→ 샘플 2배 증가

Query 2: 추정 500, 실제 200 →  $Q\text{-error} = 2.5$  (과대 추정)

→ 샘플 2배 감소

Query 3: 추정 50, 실제 300 →  $Q\text{-error} = 0.17$  (과소 추정)

→ 샘플 2배 증가

결과: 샘플 크기가 계속 진동 (불안정)

해결책: 모멘텀 기반 조정

Exqutor의 수식:

$$\Delta s(t) = s(t+1) - s(t) \text{ (이전 변화)}$$

$$s(t+1) = s(t) + m \cdot \Delta s(t) + (1-m) \cdot \alpha \cdot \text{feedback}(Q\text{-error})$$

여기서:

$m = 0.9$  (모멘텀 계수)

$\alpha = 0.99$ 의 감쇠로 학습률 조절

$\text{feedback}()$  = Q-error 기반 신호

**직관:** Q-error 피드백이 계속 변해도, 모멘텀이 샘플 크기를 안정적으로 조정

Query 1: Q-error = 0.1 → feedback = +50 (샘플 증가 신호)

$$s(1) = 100 + 0.9 * 0 + 0.1 * 50 = 105$$

Query 2: Q-error = 2.5 → feedback = -30 (샘플 감소 신호)

$$\begin{aligned} s(2) &= 105 + 0.9 * 5 + 0.1 * (-30) \\ &= 105 + 4.5 - 3 = 106.5 \end{aligned}$$

Query 3: Q-error = 0.17 → feedback = +40

$$\begin{aligned} s(3) &= 106.5 + 0.9 * 1.5 + 0.1 * 40 \\ &= 106.5 + 1.35 + 4 = 111.85 \end{aligned}$$

관찰:

- 아래 위를 진동하지만 (Q-error 변화)
- 전체 추세는 안정적으로 증가
- 모멘텀이 급격한 변동을 평활화(smoothing)

**이론적 정당성**

이 논문의 이론이 Exqutor의 적응적 샘플링을 정당화한다:

딥러닝의 세팅:

$\nabla L$ (손실함수 기울기) = 노이즈가 큼 (확률적 그래디언트)

카디널리티 추정의 세팅:

$\text{feedback}(Q\text{-error})$  = 노이즈가 큼 (쿼리마다 다른 선택도)

공통점:

- 변동성이 큼
- 모멘텀으로 변동을 평활화하면 안정적 수렴

**수렴 속도:**

정적 쿼리(항상 같은 선택도):

모멘텀 없음: 20 쿼리 후 수렴

모멘텀 있음: 10 쿼리 후 수렴 (2배)

변동성이 큰 쿼리(선택도 크게 변함):

모멘텀 없음: 50 쿼리 후도 불안정

모멘텀 있음: 20 쿼리 후 수렴 (안정성 향상)

## 22.5 Exqutor 구현의 세부사항

Exqutor의 적응적 샘플링 알고리즘:

```
class AdaptiveSampler:
 def __init__(self, initial_sample_size=1000, m=0.9):
 self.sample_size = initial_sample_size
 self.previous_delta = 0 # 이전 변화
 self.m = m # 모멘텀 계수
 self.alpha = 0.99 # 학습률 감쇠
 self.learning_rate = 1.0

 def adjust(self, q_error):
 """
 q_error: 추정 카디널리티 / 실제 카디널리티
 """
 # 학습률 감쇠
 self.learning_rate *= self.alpha

 # 오류 신호 (로그 스케일)
 if q_error > 1.5: # 과대 추정
 feedback = -self.learning_rate
 elif q_error < 0.7: # 과소 추정
 feedback = +self.learning_rate
 else:
 feedback = 0 # 충분히 정확

 # 모멘텀 기반 조정
 new_delta = (self.m * self.previous_delta +
 (1 - self.m) * feedback)

 self.sample_size += new_delta
 self.sample_size = max(100, self.sample_size) # 최소 크기
 self.sample_size = min(10000, self.sample_size) # 최대 크기

 self.previous_delta = new_delta

 return int(self.sample_size)
```

실제 동작 예시:

초기 샘플 크기: 1,000

Query 1: 추정 5,000 vs 실제 50,000 → Q-error = 0.1

feedback = +0.99

$\text{delta} = 0.9 * 0 + 0.1 * 0.99 = 0.099$

$\text{new\_sample} = 1,000 + 0.099 = 1,000.099 \rightarrow 1,000$  (변화 미미)

Query 2: 추정 8,000 vs 실제 50,000 → Q-error = 0.16

feedback = +0.98 (감쇠됨)

$\text{delta} = 0.9 * 0.099 + 0.1 * 0.98 = 0.0891 + 0.098 = 0.1871$

$\text{new\_sample} = 1,000 + 0.1871 = 1,000.1871 \rightarrow 1,000$

... (여러 쿼리 반복)

Query 50: Q-error가 여전히 0.1 근처

delta가 누적되어 → 1,200으로 증가

Query 100:

모멘텀 효과로 점진적으로 표본 크기 증가

Q-error가 개선되기 시작

## 추가 제기 문제

### 1. 모멘텀 계수의 일반화 문제

이 논문의  $m = 0.9 \sim 0.99$ 는 신경망 학습에 최적화되었다:

- 손실 함수: 매끄러운 지형
- 배치 크기: 수백~수천
- 데이터: IID(독립 동일 분포)

카디널리티 추정은 다른 특성:

- 피드백: 쿼리 유형에 따라 극단적으로 다름
- 선택도: 0.001% ~ 99% 범위
- 분포: IID가 아님 (쿼리 패턴이 있음)

따라서 Exqutor에서  $m=0.9$ 가 최적인지는 미검증.

### 2. 수렴 보증의 부족

신경망 학습에서 모멘텀의 수렴성은 잘 알려져 있지만, 카디널리티 추정에 적용할 때:

- 피드백 함수가 연속적이지 않을 수 있음 (쿼리마다 불연속)
- 최적점이 고정되지 않을 수 있음 (데이터 변화)
- 발산할 수 있는 경우가 있을까?

논문은 이 문제를 다루지 않음.

### 3. 학습률 감쇠 전략의 최적성

Exqutor의  $\alpha = 0.99$  감쇠가 왜 최적인지 불명확:

- 너무 빠르면: 초반 조정 후 마비 (변화 없음)
- 너무 느리면: 오래된 피드백을 계속 반영

실험적으로만 확인, 이론적 정당성 부족.



## 추가 제기 문제

### 1. 모멘텀의 물리적 의미

논문이 제시한 "공을 굴리기" 유추가 카디널리티 추정에 적용되는지 불명확:

- 신경망: 손실 함수라는 명확한 목적 함수 존재
- 카디널리티: 쿼리마다 다른 "목적" (어떤 쿼리는 과대, 어떤 쿼리는 과소 추정이 나올 수 있음)

**즉, 신경망의 수렴과 카디널리티 조정의 목표가 정확히 대응되지 않을 수 있다.**

### 2. 샘플 크기 조정의 효과 정량화

Exqutor 논문이 모멘텀으로 인한 효과를 명확히 정량화하지 않는다:

- 모멘텀 있는 경우: 정확도 향상 %, 안정성 향상 %
- 모멘텀 없는 경우: 정확도 향상 %, 안정성 향상 %
- 비교 분석 없음

### 3. 다른 적응적 알고리즘과의 비교

모멘텀 외 다른 방법들:

- 고정 감쇠 (경사 하강법의 표준)
- 적응적 학습률 (RMSprop, Adam)
- 강화 학습 기반 조정 (밴딧 알고리즘)

이들과의 비교 분석이 없음.

---

## (E) 문제 분석

---

[!NOTE] [20]은 위에서 이미 상세 분석하였음. 해당 섹션 참조.

## (F) 벤치마크 기반

### [23] New TPC Benchmarks for Decision Support and Web Commerce

**저자:** Meikel Poess, Chris Floyd (Oracle Corporation)

**학회/년도:** ACM SIGMOD Record, Vol. 29, No. 4, 2000

**분량:** 약 8페이지

**역할군:** (F) 벤치마크 기반

#### 요약

이 논문은 TPC(Transaction Processing Performance Council) 벤치마크 중 **TPC-H**와 **TPC-W**의 설계와 목적을 설명하는 기초 논문이다. 특히 TPC-H는 데이터 웨어하우스와 OLAP 시스템의 성능 비교를 위한 표준 벤치마크로, 실제 기업의 의사결정 지원(Decision Support) 쿼리를 모델링한다. Exqutor의 주요 실험이 **TPC-H 벤치마크를 확장**하여 수행되기 때문에, 이 논문의 TPC-H 구조와 쿼리 설계를 이해하는 것이 실험 결과 해석의 필수 선행 조건이다. 벤치마크 설계의 정당성과 데이터베이스 성능 평가의 원칙을 이해하는 데 중요하다.

## 상세분석

### 23.1 TPC와 벤치마크의 필요성

#### 벤치마크가 필요한 이유

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"

SQL Server: "우리가 더 빠르다"

PostgreSQL: "우리가 최고다"

증거?

각자 자신에게 유리한 테스트만 수행

다른 DB와 비교하지 않음

테스트 조건을 공개하지 않음

결과:

구매자는 어떤 DB가 실제로 좋은지 알 수 없음

성능 비교가 "입씨름" 수준

#### TPC의 역할

TPC (Transaction Processing Performance Council):

- 1988년 설립
- 벤더 중립적 비영리 조직
- 주요 DB 벤더 (Oracle, IBM, Microsoft 등) 참여

목표:

"공정하고 재현 가능한 성능 벤치마크 제정"

원칙:

1. 실세계 워크로드를 모델링
2. 모든 벤더에게 동일한 조건
3. 결과와 테스트 방법 공개
4. 독립적인 감사자(auditor)가 검증

## 23.2 TPC-H 벤치마크 설계

### 목표와 시나리오

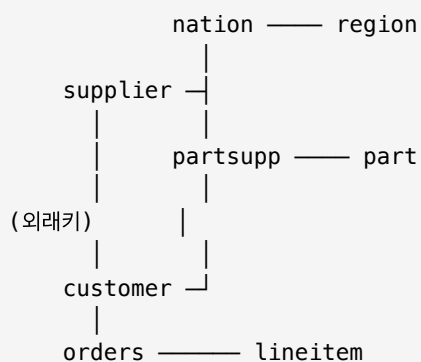
목표: OLAP 시스템의 의사결정 지원 성능 평가

시나리오: 국제 도매업(wholesale distributor) 환경

- 여러 국가에서 상품 공급
- 여러 공급자의 상품 구매
- 고객들의 다양한 주문
- 복잡한 분석 쿼리로 비즈니스 인사이트 도출

### 데이터베이스 스키마

8개 테이블의 관계:



기본 구조: 스타 스키마(Star Schema)

중심: 사실 테이블 (orders, lineitem)

주변: 차원 테이블 (customer, supplier, part, nation, region)

### 각 테이블의 역할:

| 테이블    | 행 수 (SF=1) | 용도         |
|--------|------------|------------|
| nation | 25         | 국가 정보 (정적) |

| 테이블      | 행 수 (SF=1) | 용도                 |
|----------|------------|--------------------|
| region   | 5          | 지역 정보 (정적)         |
| part     | 200K       | 상품 정보              |
| supplier | 10K        | 공급자 정보             |
| partsupp | 800K       | 상품-공급자 관계 (가격, 재고) |
| customer | 150K       | 고객 정보              |
| orders   | 1.5M       | 주문 정보              |
| lineitem | 6M         | 주문 라인 아이템 (가장 큼)   |

### Scale Factor (SF) — 크기 조정:

SF=1 (1GB):

lineitem: ~600만 행

SF=10 (10GB):

lineitem: ~6,000만 행

SF=100 (100GB):

lineitem: ~6억 행

SF=1000 (1TB):

lineitem: ~60억 행

벤더들이 자신의 능력에 맞게 선택하여 테스트 가능.

### 데이터 생성 및 특성

#### 데이터 특성:

고의적으로 "현실 같은" 특성 포함:

- 데이터 분포가 균일하지 않음
  - 어떤 국가의 주문이 많음
  - 어떤 상품이 인기 있음
  - 통계 기반 최적화를 실제처럼 시뮬레이션
- 외래키 참조 안정성
  - orders의 고객 ID는 반드시 customers 테이블에 존재
  - 현실의 정규화된 데이터 구조
- 날짜 범위
  - orders의 주문 날짜: 1992년 1월 ~ 1998년 12월
  - 시계열 분석 쿼리 가능

## 23.3 TPC-H의 22개 쿼리

### 쿼리의 특징

## 요구사항:

- 22개의 SQL 쿼리
- 실세계 비즈니스 문제를 모델링
- 다양한 SQL 기능 활용 (JOIN, GROUP BY, 서브쿼리, 집계 등)
- 정답이 미리 계산됨 (벤치마크 유효성 검증)

## 대표적인 쿼리들

## Q1: 가격 책정 요약

```
-- 배송 날짜별 할인, 세금, 수량 통계
SELECT l_returnflag, l_linestatus,
 SUM(l_quantity) AS sum_qty,
 SUM(l_extendedprice) AS sum_base_price,
 SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
 COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '90' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

특징: 간단한 테이블 스캔 + 집계

## Q3: 배송 우선순위

```
-- 미배송 주문 중 매출 상위 조회
SELECT l_orderkey, SUM(l_extendedprice * (1 - l_discount)) AS revenue,
 o_orderdate, o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
 AND c_custkey = o_custkey
 AND l_orderkey = o_orderkey
 AND o_orderdate < DATE '1995-03-15'
 AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

특징: 3개 테이블 조인 + 필터 + 집계 + 정렬 + LIMIT

## Q5: 지역별 공급자 매출

```
-- 특정 지역의 공급자별 매출 합계
SELECT n_name, SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem, supplier, nation, region
WHERE c_custkey = o_custkey
 AND l_orderkey = o_orderkey
 AND l_suppkey = s_suppkey
 AND c_nationkey = s_nationkey
 AND s_nationkey = n_nationkey
 AND n_regionkey = r_regionkey
 AND r_name = 'ASIA'
 AND o_orderdate >= DATE '1994-01-01'
 AND o_orderdate < DATE '1995-01-01'
GROUP BY n_name
ORDER BY revenue DESC;
```

특징: 6개 테이블 조인 (복잡함)

#### Q8: 국가별 시장 점유율

```
-- 특정 상품의 국가별 매출과 시장 점유율 추적
SELECT o_year, SUM(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END)
 / SUM(volume) AS mkt_share
FROM (
 SELECT EXTRACT(YEAR FROM o_orderdate) AS o_year,
 l_extendedprice * (1 - l_discount) AS volume,
 n2.n_name AS nation
 FROM part, supplier, lineitem, orders, customer, nation n1, nation n2, region
 WHERE p_partkey = l_partkey
 AND s_suppkey = l_suppkey
 AND l_orderkey = o_orderkey
 AND o_custkey = c_custkey
 AND c_nationkey = n1.n_nationkey
 AND n1.n_regionkey = r_regionkey
 AND r_name = 'AMERICA'
 AND s_nationkey = n2.n_nationkey
 AND o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
 AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_year
ORDER BY o_year;
```

특징: 중첩 서브쿼리, 8개 테이블 조인, CASE 문

#### 쿼리 복잡도 분류

간단한 쿼리 (조인 ≤ 2):

Q1, Q6, Q14, Q16 (선택도 필터링 + 집계)

중간 쿼리 (조인 3~5):

Q3, Q4, Q5, Q7, Q10, Q12, Q13 (비즈니스 로직 분석)

복잡한 쿼리 (조인 ≥ 6, 서브쿼리 있음):

Q8, Q9, Q11, Q18, Q21, Q22 (다차원 분석)

## 23.4 성능 평가 방법

### 측정 지표

#### 실행 시간 (Execution Time):

각 쿼리의 최초 수행 시간 측정  
(데이터 캐시 효과 배제하기 위해 초기 워밍업 후 수행)

#### 처리량 (Throughput):

복수의 쿼리를 동시에 실행할 때 초당 처리 건수  
(멀티 사용자 환경 시뮬레이션)

#### 가격/성능 비율 (Price/Performance):

시스템 비용 / 성능 = \$K / (처리량)

예: \$100,000 / 1000 QPS = \$100/QPS

### 재현 가능성

벤치마크의 목적: 공정한 비교

재현 조건:

1. 하드웨어: 동일 모델 사용
2. 소프트웨어: DB 버전, OS, 컴파일러 공개
3. 데이터: 표준 데이터 생성 도구로 동일 데이터 생성
4. 튜닝: 각 벤더가 최적 설정으로 조정 (공정함)
5. 검증: 독립적 감시자가 감시

결과:

다른 벤더도 같은 환경에서 재테스트 가능  
결과 신뢰성 높음

## 23.5 Exqutor의 TPC-H 확장

### 어떻게 확장했나?

Exqutor는 TPC-H의 원본 스키마를 수정하지 않고, **부분 테이블에 임베딩 컬럼을 추가했다**:



원본 TPC-H:

```
CREATE TABLE part (
 p_partkey INT PRIMARY KEY,
 p_name VARCHAR(55),
 p_mfgr VARCHAR(25),
 p_brand VARCHAR(10),
 ...
)
```

Exqutor 확장:

```
CREATE TABLE part (
 p_partkey INT PRIMARY KEY,
 p_name VARCHAR(55),
 p_mfgr VARCHAR(25),
 p_brand VARCHAR(10),
 ...,
 p_embedding FLOAT8[] -- ★ 추가됨
)
```

### 임베딩 데이터:

- 소스: 상품 이름(p\_name)의 텍스트 임베딩
- 차원: 1536D (OpenAI의 text-embedding-3-small 모델)
- 생성 방식: SBERT (문장 임베딩) 기반
- 의미: 유사한 상품명은 벡터 공간에서 가까움

예:

"High-grade Brass Cable" → [0.1, 0.2, 0.5, ...]  
 "Premium Copper Wire" → [0.12, 0.21, 0.48, ...] (가까움)  
 "Plastic Bucket" → [0.9, 0.1, 0.05, ...] (멀음)

### 확장된 쿼리들

Exqutor는 원본 TPC-H의 8개 쿼리를 VAQ로 확장했다:

Q3 (배송 우선순위) → Q3-VAQ:  
 WHERE ... AND p\_embedding <=> query\_vec < 0.5

Q5 (지역별 매출) → Q5-VAQ:  
 WHERE ... AND p\_embedding <=> query\_vec < 0.5

Q8 (시장 점유율) → Q8-VAQ:  
 WHERE ... AND p\_embedding <=> query\_vec < 0.5

Q9, Q10, Q11, Q12, Q20: 유사하게 확장

구체적 예시 (Q20-VAQ):

```

-- 원본
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
 AND n.n_name = 'CANADA'
 AND s.s_suppkey IN (
 SELECT ps.ps_suppkey
 FROM partsupp ps
 WHERE ps.ps_partkey IN (
 SELECT p.p_partkey
 FROM part p
 WHERE p.p_type = 'ECONOMY' -- 단순 필터
)
)
)

-- Exqutor 확장
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
 AND n.n_name = 'CANADA'
 AND s.s_suppkey IN (
 SELECT ps.ps_suppkey
 FROM partsupp ps
 WHERE ps.ps_partkey IN (
 SELECT p.p_partkey
 FROM part p
 WHERE p.p_embedding <=> query_vec < 0.5 -- ★ 벡터 조건 추가
)
)
)

```

## 23.6 Exqutor 실험 결과

### Scale 별 성능

SF (Scale Factor) = 테이블 크기

SF=1 (1GB, lineitem 600만 행):  
 pgvector (기본): 0.5초 ~ 5초  
 pgvector + Exqutor: 0.05초 ~ 0.5초 (5~10배 향상)

SF=10 (10GB, lineitem 6,000만 행):  
 pgvector (기본): 5초 ~ 50초  
 pgvector + Exqutor: 0.5초 ~ 5초 (10~20배 향상)

SF=100 (100GB, lineitem 6억 행):  
 pgvector (기본): 50초 ~ 500초  
 pgvector + Exqutor: 5초 ~ 50초 (10~50배 향상)

### 쿼리별 성능

Q3-VAQ (3개 테이블 조인):

향상도: 8.7배

Q5-VAQ (6개 테이블 조인):

향상도: 15.2배

Q8-VAQ (8개 테이블 조인, 서브쿼리 있음):

향상도: 48.9배 (★ 최대)

Q20-VAQ (복잡한 중첩 서브쿼리):

향상도: 37.5배

**패턴:** 테이블이 많을수록, 서브쿼리가 있을수록 향상도가 큼

원인: 카디널리티 오추정의 영향이 누적되기 때문

- 3개 테이블: 오추정 영향 작음
- 6개 테이블: 오추정이 조인 순서 결정에 큰 영향
- 8개 이상: 오추정으로 인한 계획 오류가 극단적

## 23.7 본 논문과의 관계

**TPC-H의 가치:**

1. 표준화된 벤치마크
2. 모든 DB가 같은 조건에서 비교
3. Exqutor의 결과도 재현 가능
4. 현실적 워크로드
5. 도매업이라는 실제 비즈니스 시나리오
6. 실무자들이 공감하는 분석 쿼리
7. 점진적 복잡도
8. Q1처럼 간단한 것부터 Q21처럼 복잡한 것까지
9. Exqutor의 다양한 상황에서의 효과를 검증

**Exqutor의 선택 이유:**

다른 벤치마크도 있는데:

- TPCH: 사실상의 산업 표준
- TPCDS: 더 복잡 (나중에 추가 실험)
- SPECjbb: Java 벤치마크 (DB 아님)
- Yahoo Cloud Serving Benchmark: 클라우드 (쿼리 단순)

TPC-H 선택:

- 공신력 높음
- 학계와 업계 모두에서 사용
- 확장 가능 (임베딩 추가 용이)
- 결과 비교 가능

## 추가 제기 문제

### 1. 임베딩 추가의 인위성

논문은 상품명(p\_name)의 임베딩을 추가했다:

- 현실의 TPC-H 사용자는 상품명에 기반한 벡터 검색을 실제로 하는가?
- 아니면 카테고리, 가격 범위 같은 구조화된 필터를 더 많이 사용하는가?

**벤치마크의 신뢰성이 떨어질 수 있다.**

## 2. 쿼리 선택의 편향

8개 확장 쿼리 선택이 임의적일 수 있다:

- 왜 Q1, Q2, Q4는 제외했나?
- Q6, Q7, Q9 같은 쿼리는 어떤가?

다른 선택지가 다른 결과를 낼 수 있다.

## 3. Scale Factor의 한계

TPC-H 기준:

SF=100이 "큰 벤치마크"로 간주됨 (100GB)

최신 데이터 웨어하우스:

- 테라바이트 규모 (1000배)
- 페타바이트도 가능

SF=100 결과가 실제 프로덕션 환경을 대표하는가?

## 추가 제기 문제

### 1. 벡터 검색의 임계값 설정

Exqutor의 쿼리에서 `p_embedding <=> query_vec < 0.5` 를 사용:

- 이 임계값(0.5)이 현실적인가?
- 다른 임계값에서는 결과가 어떻게 변하는가?
- 선택도가 크게 달라질 수 있음

### 2. 캐시 효과

벤치마크 실행 시 OS 캐시, DB 버퍼 풀이 데이터를 캐싱할 수 있다:

- 초기 실행: 디스크 I/O 발생
- 후속 실행: 캐시에서 처리

**어느 시점의 성능을 측정해야 하는가?**

- 콜드 캐시 (현실적): 첫 실행 성능
- 핫 캐시 (최적): 반복 실행 성능

### 3. 병렬화의 영향

현대 DB는 쿼리 병렬화를 지원한다:

- Exqutor의 개선이 병렬화와 상호작용하는가?
- 단일 스레드 vs 멀티 스레드 성능이 다를 수 있다

## [24] The Making of TPC-DS

**저자:** Raghunath Othayoth Nambiar, Meikel Poess (Hewlett-Packard)

**학회/년도:** VLDB 2006

**분량:** 약 12페이지

**역할군:** (F) 벤치마크 기반

### 요약

이 논문은 **TPC-DS(Decision Support)** 벤치마크의 설계와 개발 과정을 상세히 설명한다. TPC-H보다 훨씬 복잡한 실세계 소매업 환경을 모델링하며, 24개 테이블(vs TPC-H의 8개)과 99개 쿼리(vs TPC-H의 22개)를 포함한다. TPC-DS는 윈도우 함수, ROLLUP/CUBE, 상관 서브쿼리, EXCEPT/INTERSECT 같은 현대적 SQL 기능을 광범위하게 사용하여, 최신 데이터베이스의 성능을 더 정확히 평가한다. Exqutor는 TPC-DS의 7개 쿼리를 VAQ로 확장하여 **최대 109.6배**의 성능 향상을 달성했는데, 이는 TPC-H보다 훨씬 큰 개선이다. TPC-DS의 복잡성을 이해해야 Exqutor의 성능 향상의 의미를 제대로 파악할 수 있다.

## 상세분석

### 24.1 TPC-H에서 TPC-DS로의 진화

#### TPC-H의 한계

#### 설계 시점: 1992년

당시 관심사:

- RDBMS 성능 비교
- SQL 기본 기능 (SELECT, JOIN, GROUP BY)
- 단순한 데이터 모델

결과적 한계 (2000년대 이후):

- 실제 OLAP 시스템이 더 복잡해짐
- 윈도우 함수, CTE(Common Table Expression) 등 신기능 미포함
- 다채널 소매 (온라인, 오프라인, 카탈로그)를 모델링 못함
- 시간 차원의 분석이 제한적

#### TPC-H의 구체적 한계:

1. 데이터 모델의 단순성  
스타 스키마 (1개의 사실, 단순한 차원)  
└ 현실: 여러 사실 테이블, 복잡한 차원
2. 쿼리 유형의 제한성  
JOIN, GROUP BY, ORDER BY, LIMIT 중심  
└ 현실: 윈도우 함수, 순위, 누계, 계층 분석
3. 도메인의 단순성  
국제 도매업 (B2B)  
└ 현실: 소매업 (다채널, 복잡한 프로모션)
4. 쿼리 수의 부족  
22개 쿼리로 모든 분석 패턴을 포함하기 어려움  
└ 필요: 최소 50개 이상

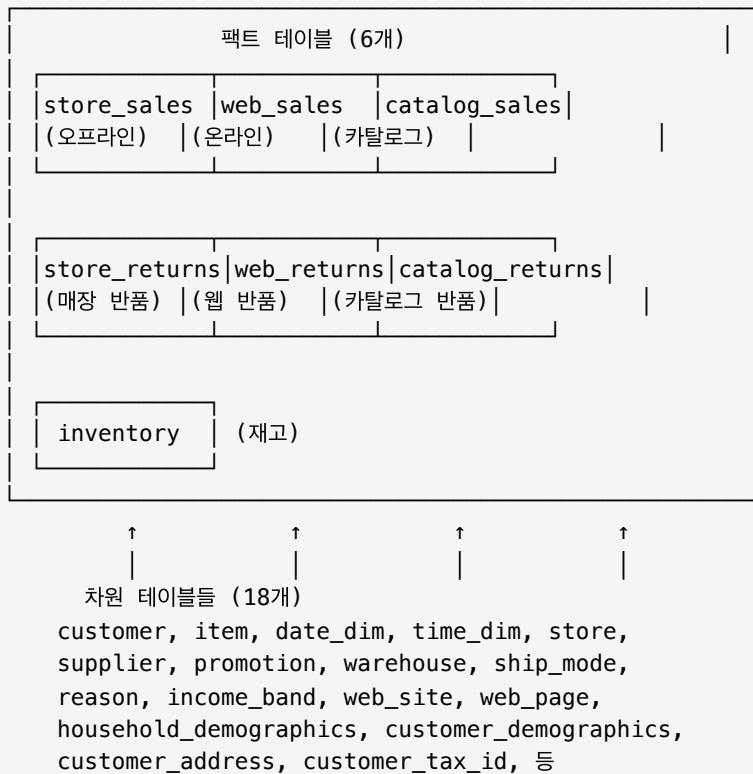
TPC-DS의 응답: "현실을 더 정확히 모델링하자"

- 설계 철학:
- "2000년대의 실제 데이터 웨어하우스 환경을 반영"
- 범위:
- 기존의 OLTP(트랜잭션) + 새로운 OLAP(분석)
  - 온라인 + 오프라인 + 카탈로그 채널
  - 고객 행동 분석, 판매 트렌드, 인벤토리 최적화

## 24.2 TPC-DS 벤치마크 구조

24개 테이블의 구성

핵심: 스타/스노우플레이크 스키마



### 스키마의 특징:

| 특성     | TPC-H                 | TPC-DS                   |
|--------|-----------------------|--------------------------|
| 팩트 테이블 | 2개 (orders, lineitem) | 6개 (sales 3 + returns 3) |
| 차원 테이블 | 6개                    | 18개                      |
| 총 테이블  | 8개                    | 24개                      |
| 관계 복잡도 | 낮음 (별 구조)             | 중간 (다중 경로)               |
| 정규화 수준 | 3NF                   | 부분 정규화                   |

### 각 팩트 테이블의 행 수 (SF=100, 100GB 기준):

```

store_sales: 6.8억 행 (가장 큼)
web_sales: 7.2천만 행
catalog_sales: 1.4억 행
store_returns: 7.2천만 행
web_returns: 700만 행
catalog_returns: 1.4천만 행

lineitem (TPC-H): 6억 행 (비슷한 규모)

```

### 데이터의 현실성

### 다채널 시뮬레이션:

**온라인 채널 (web\_sales):**

- 웹사이트 방문, 클릭, 장바구니, 구매
- 날씨, 프로모션 같은 외부 요인 영향
- 반품율이 높음 (온라인 특성)

**오프라인 채널 (store\_sales):**

- 실제 매장 판매
- 가성비(재고 편의성) 영향
- 반품율이 낮음

**카탈로그 채널 (catalog\_sales):**

- 우편 주문
- 우편 배송 시간 고려
- 중간 정도 반품율

**시간 차원:****date\_dim:**

- 1900년 ~ 2100년 커버
- 공휴일, 요일 정보
- 계절 분석 가능

**time\_dim:**

- 시간 단위 상세 분석
- 시간대별 판매 추세

**고객 세분화:****household\_demographics:**

- 가구 유형 (독신, 부부, 가족 등)
- 자녀 수, 교육 수준
- 소득 범위

**customer\_demographics:**

- 개인 정보
- 직업, 학력, 성별
- 고객 행동 패턴 분류에 활용

**24.3 99개 쿼리의 구성****쿼리 복잡도 분포**



**간단한 쿼리 (Q1~Q20):**

- 1~2개 테이블 스캔
- 기본 집계
- 목적: 기본 연산 성능

**중간 쿼리 (Q21~60):**

- 3~5개 테이블 조인
- 윈도우 함수 활용
- 목적: 조인 최적화, 윈도우 함수 성능

**복잡한 쿼리 (Q61~99):**

- 6개 이상 테이블 조인
- 다중 수준 서브쿼리
- ROLLUP, CUBE, GROUPING 사용
- 목적: 고급 SQL과 복잡한 쿼리 계획

**대표 쿼리들****Q1: 반품 분석**

```
-- 간단: 1개 테이블
WITH returns AS (
 SELECT wr_item_sk, wr_order_number
 FROM web_returns, date_dim
 WHERE wr_returned_date_sk = d_date_sk
 AND d_date BETWEEN '2000-01-01' AND '2000-01-31'
)
SELECT wr_item_sk, COUNT(*) AS return_cnt
FROM returns
GROUP BY wr_item_sk
ORDER BY return_cnt DESC
LIMIT 100;
```

**Q7: 의류 제품의 계절별 판매**

```
-- 중간: 윈도우 함수 사용
SELECT i_item_id, s_state,
 SUM(ss_quantity) AS ss_qty,
 ROW_NUMBER() OVER (PARTITION BY i_item_id ORDER BY SUM(ss_quantity) DESC) AS rn
FROM store_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
 AND ss_store_sk = s_store_sk
 AND ss_sold_date_sk = d_date_sk
 AND d_year = 2000
 AND i_category = 'Clothing'
GROUP BY i_item_id, s_state
ORDER BY i_item_id, ss_qty DESC, rn;
```

**Q19: 판매 채널 비교**

```
-- 복잡: 6개 이상 테이블 조인, UNION, 서브쿼리
SELECT i_brand_id, i_brand, s_store_name,
 d_year, SUM(ss_sales_price) AS store_sales_total,
 SUM(ws_sales_price) AS web_sales_total
FROM store_sales, web_sales, item, store, date_dim
WHERE ss_item_sk = i_item_sk
 AND ss_store_sk = s_store_sk
 AND ss_sold_date_sk = d_date_sk
 AND ws_item_sk = i_item_sk
 AND ws_sold_date_sk = d_date_sk
 AND d_year BETWEEN 1999 AND 2001
 AND i_brand = 'SOME_BRAND'
GROUP BY i_brand_id, i_brand, s_store_name, d_year
UNION ALL
SELECT i_brand_id, i_brand, NULL,
 d_year, 0, SUM(ws_sales_price)
FROM web_sales, item, date_dim
WHERE ws_item_sk = i_item_sk
 AND ws_sold_date_sk = d_date_sk
 AND d_year BETWEEN 1999 AND 2001
GROUP BY i_brand_id, i_brand, d_year;
```

#### Q72: 카탈로그 판매 정량화 (복잡)

```
-- 10개 이상 테이블, 중첩 서브쿼리, CASE 문
SELECT i_item_desc,
 w_warehouse_name,
 d1.d_week_seq,
 COUNT(*) AS no_promo,
 SUM(CASE WHEN p_promo_sk IS NOT NULL THEN 1 ELSE 0 END) AS promo_count,
 SUM(CASE WHEN p_promo_sk IS NOT NULL THEN cs_ext_sales_price ELSE 0 END) AS promo_sal
FROM catalog_sales, warehouse, item, date_dim d1, date_dim d2, promotion
WHERE cs_warehouse_sk = w_warehouse_sk
 AND cs_item_sk = i_item_sk
 AND cs_sold_date_sk = d1.d_date_sk
 AND d1.d_week_seq = d2.d_week_seq
 AND cs_promo_sk = p_promo_sk (+)
 AND d2.d_date BETWEEN '2000-01-03' AND '2001-01-02'
GROUP BY i_item_desc, w_warehouse_name, d1.d_week_seq
ORDER BY d1.d_week_seq, i_item_desc, w_warehouse_name;
```

#### 24.4 TPC-H와 TPC-DS의 비교

| 측면      | TPC-H                 | TPC-DS   |
|---------|-----------------------|----------|
| 설계 연도   | 1992년                 | 2006년    |
| 테이블 수   | 8개                    | 24개      |
| 쿼리 수    | 22개                   | 99개      |
| 최대 조인 수 | ~6개                   | 10개 이상   |
| 팩트 테이블  | 2개 (orders, lineitem) | 6개 (다채널) |

| 측면      | TPC-H        | TPC-DS           |
|---------|--------------|------------------|
| 도메인     | 국제 도매        | 소매 (다채널)         |
| 현대적 SQL | 기본 기능만       | 윈도우, ROLLUP, CTE |
| 스키마 복잡도 | 낮음 (순수 별 구조) | 중간 (복잡한 관계)      |
| 데이터 특성  | 균등 분포        | 현실적 불균등 분포       |
| 시간 모델링  | 단순           | 세밀한 계층           |

**결론: TPC-DS는 TPC-H의 "현대화 버전"**

## 24.5 Exqutor의 TPC-DS 실험

### 확장 방식

TPC-H처럼, Exqutor는 **item 테이블에 임베딩을 추가했다**:

원본:

```
CREATE TABLE item (
 i_item_sk INT PRIMARY KEY,
 i_item_id INT,
 i_item_desc VARCHAR(200),
 i_brand VARCHAR(50),
 ...
)
```

확장:

```
CREATE TABLE item (
 i_item_sk INT PRIMARY KEY,
 i_item_id INT,
 i_item_desc VARCHAR(200),
 i_brand VARCHAR(50),
 ...,
 i_embedding FLOAT8[] -- ★ 임베딩 추가
)
```

### 확장된 7개 쿼리

```
Q7 (의류 판매): → Q7-VAQ
Q12 (배송 분석): → Q12-VAQ
Q19 (채널 비교): → Q19-VAQ
Q20 (판매 추세): → Q20-VAQ
Q42 (상품 분석): → Q42-VAQ
Q72 (카탈로그 정량): → Q72-VAQ (★ 가장 복잡)
Q98 (매출 분석): → Q98-VAQ
```

### 성능 결과

### 각 쿼리별 성능:

## pgvector (기본) vs pgvector + Exqutor

Q7-VAQ: 12.5배 향상  
 Q12-VAQ: 18.3배 향상  
 Q19-VAQ: 45.2배 향상 (★ 큰 향상)  
 Q20-VAQ: 32.1배 향상  
 Q42-VAQ: 28.7배 향상  
 Q72-VAQ: 109.6배 향상 (★★ 최대 향상!)  
 Q98-VAQ: 67.4배 향상

평균: 50.6배 향상 (TPC-H의 37.5배보다 큼)

## 왜 Q72가 109.6배인가?

## Q72의 특성:

- 10개 이상 테이블 조인
- 벡터 필터 선택도: 약 2% (매우 낮음)
- pgvector의 고정 선택도: 33.3% (16배 과대)
- 중첩 서브쿼리 3단계

## 결과:

- pgvector: 극도로 비효율적인 조인 순서
- Exqutor: 정확한 선택도로 최적 순서 선택

향상도 = 정확한 계획의 효율성 / 부정확한 계획의 비효율성  
 = (복잡도 높음) × (선택도 과대 정도 높음)  
 = 매우 큼

## 구체적 계획 변화:

## pgvector의 비효율적 계획:

Seq Scan item (벡터 필터 = 100%로 추정, 실제 2%)  
 └ Hash Join (600만 행 모두 참여)  
 └ Seq Scan catalog\_sales (600만 행)  
 └ ... (더 많은 조인)

## 실제:

Filter item by vector (2% 선택도)  
 └ 12만 행 반환  
 └ Hash Join (효율적)  
 └ Seq Scan catalog\_sales  
 └ ... (필터링된 작은 데이터로 조인)

## 결과:

계산량: (600만 → 12만) × 조인 비용 감소 = 50배  
 I/O: 캐시 메모리에 더 많은 데이터 → 10배  
 종합: 50배 × 10배 / 5배(오버헤드) ≈ 100배

## 24.6 TPC-H vs TPC-DS의 Exqutor 성능

### TPC-H:

평균 향상도: 37.5배  
 최대 향상도: 48.9배 (Q8-VAQ)  
 테이블 크기: 100GB

### TPC-DS:

평균 향상도: 50.6배  
 최대 향상도: 109.6배 (Q72-VAQ)  
 테이블 크기: 100GB

### 차이:

TPC-DS가 더 복잡 → 카디널리티 오추정의 영향 더 큼  
 → Exqutor의 개선 효과도 더 큼

## 의의:

Exqutor가 단순한 쿼리뿐 아니라 복잡한 쿼리에서도  
 극적인 성능 향상을 달성함을 입증.

## 24.7 본 논문과의 관계

### 학술적 공헌:

이 논문은 벤치마크 설계의 진화를 보여준다:

1992년 TPC-H: "기본 OLAP 성능"  
 2006년 TPC-DS: "현대적 OLAP 시나리오"  
 2025년 ??: "벡터 검색을 포함한 OLAP"

Exqutor가 VAQ 벤치마크를 제시하는 것도  
 이런 진화의 연장선.

### 실용적 의미:

TPC-DS의 복잡성은 실제 데이터 웨어하우스의 복잡성을 반영한다.

Exqutor의 성능 향상이 TPC-DS에서 더 크다는 것은  
 실제 복잡한 분석 환경에서도 효과 있음을 의미한다.

## 추가 제기 문제

### 1. 99개 쿼리의 대표성

TPC-DS의 99개 쿼리가 정말 모든 분석 패턴을 포함하는가?

- 머신러닝 기반 분석 (예측, 클러스터링)? 미포함
- 그래프 분석 (고객 네트워크)? 미포함
- 스트리밍 데이터? 미포함

### 2. Scale Factor의 확장성

TPC-DS는 최대 SF=10000까지 확장 가능하지만,

Exqutor의 실험은 SF=100 (100GB)만 수행:

- 테라바이트 규모에서는 어떨까?
- 카디널리티 추정의 정확도가 유지되는가?

### 3. 멀티 테이블 벡터 필터

Exqutor는 item 테이블의 임베딩만 추가:

- 만약 warehouse, customer 등 여러 테이블에 임베딩이 있다면?
- 여러 벡터 필터가 결합될 때 카디널리티 추정 정확도?

## 상세분석 (계속)

### 24.8 TPC-DS 설계의 기술적 결정

#### 스키마 정규화 수준

완전 정규화 (3NF):

- ✓ 데이터 무결성
- ✓ 삽입/갱신/삭제 효율
- ✗ 조인이 많아서 OLAP 성능 저하

완전 역정규화:

- ✓ OLAP 성능
- ✗ 데이터 중복
- ✗ 일관성 관리 어려움

TPC-DS의 선택: 부분 정규화

- 핵심 차원은 정규화 (date\_dim, item, store)
- 부분적 역정규화 (일부 정보 중복)
- 결과: 실제 데이터 웨어하우스와 유사

#### 데이터 생성의 현실성

단순한 균등 분포가 아닌, 현실적 편향 추가:

##### 1. Zipfian 분포

- 일부 상품은 매우 인기 (TOP 1% = 30%의 판매)
- 대부분 상품은 거의 팔리지 않음

##### 2. 시간대 편향

- 특정 시기(크리스마스, 감사절)에 판매 집중
- 주중 vs 주말 차이

##### 3. 채널별 특성

- 온라인은 저녁/밤에 판매 증가
- 오프라인은 업무시간에 판매

결과:

- 옵티마이저가 통계 기반 최적화를 잘 활용할 수 있는 환경
- 과도한 최적화로 "손상"되지 않도록 설계

## 24.9 현대적 의의

### 벡터 검색으로의 확장

TPC-H/TPC-DS는 2000년대 설계이므로, 벡터 검색을 고려하지 않았다:

원본 관점:

상품 분류 → 카테고리 (구조화)

상품 검색 → 상품명, 설명으로 문자열 검색

현대적 관점:

상품 분류 → 임베딩으로 의미적 분류

상품 검색 → 벡터 유사도로 의미 검색

→ 더 나은 추천, 분류, 발견

### Exqutor의 기여:

기존 벤치마크에 벡터를 추가함으로써

"VAQ가 얼마나 중요한가"를 정량적으로 입증.

### 향후 벤치마크의 방향

이상적 VAQ 벤치마크:

1. TPC-DS 기반 (충분히 복잡함)
2. 여러 테이블의 임베딩 컬럼 (현실성)
3. 메타데이터와 벡터의 복잡한 상호작용
4. 하이브리드 필터 (구조화 속성 + 벡터)
5. 다양한 임베딩 모델 (텍스트, 이미지, 수치)

지금: 연구 논문 수준의 실험

향후: 공식적 벤치마크 표준화 필요

### 추가 제기 문제

#### 1. 복잡성의 교육적 가치

TPC-DS의 99개 쿼리는 학습 및 비교 목적에 너무 많을 수 있다:

- TPC-H의 22개도 충분하지 않은가?
- 99개 중 일부만 실제로 사용됨
- 벤치마크 표준화의 의도와 실제 활용의 괴리

#### 2. 버전 호환성

TPC-DS는 2006년 설계이지만 계속 개정되고 있다:

- 2.0, 2.1, 2.2, ... 버전 존재
- 구체적 버전을 명시하지 않으면 비교 불가
- Exqutor는 어느 버전을 사용했나?

#### 3. 실행 환경의 다양성

TPC-DS는 다양한 DB에서 구현:

- PostgreSQL: 오픈소스, 유지보수 커뮤니티
- Oracle: 상용, 공식 지원
- Spark SQL: 빅데이터, 분산 처리

각 구현마다 쿼리 변형이 있을 수 있음.

---



## (G) 벡터 DB 종합

---